

Универсальное пособие • полный исходный код

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 14.09.2026, 16:56:01
Файлов исходного кода: 96
Суммарный объём кода: 1.42 MB
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs-bfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler-cycle]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

.....

- 24. src/data/vizStepBus.ts

- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_5.ts

- 27. src/data/tickets/ticket14_20.ts

- 28. src/data/tickets/ticket21_24.ts

- 29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 30. src/components/ArticulationPointsViz.tsx

- 31. src/components/BellmanFordViz.tsx

- 32. src/components/BfsViz.tsx

- 33. src/components/ChapterImage.tsx

- 34. src/components/ChapterNav.tsx

- 35. src/components/ChapterView.tsx

- 36. src/components/ColoringViz.tsx

- 37. src/components/ComplexityClassesViz.tsx

- 38. src/components/ComponentsViz.tsx

- 39. src/components/DfsBridgesSimulator.tsx

- 40. src/components/DijkstraViz.tsx

- 41. src/components/DPVisualizer.tsx

- 42. src/components/EulerSimulator.tsx

- 43. src/components/ExpressQuiz.tsx

- 44. src/components/FloydViz.tsx

- 45. src/components/GraphTraversalViz.tsx

- 46. src/components/HeapViz.tsx

- 47. src/components/hints/demoKit.tsx

- 48. src/components/hints/demosExtra.tsx

- 49. src/components/hints/demosGraph.tsx

- 50. src/components/hints/demosStruct.tsx

- 51. src/components/hints/MiniDemo.tsx

- 52. src/components/hints/SimulatorModal.tsx

- 53. src/components/hints/TermPopover.tsx

- 54. src/components/JohnsonViz.tsx

- 55. src/components/KosarajuViz.tsx

- 56. src/components/KruskalSimulator.tsx

- 57. src/components/MnemonicCards.tsx

Универсальное пособие • полный исходный код

```
.....
88. scripts/export/common.mjs
89. scripts/export/guide-pdf-entry.tsx
90. scripts/export/render-pages.mjs
91. scripts/selftest/bundle.ts
92. scripts/selftest/harness-dump.ts
93. scripts/selftest/harness-entry.ts
94. scripts/selftest/lazy.ts
```

CI / автоматизация

```
95. .github/workflows/deploy-pages.yml
96. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

.....
ФАЙЛ 1/96: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

.....
Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

` .github/workflows/deploy-pages.yml`

Универсальное пособие • полный исходный код

```
.....
  run: npm ci --no-audit --no-fund

- name: Build for the repository subpath
  env:
    VITE_BASE: /algv0/
  run: npm run build

- name: Make GitHub Pages fallback
  run: cp dist/index.html dist/404.html

- name: Upload Pages artifact
  uses: actions/upload-pages-artifact@v5
  with:
    path: ./dist

deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
```





## Универсальное пособие • полный исходный код

```
.....
 fi
 git commit -m "chore(export): пересборка full"
 git push origin "HEAD:${GITHUB_REF_NAME}"

Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdcb3c3..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

    steps:
      - name: Checkout
        uses: actions/checkout@v4
+       uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v4
+       uses: actions/setup-node@v7
        with:
          node-version: "22"
+          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
        uses: actions/upload-artifact@v4
+       uses: actions/upload-artifact@v7
        with:
```

Универсальное пособие • полный исходный код

```
-----
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
    <head>
      <meta charset="UTF-8" />
      <meta name="viewport" content="width=device-width,
+    <link rel="icon" type="image/svg+xml" href="%BASE_
      <title> Дискретка для тупых – Интерактивный гид по
    </head>
    <body class="bg-slate-950 text-slate-100 min-h-screen
....
```

`public/favicon.svg`

Полное содержимое после изменений

```
````xml
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
````
```

Patch относительно `main`

```
````diff
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```



```

.....
const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
};
window.addEventListener("keydown", onKey);
return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />
 </>
) : (
 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId} />
 </div>
 <div className="flex-1">
 <main id="main-content" className="flex-1 min-h-[600px]">
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">

```

```

 </main>
) : (
 <PythonCompiler />
)}

 <SimulatorModal vizId={modalVizId} onClose={() =>

 <footer className="p-8 text-center text-slate-600"
 © 2026 Universal Educational Guide. Интерактивный
 </footer>
 </div>
);
}
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("1");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
  </div>
</>

```

```

    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
            <Sparkles className="w-6 h-6" />
          </div>
          <div className="min-w-0 flex-1">
            <h1 className="text-xl font-extrabold text-white">
              Универсальное пособие
            </h1>
            <p className="text-xs text-indigo-400 font-medium">
              Подготовка к экзаменам: Алгоритмы, Структуры данных
            </p>
          </div>
        </div>
      </div>

      <div className="flex items-center w-full lg:w-auto">
        <button
          id="tab-guide-btn"
          onClick={() => setActiveTab('guide')}
          className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
            activeTab === 'guide'
              ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
              : 'text-slate-400 hover:text-white'
          }`}
        >
          <BookOpen className="w-4 h-4" /> Учебное пособие
        </button>
      </div>
    </header>
  );
}

```

Универсальное пособие • полный исходный код

```
.....
      href={` ${import.meta.env.BASE_URL}export/full
      download="full_code_all_pages.txt"
      target="_blank"
      rel="noreferrer"
      className="flex items-center justify-center
      :bg-sky-600/20 border border-sky-500/30 tra
      title="Скачать полный TXT файл со всем кодом
    >
      <FileText className="w-4 h-4" /> TXT
    </a>
    <button
      id="download-html-btn"
      onClick={saveBook}
      disabled={busy}
      className="flex items-center justify-center
      er:bg-amber-600/20 border border-amber-500/
      title="Скачать всё пособие одним автономным
    >
      {busy ? <Loader2 className="w-4 h-4 animate
    </button>
  </div>
</div>
</header>
);
};
....
```

Patch относительно `main`

```
```diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
 -import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 +import { BookOpen, Cpu, Sparkles, FileDown, FileText,
```

Универсальное пособие • полный исходный код

```
.....
- href="/export/full_code_all_pages.txt"
+ href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.txt`
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
.....
```

## `src/components/PythonCompiler.tsx`

### Полное содержимое после изменений

```
````tsx  
import { useCallback, useState } from "react";  
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";  
  
const PYODIDE_VERSION = "0.27.7";  
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;  
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;  
  
const STARTER_CODE = `# Первый запуск загрузит Python в интерпретатор  
name = "алгоритмы"  
  
for number in range(1, 4):  
    print(f"{number}. Учим {name}!")  
`;  
  
type PyodideRuntime = {  
  runPythonAsync: (source: string) => Promise<unknown>;  
  setStdout: (options: { batched: (message: string) => void }) => void;  
  setStderr: (options: { batched: (message: string) => void }) => void;  
  setStdin: (options: { stdin: () => string | number | null }) => void;  
};  
  
type PyodideModule = {  
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;  
};  
  
let runtimePromise: Promise<PyodideRuntime> | null = null;
```

```

.....

runtime.setStdout({ batched: (message) => stdout,
runtime.setStderr({ batched: (message) => stderr,
runtime.setStdin({ stdin: () => (inputLines.length
await runtime.runPythonAsync(code);

const text = [...stdout, ...stderr].join("");
setOutput(text || "Готово: программа ничего не выв
} catch (error) {
const captured = [...stdout, ...stderr].join("");
setOutput(`${captured}${captured && !captured.end
setRuntimeMessage("Не удалось выполнить код – про
} finally {
setRunning(false);
}
}, [code, stdin, running, runtimeReady]);

const reset = () => {
setCode(STARTER_CODE);
setStdin("");
setOutput("Нажмите «Запустить», чтобы увидеть резул
});

return (
<main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
  <div className="max-w-6xl mx-auto">
    <div className="flex flex-col sm:flex-row sm:items-center">
      <div>
        <div className="inline-flex items-center gap-2">
          <Terminal className="w-4 h-4" /> Боковая панель
        </div>
        <h2 className="text-2xl sm:text-3xl font-extrabold">
          <p className="text-slate-400 mt-2 max-w-2xl">
            Пишите небольшой код и запускайте его прямо в браузере,
            который переводит CPython в WebAssembly.
          </p>
        </div>
      <a href="#">

```

```

<textarea
  value={code}
  onChange={(event) => setCode(event.target.value)}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) &&
      event.preventDefault();
      void runCode());
  }}
  spellCheck={false}
  aria-label="Код Python"
  className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-emerald-500"
  placeholder="Напишите Python-код..."
/>

<div className="border-t border-slate-800 border-b border-slate-800" style={{padding: 10px}}>
  <label className="block px-4 pt-3 text-slate-500" style={{display: block}}>
    Ввод для input() • по одной строке на каждую строку
  </label>
  <textarea
    id="python-stdin"
    value={stdin}
    onChange={(event) => setStdin(event.target.value)}
    spellCheck={false}
    rows={2}
    placeholder={'Например: \nАлиса'}
    className="block w-full resize-y bg-transparent border border-slate-800"
    style={{padding: 5px}}
  </textarea>
</div>
</section>

<section className="rounded-2xl border border-slate-800" style={{padding: 10px}}>
  <div className="flex items-center justify-between" style={{padding: 5px 0}}>
    <div className="flex items-center gap-2" style={{padding: 0 10px}}>
      <Terminal className="w-4 h-4 text-emerald-500" style={{border: 1px solid transparent, border-radius: 5px, padding: 2px 5px}}>

```

Универсальное пособие • полный исходный код

```
.....
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>;
+  setStdout: (options: { batched: (message: string) => void }) => void;
+  setStderr: (options: { batched: (message: string) => void }) => void;
+  setStdin: (options: { stdin: () => string | number | null }) => void;
+};
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {
+      (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL })
+    });
+  }
+}
```


Универсальное пособие • полный исходный код

```
.....
+     setOutput(text || "Готово: программа ничего не вывела");
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("");
+     setOutput(`${captured}${captured && !captured.endsWith("\n")}\n`);
+     setRuntimeMessage("Не удалось выполнить код – произошла ошибка");
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть результат");
+ };
+
+ return (
+   <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:items-center">
+         <div>
+           <div className="inline-flex items-center gap-2">
+             <Terminal className="w-4 h-4" /> Боковая панель
+           </div>
+           <h2 className="text-2xl sm:text-3xl font-extrabold">
+             <p className="text-slate-400 mt-2 max-w-2xl">
+               Пишите небольшой код и запускайте его прямо в браузере,
+               который переводит CPython в WebAssembly.
+             </p>
+           </div>
+           <a
+             href="https://pyodide.org/"
+             target="_blank"
+             rel="noreferrer"
+             className="inline-flex items-center gap-1.5"
+           >
+             Как это работает <ExternalLink className="text-slate-400" />
+           </a>
+         </div>
+       </div>
+     </div>
+   </main>
+ );
```

```

+         void runCode();
+     }
+ }
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+   <label className="block px-4 pt-3 text-[
+     Ввод для input() • по одной строке на
+   </label>
+   <textarea
+     id="python-stdin"
+     value={stdin}
+     onChange={(event) => setStdin(event.ta
+     spellCheck={false}
+     rows={2}
+     placeholder={'Например:\nАлиса'}
+     className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+   />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+   <div className="flex items-center justify-i
+     <div className="flex items-center gap-2
+       <Terminal className="w-4 h-4 text-emera
+     </div>
+     <span className={`inline-flex items-centr
+       {runtimeReady && <CheckCircle2 classNam
+       {runtimeReady ? "готов" : "не загружен
+     </span>
+   </div>
+   <pre className="min-h-[360px] max-h-[560px]

```

Универсальное пособие • полный исходный код

```
.....

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локально
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
....
```

Patch относительно `main`

```
````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
```

Универсальное пособие • полный исходный код

.....  
# Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или в Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу:  
\*\*строго\*\* придерживаться готовых шаблонов разметки, не добавляя новых.

## Правила для малых страниц (Билетов / Глав)

1. **\*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML.
2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть в формате `{ "id": "...", "content": ... }` для вставки в `src/data/content.ts`.
3. **\*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, `bg-indigo-500`).
  - Определение/правило в центре (`border-blue-500` или `border-purple-500`).
  - Обязательная сетка аналогий (2 колонки: неформальный язык / строгий язык).
  - "Душные" детали (код, формулы, сложные доказательства).
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы из файлов в папках `src/data/content.ts`. Не используй чистый markdown.

.....  
ФАЙЛ 4/96: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 байт

.....  
<!doctype html>  
<html lang="ru" class="dark bg-slate-950 text-slate-100">  
 <head>  
 <meta charset="UTF-8" />  
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />  
 <link rel="icon" type="image/svg+xml" href="%BASE\_URL%/icon.svg" />  
 <title> Универсальное пособие по алгоритмам – билет №4  
 </head>  
 <body class="bg-slate-950 text-slate-100 min-h-screen">  
 <div id="root"></div>  
 <script type="module" src="/src/main.tsx"></script>  
 </body>  
</html>

```
.....
 "selftest": "npx esbuild scripts/selftest/bundle.ts
 el=error --loader:.jpg=file && node /tmp/bundle-s
 at=cjs --outfile=/tmp/lazy-selftest.cjs --jsx=auto
 esbuild scripts/selftest/harness-entry.ts --bundl
 on3 scripts/selftest/harness_check.py"
},
"dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 "svg-to-pdfkit": "^0.1.8",
 "tailwind-merge": "3.4.0"
},
"devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.24.0",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и
}
```

## Универсальное пособие • полный исходный код

### ### Автоматизация (CI)

`.github/workflows/rebuild-pdf.yml` запускается \*\*на каждой коммита\*\* и пересобирает TXT → PNG → PDF, кладёт PNG-страницы и документы в репозиторий и коммитит обновлённые `public/export/full\_code\_all\_pages.pdf` и `public/export/full\_code\_all\_pages.png` (с меткой `{skip ci}`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code\_all\_pages.pdf` и `/export/full\_code\_all\_pages.png`.

### ## Аудит контента

```
```bash
node scripts/audit-coverage.mjs          # какие темы без кода
node scripts/audit-coverage.mjs --md     # markdown-таблицы
```
```

Актуальный отчёт: [ `TOPICS\_AUDIT.md` ](./TOPICS\_AUDIT.md)

### ## Как устроен интерфейс

- \* **Темы 1–24** — канонический список билетов; одна страница на тему (`src/data/content.ts`, таблица `CANON` задаёт точные названия).
- \* **Python-компилятор** — боковая панель (кнопка «Python-компилятор», отладчик. Список «переменных темы» для каждой страницы в `src/data/vizSync.ts`; значения из выполняемого Python-кода — объекты демонстрации (матрицы, графы, деревья) напрямую в код, инициализация подставляет заранее заданные значения).
- \* **Содержание** — на десктопе боковая панель с поиском, на телефоне — липкая строка «Содержание · N из M» и вкладки «Содержание», «Словарь» и «Регистр».
- \* **Переходы между темами** — кнопки «Назад / Вперёд» в боковой панели, «Предыдущая / Следующая тема» под текстом; на клавиатуре — «←» и «→».
- \* **Всплывающие подсказки по терминам** — текст главы автоматически подсвечивает известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта термина мини-анимацией и кнопкой «Показать симулятор» (симулятор термина — `src/data/glossary.ts`, мини-анимации — `src/components/vizRegistry`).
- \* **Регистр интерактивов** — `src/components/vizRegistry`.

Универсальное пособие • полный исходный код

.....

Вместо обычных `` в приложении используются векторы

\* Они помещены внутрь оберток с классом `

\* Все отрисовки находятся внутри векторного тега ``

\* Подпись к картинке, если она есть, идет сразу после SVG

## Списки

\* Стандартные списки оформлены через `



## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `

## ` и `` заменяются на `##` и `###`

2. Элементы ``.analogy-badge` получают выделение маркера`

3. Теги `` заменяются строкой-заглушкой `[ Иллюстрация (картинка)]`.

4. Элементы списка `- ` получают маркер `-`.

5. Результат извлекается в виде текста и чистится от лишнего

.....

ФАЙЛ 8/96: TOPICS\_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

.....

# Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.m`

> Критерии:

> \* **Аналогия** — в HTML главы есть блок «Аналогия ...»

> \* **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

...

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |

| --- | --- | --- |

## Универсальное пособие • полный исходный код

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации (

> Замкнуто: «5. Двумерная матрица префиксных сумм» полн

|  |                                      |    |                    |                 |                     |
|--|--------------------------------------|----|--------------------|-----------------|---------------------|
|  | Глава                                | id |                    | Чего не хватает |                     |
|  | ---                                  |    | ---                |                 | ---                 |
|  | 3. Декартово дерево (Tgear)          |    | `tgear`            |                 | Нет ни визуализации |
|  | 8. Графы. Компоненты связности       |    | `graph-components` |                 |                     |
|  | 24. Классы сложности, сведение задач |    | `complexity-cl     |                 |                     |

## Сводная таблица по всем главам

|     |                                                 |          |         |     |          |   |
|-----|-------------------------------------------------|----------|---------|-----|----------|---|
| #   | Глава                                           | Аналогий | 2D-виз. | SVG | Картинка |   |
| --- | ---                                             | ---      | ---     | --- | ---      |   |
| 2   | 2. Двумерная разреженная матрица (Sparse Table) |          |         |     |          |   |
| 3   | 3. Декартово дерево (Tgear)                     | 2        | -       | -   | -        |   |
| 4   | 4. Splay-дерево                                 | 2        |         | -   | -        |   |
| 5   | 5. Двумерная матрица префиксных сумм            | 1        |         |     | -        |   |
| -   | Планарность: K5, K3,3 и формула Эйлера          | -        |         |     | -        |   |
| 6   | 6. Графы. Представление, DFS, BFS               | 4        |         |     | -        | - |
| 8   | 8. Графы. Компоненты связности                  | 1        | -       | -   | -        |   |
| 9   | 9. Графы. Топологическая сортировка             | 1        |         |     | -        |   |
| 10  | 10. Графы. Компоненты сильной связности (SCC)   |          |         |     | -        |   |
| 12  | 12. Графы. Точки сочленения                     | 1        |         |     | -        | - |
| 14  | 14. Графы. Кратчайший путь. Дейкстра            | 3        |         |     | -        |   |
| 15  | 15. Графы. Кратчайший путь. Форд-Беллман        | 1        |         |     | -        |   |
| 16  | 16. Графы. Кратчайший путь. Флойд               | -        |         |     | -        | - |
| 17  | 17. Графы. Алгоритм Джонсона                    | -        |         | -   | -        |   |
| 18  | 18. Графы. Остовное дерево. Краскал             | 1        |         |     | -        |   |
| 19  | 19. Графы. Остовное дерево. Прима               | 1        |         |     | -        |   |
| 20  | 20. Графы. Остовное дерево. Борувка             | 1        |         |     | -        |   |
| 21  | 21. Префикс-функция ( $\pi$ ), КМП              | 4        |         | -   | -        |   |



```
.....
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

.....  
ФАЙЛ 10/96: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 36 | РАЗМЕР: 1

```
.....
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локаль
```

## Универсальное пособие • полный исходный код

```
.....
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/Simu
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeChapterId, setActiveChapterId] = useState<string | null>();
 const [modalVizId, setModalVizId] = useState<string | null>();
 /** Боковая (на мобильном – нижняя) панель компилятора */
 const [compilerOpen, setCompilerOpen] = useState(false);
 /** Ширина панели компилятора (desktop), растягивается */
 const [compilerWidth, setCompilerWidth] = useState<number>(0);
 const cached = typeof window !== "undefined" ? Number(window.localStorage.getItem("compilerWidth")) : 0;
 return Number.isFinite(cached) && cached >= 380 ? cached : 380;
});
useEffect(() => {
 window.localStorage.setItem("compilerWidth", String(compilerWidth));
}, [compilerWidth]);
/** Высота терминала: null оставляет адаптивную высоту */
const [compilerHeight, setCompilerHeight] = useState<number>(0);
const cached = typeof window !== "undefined" ? Number(window.localStorage.getItem("compilerHeight")) : 0;
return Number.isFinite(cached) && cached >= 260 ? cached : 260;
});
useEffect(() => {
 if (compilerHeight !== null) window.localStorage.setItem("compilerHeight", String(compilerHeight));
}, [compilerHeight]);
/** Содержание свернуто (по умолчанию сворачиваем sidebar) */
const [sidebarCollapsed, setSidebarCollapsed] = useState<boolean>(true);
const cached = typeof window !== "undefined" ? window.localStorage.getItem("sidebarCollapsed") : null;
if (cached !== null) return cached === "1";
return typeof window !== "undefined" && window.innerWidth < 380;
});
const toggleSidebar = useCallback(() => {
 setSidebarCollapsed((c) => {
 window.localStorage.setItem("sidebarCollapsed", c ? "1" : null);
 return !c;
 });
});
```

```

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLInputElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen(!compilerOpen)}
 />

 <div ref={contentRef} className={compilerOpen ? "h-screen" : "h-[calc(100vh-50px)]"}>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row min-h-[calc(100vh-100px)]">
 <div
 className={`hidden lg:block shrink-0 border-right border-slate-800 p-4 transition-all duration-200 ${sidebarCollapsed ? "hidden" : ""}`}
 >

```

```

.....
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 onOpenCompiler={openCompilerWithViz}
 />
</main>
</div>
</>

<footer className="p-8 text-center text-slate-600">
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
 {compilerOpen && {
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 width={compilerWidth}
 height={compilerHeight}
 onWidthChange={setCompilerWidth}
 onHeightChange={setCompilerHeight}
 onOpenGuide={() => {
 document.getElementById("chapter-viz")?.scrollTo(0,0)
 }}
 onClose={() => setCompilerOpen(false)}
 />
 }}
</div>
});
}

```

```

.....
*/
export function annotateTerms(root: HTMLElement): void {
 if (!cachedRegex) cachedRegex = buildRegex();
 const re = cachedRegex;

 const perTerm = new Map<string, number>();
 const perSurface = new Map<string, number>();
 const walker = document.createTreeWalker(root, NodeFilter.
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
 if (!node.nodeValue || node.nodeValue.trim().length === 0)
 return NodeFilter.FILTER_REJECT;
 return NodeFilter.FILTER_ACCEPT;
 },
 true);

 const targets: Text[] = [];
 let current = walker.nextNode();
 while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
 }

 for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

```

```
.....
 } catch {
 /* если браузер не умеет lookbehind – просто оста
 }
 };

 // при смене главы
 // eslint-disable-next-line react-hooks/exhaustive-dep
 useEffect(run, deps);

 // страховка: контент перерисовали, а подсказок в нём
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
 });
}
```

.....  
ФАЙЛ 13/96: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
.....

@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
}
```

```
.....
 transform: translateY(4px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
}

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {
```

```
.....
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
```



Универсальное пособие • полный исходный код

```
}
.....
```

```
.....
ФАЙЛ 16/96: src/vite-env.d.ts
```

```
КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В
.....
```

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

```
.....
РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ
.....
```

```
.....
ФАЙЛ 17/96: src/data/additionalTickets.ts
```

```
КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 902 | РАЗМЕР: 1000 В
.....
```

```
import { Chapter } from "../types";
```

```

 </div>
 <p class="text-slate-300 text-sm mb-5">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
 </div>

 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-amber-400"
 <p class="text-slate-300 text-sm mb-5">Част
 а ту же скорость». <b class="text-rose-300"
 олько для суммы, разреженная — огромный
 посчитают клетки дважды), минимум нельзя ч
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-5 rounded-lg"
 <div class="absolute -top-3 left-4
 border border-emerald-500 shadow-md">
 Сетка из вафельных трубочек —
 </div>

 имается по включениям-исключениям — как
 <p class="text-slate-300 text-sm">И

```

```

.....
 <div class="flex flex-col items-center justify-center gap-1">
 <div class="border border-rose-400/60 text-slate-900 text-sm font-bold shadow-sm">3</div>
 <div class="w-8 h-8 border-rose-400/60 text-slate-900 text-[11px] sm:text-sm font-bold shadow-sm rounded bg-amber-100/90 border border-rose-400/60">62</div>
 <div class="w-8 h-8 sm:w-9 sm:h-9 border-rose-400/60 text-slate-900 text-[11px] sm:text-xs font-bold shadow-sm rounded bg-amber-100/90 border border-rose-400/60">
 <div class="text-[9px] sm:text-sm font-mono">
 <div class="text-slate-300 text-sm mb-2">
 <p class="text-slate-300 text-sm mb-2">
 емо 1D выше (массив 2, 3, 5, 62, 3, 21, 1, ...).
 длины 2^j в массиве из 8 элементов равно 8-2^j.
 его 21 против 8 в исходном массиве. В 2D упрощении
 з больше исходной матрицы.</p>
 <p class="text-slate-300 text-sm">
 2 - 2 = 0 – прибавил кусок, вычитая
 min(2, 2) = 2 – а сколько
 ть» – его печём заранее и целиком.</p>
 </div>
 </div>
 <div class="overflow-x-auto mt-6">
 <table class="w-full text-sm text-slate-900">
 <thead>
 <tr class="text-left text-[11px] sm:text-sm">
 <th class="py-2 pr-4 font-bold">
 <th class="py-2 pr-4 font-bold">
 <th class="py-2 font-bold">
 </th>
 </tr>
 </thead>
 <tbody class="align-top">

```

```

</div>
</section>',
 }, {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 3
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">Декартово дерево (Treap)</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">Описание</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Декартово дерево (Treap) – это бинарное дерево поиска, в котором каждый узел содержит ключ и приоритет. Оно сочетает в себе свойства бинарного дерева поиска и кучи. В Treap узлы упорядочены по ключу (левый узел имеет меньший ключ, чем правый), а также по приоритету (корень имеет максимальный приоритет). Приоритет обычно выбирается случайным образом, что гарантирует сбалансированность дерева. В результате операции поиска, вставки и удаления выполняются за $O(\log n)$.</p>
 <p class="text-xl font-mono text-white">Пара (ключ, приоритет) называется двоичная куча. Пара – точка на декартовой координатной плоскости.</p>
 </div>
 <p class="text-slate-300 text-sm mb-4">Построение Treap выполняется за $O(n \log n)$ сравнений, counting по максимуму. Каждая точка вставляется спуском от корня: $x \leq \text{узлов}$ – левый, $x > \text{узлов}$ – правый. Единственность. Весь процесс – в симуляторе под названием «соединить сам» – кликаешь две точки, получаешь «родитель», «получился бы цикл», «равные», «Миф $2k/2k+1$ » и «Поиск ≠ сортировка». Каждый запрещённый ход.</p>
 <div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">Симулятор</p>
 <p class="my-1 font-bold leading-none">И
 И
 </p>
 </div>
 </div>
 </div>
`
 },

```

```

.....
 text-xs font-mono text-rose-300 border border-rose-300 rounded
ed text-xs font-mono text-rose-300 border border-rose-300 rounded

<p class="text-slate-500 text-xs mt-3 leading-5">
и останется набор букв, как treap без у остальных,
но кричат по «!»: корень самый громкий, родителю
; равные x — идут влево по уставу.</p>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Код: split (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Код: вставка, merge, erase (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
def insert(root, key, pri):

```

```
.....
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ротов поднимает x в корень. Единственный ин
 </div>

 <div class="grid grid-cols-1 md:grid-cols-3">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">мож
 <p class="text-slate-400 text-xs mt-2">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-emerald-300 font-bold">
 <p class="text-slate-400 text-xs mt-2">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">O(n
 <p class="text-slate-400 text-xs mt-2">
 </div>
 </div>

 <p class="text-slate-300 text-sm mb-4">От к
 ет O(n). AVL и красно-чёрные выбирают пр
 по факту: ничего не хранит, дереву разр

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 i
 rder border-emerald-500 shadow-md">
 Аналогия 1: Стопка бумаг
```

```

.....
 <th class="py-2 pr-3 font-b
 <th class="py-2 font-bold">
 </tr>
</thead>
<tbody class="text-slate-300">
 <tr class="border-b border-slate
 <td class="py-3 pr-3 font-b
 <td class="py-3 pr-3">деда
 <td class="py-3 pr-3">сам х
 <td class="py-3">х – корень
 </tr>
 <tr class="border-b border-slate
 <td class="py-3 pr-3 font-b
 <td class="py-3 pr-3">х и р
 <td class="py-3 pr-3"><span
 <td class="py-3">ветка укор
 </tr>
 <tr>
 <td class="py-3 pr-3 font-b
 <td class="py-3 pr-3">х и р
 <td class="py-3 pr-3"><span
 <td class="py-3">р и г стал
 </tr>
</tbody>
</table>
</div>
<p class="text-slate-300 text-sm mt-3">И по
у. Zig случается только последним.</p>

<div class="bg-rose-950/30 rounded-xl borde
 <p class="text-rose-300 font-bold text-
 <pre class="bg-slate-950 p-4 rounded-lg
аивно (х, потом х): 60→40→20 стало 20→60→
Zig-Zig (р, потом х): 60→40→20 стало кустом 20(40(60)),
 <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>

```

```

.....
 <pre class="bg-slate-950 p-4 rounded
узел: { key, left, right, parent }

def rotate_up(x): # x встаёт на место
 p = x.parent
 g = p.parent
 if p.left is x: # правый поворот
 p.left = x.right
 if x.right: x.right.parent = p
 x.right = p
 else: # левый (зеркально)
 p.right = x.left
 if x.left: x.left.parent = p
 x.left = p
 p.parent = x
 x.parent = g
 if g:
 if g.left is p: g.left = x
 else: g.right = x

def splay(x): # поднять x в корень
 while x.parent:
 p, g = x.parent, x.parent.parent
 if g is None: # ZIG
 rotate_up(x)
 elif (g.left is p) == (p.left is x): # ZIG-ZIG
 rotate_up(p)
 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + сп
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break

```



```
.....
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">7. Графы. Планарные. Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
}
```

```

.....
 </div>
 <p class="text-slate-300 text-sm mb-1">получить опыт, нужна работа". Алгоритм выявления
 </div>
 </div>
</div>
</div>
</section>`,
},
{
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Макро-вершины — это вершины, которые принадлежат одной и той же
 <p class="text-xl font-mono text-white">сильно связной компоненте (SCC) графа. Они инвертированы в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-1">Представьте себе карту города, где улицы имеют только одно направление движения. Если вы можете пройти по улицам от точки А к точке Б, но не наоборот, то А и Б находятся в разных "блоках" (SCC).
 </div>
 </div>
 </div>
 </div>
`

```

```

</section>',
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">Точки сочленения
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-rose-400">Точки сочленения
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Вершины, удаление которых убивает связность
 <p class="text-xl font-mono text-white">
 $\text{deg}(v) > 1$
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса без
 </p>
 </div>
 </div>
 </div>
 </div>
</section>',
 },
 {
 id: "graph-euler",

```

```

.....
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <p>д-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ней земли. Но если начать копать одновременно
 в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

.....
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
 </div>
 </div>
 </div>
 </section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400">

```

```

.....
 i}, который одновременно является её суффикс
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
мя перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>',
},
{
 id: "string-z-func",
 title: "22. Z-функция",

```

```
.....
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
</pre>

```

</div>

</details>

</div>

</div>

</section>`,

},

{

id: "complexity-classes",

title: "24. Классы сложности, сведение задач",

type: "html",

description: "",

category: "Теория сложности",

content: `

<section id="complexity-classes" class="mb-12 scroll-mt

<div class="flex items-center mb-6 flex-wrap gap-3">

<span class="bg-purple-600 text-white px-4 py-1

<h2 class="text-2xl sm:text-3xl font-bold text-

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl bord

<h3 class="text-xl font-bold text-purple-400

<div class="bg-slate-900 p-4 rounded-lg mb-1

<p class="text-lg text-purple-300 itali

<p class="text-xl font-mono text-white">

Сведение (Reduction) A->B: Если я умею реша

</div>

<div class="grid grid-cols-1 xl:grid-cols-2

<!-- Аналогия 1 -->

<div class="bg-slate-800 p-6 rounded-lg

<div class="absolute -top-3 left-4

rder border-emerald-500 shadow-md">

Аналогия 1: Судoku (P vs NP)

</div>

<p class="text-slate-300 text-sm mb

пример сортировка чисел). Класс <b>NP</b> –

```

<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Эйлер
 </div>

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">

```

```

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded">
 <p class="text-sm text-blue-300">
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
 </div>

```

```

 <p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
 </p>
 </div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap">
 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Путь: Нормальная прогулка
 </div>

```



```
.....
TSP – NP-трудная: полиномиальный ал
В общем, не парься. Просто запомни
 </p>
 </div>
</details>
</div>
</section>',
},
{
 id: "bridges-code",
 title: "Мосты в графах: код + визуализация",
 type: "html",
 content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-emerald-300 ital
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">tin{v
 <p><strong class="text-white">low{v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков <str
 Единственная ниточка. Порвётся – граф р
```

.....  
оходов.</p>

```
<div class="bg-slate-950 p-4 rounded-lg" style="background-color: #1a202c; color: white; padding: 10px; border-radius: 0.5em;"><pre class="text-xs font-mono text-white" style="font-family: monospace; font-size: 0.8em; color: white;">#include <vector>
#include <algorithm>

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежностей
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем время

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}
```

.....

</div>

</div>

<p class="text-slate-300 text-sm">

<strong>Следствие:</strong> Для планарного графа  $G$  с  $n$  вершинами и  $m$  ребрами:

Если  $m > n - 1$ , то  $G$  не планарен.

</p>

</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">

<div class="bg-slate-800 p-6 rounded-lg border">

<div class="absolute -top-3 left-4 bg-slate-800 text-white p-2 border order-green-500 shadow-md">

K5: Полный граф на 5 вершинах

</div>

<p class="text-slate-300 text-sm mb-4">

У K5: `<code class="text-white">V=5, E=10 > 9</code>` – БАХ, непланарен!

Все 5 вершин соединены со всеми. На K5 не планарно.

Теорема Куратовского: K5 – эталон непланарности.

Если внутри графа спрятан K5 – забудем про него.

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border">

<div class="absolute -top-3 left-4 bg-slate-900 text-white p-2 border order-rose-500 shadow-md">

K3,3: 3 дома и 3 колодца

</div>

<p class="text-slate-300 text-sm mb-4">

Три дома хотят соединиться с тремя колодцами.

Нарисовать без пересечений невозможно.

У него `<code class="text-white">V=6, E=9 <code>` – проходит, но он не планарен.

Почему? Потому что у двудольного графа  $G$  с  $n$  вершинами:

Отсюда более строгое ограничение: `<code class="text-white">E <code>`

`<code class="text-white">E > 8</code>`

Красим карту регионов: соседние  
Регион  $\approx$  вершина, общая граница  
Для плоской карты (планарный граф)  
теорема о четырёх красках (Аппел).

```
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000000; border-radius: 10px; width: 100%; margin: 10px auto; text-align: center;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-amber-500 shadow-md">
```

## Что нужно знать на экзамен

```

 </div>
 <ul class="text-slate-300 text-sm s
 <strong class="text-white">
вободный у соседей. Дал $\leq \Delta+1$ цветов (Δ – м
 <strong class="text-white">
тит 2 цветов, хоть он и непланарен.
 <strong class="text-white">
 $= n$, $\chi(\text{цикла чётной длины}) = 2$, $\chi(\text{цикла неч}$
 <strong class="text-white">
й и не нечётный цикл, $\chi(G) \leq \Delta$.


```

</u>  
</div>  
</div>

<p class="text-slate-400 text-xs mt-4">

Почему раскраска «сложная»: проверить раскраску — полная задача.

Даже «раскрасима ли карта в 3 цвета?» —

</p>

&lt;/section&gt;

```
id: "graph-articulation",
title: "12. Графы. Точки сочленения",
type: "html",
```

```
.....
 её перекроют (удалят вершину) – районы буд
g>хрупкость системы.
 </p>
</div>

<div class="bg-slate-900 p-6 rounded-lg border-
 <div class="absolute -top-3 left-4 bg-em
er border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
абелем (мостом). Сами свитчи – это точки со
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border-
 <summary class="p-4 font-bold text-slate-400
order-slate-700/50">
 Душный мод: Код и корень DFS (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300 space-
 <p class="text-blue-400 font-bold">Особ
 <p>
 Для стартовой вершины обхода правил
то выше неё прыгать некуда). Поэтому для не
у него больше 1 независимого ребенка
 </p>
 <div class="bg-slate-950 p-4 rounded-lg
 <pre class="text-xs font-mono text-
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
```

ФАЙЛ 19/96: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 129 | РАЗМ

```
import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets1to5 } from "../tickets/ticket1_5";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as extraChapters } from "../content_tem
```

```
/**
```

```
 * Канонический порядок тем (билеты 1–24) и «правильные»
 * одна страница может раскрывать одну тему или несколько.
 * поэтому названия выверяются по единому списку ниже.
 */
```

```
const CANON: Record<string, { num: number; title: string }> = {
 "segment-trees": { num: 1, title: "1. Дерево отрезков" },
 "sparse-table": { num: 2, title: "2. Двумерная разреженная таблица" },
 "treap": { num: 3, title: "3. Декартово дерево (Treap)" },
 "splay-tree": { num: 4, title: "4. Splay-дерево" },
 "prefix-sums-2d": { num: 5, title: "5. Двумерная матрица префиксов" },
 "graph-dfs-bfs": { num: 6, title: "6. Графы. Представление и обход" },
 "planarity-euler-formula": { num: 7, title: "7. Графы. Плоскостность и формула Эйлера" },
 "graph-components": { num: 8, title: "8. Графы. Компоненты связности" },
 "top-sort": { num: 9, title: "9. Графы. Топологическая сортировка" },
 "scc-kosaraju": { num: 10, title: "10. Графы. Компоненты сильной связности" },
 "bridges-code": { num: 11, title: "11. Графы. Мосты" },
 "graph-articulation": { num: 12, title: "12. Графы. Точки articulation" },
 "euler-path-vs-cycle": { num: 13, title: "13. Графы. Пути Эйлера и циклы" },
 "dijkstra": { num: 14, title: "14. Графы. Поиск кратчайших путей" },
 "bellman-ford": { num: 15, title: "15. Графы. Поиск кратчайших путей" },
 "floyd": { num: 16, title: "16. Графы. Поиск кратчайших путей" },
}
```

```

// Нормализуем названия по каноническому списку тем, а
// по учебным блокам: блоки идут строго в порядке чтени
// номерам билетов. Близкие темы {1–5, 14–17, 18–20, 21–23}
const BLOCKS: [number, number, string][] = [
 [1, 5, "Билеты 1–5 · Структуры и матрицы"],
 [6, 13, "Билеты 6–13 · Графы: обходы и связность"],
 [14, 17, "Билеты 14–17 · Кратчайшие пути"],
 [18, 20, "Билеты 18–20 · Остовные деревья"],
 [21, 23, "Билеты 21–23 · Строки"],
 [24, 24, "Билет 24 · Теория сложности"],
];

const blockOf = (num: number): string => BLOCKS.find(([, n]) => n === num);

const withCanon = Array.from(dedup.values()).map((c) => {
 const canon = CANON[c.id];
 if (canon) return { ...c, title: canon.title, category: canon.category };
 return { ...c, category: "Введение" }; // страницы вне канона
});

// Сначала вводная страница, затем билеты 1–24 по порядку
const numbered = withCanon.filter((c) => CANON[c.id]);
numbered.sort((a, b) => CANON[a.id].num - CANON[b.id].num);
const extras = withCanon
 .filter((c) => !CANON[c.id])
 .sort((a, b) => a.title.localeCompare(b.title, "ru"));

export const chapters: Chapter[] = [...extras, ...numbered];

/**
 * «Читать рядом»: связанные билеты, которые логично от
 * (блок рисуется внизу страницы). Ссылки кликабельны и
 */
export const RELATED: Record<string, string[]> = {
 "segment-trees": ["sparse-table", "prefix-sums-2d"],
 "sparse-table": ["prefix-sums-2d", "segment-trees"],
 "prefix-sums-2d": ["sparse-table", "segment-trees"],
 treap: ["splay-tree", "segment-trees"],
};

```

## Универсальное пособие • полный исходный код

```
.....
 line: number;
 /** Имя функции-фрейма {"<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**
 * JSON-снимок доступных имён (globals + locals). В о
 * можно напрямую отдать визуализации: i/j остаются ч
 */
 values?: Record<string, DebugValue>;
}

export interface DebugResult {
 steps: DebugStep[];
 /** Локали в момент возврата из модуля – «результат»
 finalLocals?: Record<string, string>;
 /** Финальный машинно-читаемый снимок для визуализации
 finalValues?: Record<string, DebugValue>;
 /** Текст последней необработанной ошибки (или пустая
 error: string;
 /** true, если уперлись в лимит шагов. */
 truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 1200;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
 const srcLiteral = JSON.stringify(userSrc); // JSON-с
 return `import sys as __sys, json as __json

__src = ${srcLiteral}
steps = []
```



```

.....
def __visible(items):
 return {
 k: v for k, v in items
 if not k.startswith("__") and type(v).__name__
 }

def __capture(frame):
 local_values = __visible(frame.f_locals.items())
 # Внутри функции алгоритма важные структуры (memo,
 # глобальные. Локальные значения имеют приоритет на
 scope_values = __visible(frame.f_globals.items())
 scope_values.update(local_values)
 loc = {k: __safe(v) for k, v in local_values.items()}
 values = {k: __snapshot(v) for k, v in scope_values.items()}
 return loc, values

def __tracer(frame, event, arg):
 global __was_truncated
 if event == "line" and frame.f_code.co_filename == __file__
 and not (frame.f_code.co_name.startswith("<
 loc, values = __capture(frame)
 # PEP 709: с 3.12 компрехенشنы встроены во фреймворк
 # на своей строке каждую итерацию. Склеиваем по
 if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
 else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __file__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)

```

```
.....
 return out;
}

/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name.slice(0, name.indexOf('['))

/**
 * Порядок показа в панели переменных: сначала переменные текущей
 * страницы (те, что подсвечивает визуализация), остальные в алфавитном порядке.
 */
export function orderVars(
 cur: Record<string, string>,
 syncNames: string[]
): [string, string][][] {
 const inSync = new Set(syncNames);
 const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
 const rest = Object.entries(cur)
 .filter(([k]) => !inSync.has(k))
 .sort(([a], [b]) => a.localeCompare(b));
 return [...syncEntries, ...rest];
}
```

.....  
ФАЙЛ 21/96: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 636 | РАЗМЕР: 10.5 КБ

```
.....
import type { DemoKind } from '../components/hints/MiniHints';

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
```

```

.....
 demo: "dfs",
 simulator: "stack-dfs",
},
{
 id: "binsect",
 labels: ["binsect", "бинсект", "бинарный поиск", "д",
 title: "Бинарный поиск (и его самозванец binsect)",
 short:
 "Бинарный поиск НАХОДИТ готовый элемент в уже отс
 о не сортирует. «binsect» из конспекта – шутлив
 quicksort.",
 formal:
 "Поиск одного элемента в отсортированном массиве -
 $O(n \log n)$. Бинарный поиск сортировку не выполн
 complexity: "поиск: $O(\log n)$; сортировкой не являет
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "к",
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого
 formal:
 "Полное бинарное дерево в массиве: дети узла i ле
 complexity: "вставка/извлечение $O(\log n)$, «посмотре
 demo: "heap",
 simulator: "heap-beam-search",
},
{
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO",
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь свер
 formal: "push / pop / top за $O(1)$. На стеке живёт р
 demo: "stack",
 simulator: "stack-dfs",
},
{

```

```

.....
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
},
{
 title: "Обход бора в ширину (тот самый «какой-то обход бора»)",
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксную очередь

 о есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень). Если

 есть ссылка, то кладём её в очередь.",
 complexity: "O(суммарной длины словаря × алфавит)",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки"],
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошла к

 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает суффиксальную

 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топосорт"],
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая ссылка

 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, лемма о топосорте",
 complexity: "O(V + E)",
 demo: "toposort",

```

```

.....
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дер"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь ма
 товых кусков.",
 formal: "Строится за $O(n)$, запрос и обновление – $O($
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостов"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается.
 formal: "Ребро (u,v) – мост $\text{low}[v] > \text{tin}[u]$, где
 ребро в родителя – нет).",
 complexity: " $O(V + E)$ одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "т"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её – и колл
 formal: "Для не-корня: v – точка сочленения $\exists$ реб
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [

```

```

.....
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»:
 formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$, где h — ра
 к путей сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
},
{
 id: "scc",
 labels: ["компонента сильной связности", "компоненты
 title: "Компоненты сильной связности",
 short:
 "Группы вершин, где из каждой можно дойти до каждо
 formal: "Косарайю: DFS с сохранением порядка выхода
 complexity: " $O(V + E)$ ",
 demo: "scc",
 simulator: "scc-kosaraju",
},
{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу
 "префикс", "префикса", "суффиксом", "суффикс", "с
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начал
 от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s
 возвращая указатель по тексту.",
 complexity: " $O(n + m)$ ",
 demo: "prefix-function",
 simulator: "string-kmp",
}

```

```

.....
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикла"],
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»",
 formal:
 "Форд–Беллман: если на V-й итерации ещё удаётся с",
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешёвое ребро", "самое дешёвое ребро", "сам",
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас",
 formal:
 "Корректность обычно доказывается «леммой о разрезе»",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь",
 formal:
 "Связный граф: эйлеров цикл все степени чётные;
эйлеров путь все степени чётные, кроме двух",
 simulator: "euler-path-vs-cycle",
},

```

```

.....
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали
 complexity: "O(E log V) с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
 title: "Алгоритм Форда-Беллмана",
 short:
 "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбра
 ",
 formal:
 "После k-й итерации гарантированно верны все крат
 ось — есть отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки
 шить пересадку в k?",
 formal:
 " $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$. Поряд
 ами.",
 complexity: "O(V³) времени, O(V²) памяти",

```



```

.....
 simulator: "mst-kruskal",
 },
 {
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "к",
 "компоненте связности", "связности", "связный", "с",
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мостов",
 formal:
 "Считаются одним проходом: пока есть непосещённая",
 complexity: "O(V + E)",
 demo: "components",
 simulator: "graph-dfs-bfs",
 },
 {
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "граф",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города",
 formal:
 "G = (V, E). Ребро бывает ориентированным (улицы)",
 demo: "graph",
 simulator: "graph-dfs-bfs",
 },
 {
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "с",
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам ну

```

```

short:
 "Чтобы получить сумму прямоугольника, берём больш
 ый отрезали дважды.",
formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
complexity: "запрос O(1) после O(nm) предподсчёта",
demo: "prefix-sum",
simulator: "prefix-sums-2d",
},
{
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведён
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно
 formal:
 "A ≤p B: есть полиномиальная функция, переводящая
 NP-полную задачу к новой.",
 demo: "np",
},
{
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива
 предподсчёты.",
 formal:
 "Суммы – префиксные суммы O(1). Минимум без изменен
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "с
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: г

```

```

.....
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
demo: "zig-zig",
simulator: "splay-rotations",
},
{
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – прав
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева
demo: "zig-zag",
simulator: "splay-rotations",
},
};

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id }))
}).sort((a, b) => b.label.length - a.label.length);

```

.....  
ФАЙЛ 22/96: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРок: 275 | РАЗМ

```

.....
import { Chapter } from "../types";

```

```

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Обзор: Кратчайшие пути и Графы",

```

```

.....

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
 Аналогия 1: Позитивное плани
 </div>
 <p class="text-slate-300 text-sm mb
(нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstra"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 i
border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb
охождение улицы), Дейкстра сломается, так к
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg i
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 .
 <p>Алгоритм использует приоритетную
g V). Гарантирует корректность только для н
 </div>
</details>

<div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>

```



```
.....
 <p class="text-sm text-blue-300 italic mb-2">0c
 <p class="text-lg font-mono text-white">Бор (Тр
</div>
<p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы движемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 терминальные ссылки (term_link) – прямые мос
 <p>
 </div>
 </div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 outl
 pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скры
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
```

```

 }
 }
 </div>
</div>
</details>
</div>
</section>`
 },
 {
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-12">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">Алгоритмы на карте
 <p class="text-slate-300 text-sm mb-4">Когда мы имеем дело с графами, которые могут
 содержать большие и маленькие числа (долгота и широта), мы часто хотим
 использовать целые числа, например, от 0 до N, и уже по ним строим графы.</p>
 </div>
 </div>
 </div>
 </section>`
 }
];

```

ФАЙЛ 23/96: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМЕР: 1.5 КБ

```
export interface QuizQuestion {
```

```

.....
 explanation: "Если нечётных вершин больше 2, эйлерово графа не существует. Если ровно 2 нечётные или 0."
}
},
"bridges-code": {
{
 question: "После DFS у нас low[u] = 4, а tin[v] = 2. Это условие моста?",
 options: [
 "Да, так как low[u] (4) > tin[v] (2)",
 "Нет, low[u] должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: low[u] > tin[v]. В неориентированном графе low[u] — минимальное время входа в вершину u, а tin[v] — время входа в вершину v. Если low[u] > tin[v], то вершина v не является предком u, и ребро (u, v) не является мостом."
},
{
 question: "Для чего нужен массив low[v]?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого нужно вернуться",
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: "low[v] хранит минимальное время входа в вершину v или в одну из её дочерних вершин. Это используется для проверки условия моста: low[u] > tin[v]. В неориентированном графе low[u] — минимальное время входа в вершину u, а tin[v] — время входа в вершину v. Если low[u] > tin[v], то вершина v не является предком u, и ребро (u, v) не является мостом."
},
{
 question: "Какова временная сложность алгоритма поиска мостов?",
 options: [
 "O(V^2) — ищем мосты в квадрате",
 "O(V + E) — каждый элемент один раз",
 "O(2^V) — полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое ребро ровно один раз. Поэтому временная сложность алгоритма поиска мостов — O(V + E)."
}
},
"planarity-euler-formula": {

```



Универсальное пособие • полный исходный код

```
.....
.....

import { createContext, useContext, useEffect, useState } from "react";
import type { DebugValue } from "../debugRunner";

/**
 * Прямая связь «компилятор ↔ открытая визуализация».
 *
 * Компилятор передаёт не «магический номер строки эталонного
 * выполненного Python-кода. Визуализация читает захардкоженные
 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие строки.
 * любой пользовательский код работает, если в нём использованы
 */

export const VizChapterContext = createContext("");

const CHANNEL = "algo:viz-state";

export interface VizStateEventDetail {
 chapterId: string;
 /** Строка и функция реально выполненного пользователя */
 line?: number;
 func?: string;
 /** Машинно-читаемые globals + locals: числа остаются строками */
 variables?: Record<string, DebugValue>;
 /** Имена, изменённые выделенной строкой. */
 changed?: string[];
}

/** Последний снимок хранится, чтобы поздно смонтировать визуализацию
const latestByChapter = new Map<string, VizStateEventDetail>();

/** Транслировать реальные переменные визуализациям текстового кода
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
```

Универсальное пособие • полный исходный код

```
.....
* Подписка старых покадровых демонстраций. Если переда
* номер кадра выбирается по именам/значениям переменны
*/
```

```
export function useVizStepSync(
 currentStep: number,
 goToStep: (n: number) => void,
 maxStep: number,
 selectByVariables?: VizStepSelector
) {
 const chapterId = useContext(VizChapterContext);

 useEffect(() => {
 if (!chapterId) return;

 const apply = (detail?: VizStateEventDetail) => {
 if (!detail || detail.chapterId !== chapterId) re
 // Без явного селектора компонент либо рисует use
 // либо остаётся ручным. Номер Python-строки нико
 const requested =
 selectByVariables && detail.variables ? selectB
 // null означает: компонент рисует runtime-перемен
 // не увидел знакомых захардкоженных имён.
 if (requested === null || !Number.isFinite(reques
 const target = Math.max(0, Math.min(Math.max(0, m
 if (target !== currentStep) goToStep(target);
 };

 apply(latestByChapter.get(chapterId));
 const handler = (e: Event) => apply((e as CustomEven
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, ha
 }, [chapterId, currentStep, maxStep, goToStep, select
}
```

```
/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = {value: DebugValue | undefined
 typeof value === "number" && Number.isFinite(value) ?
```

```
.....
const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
 if (detail && detail.chapterId === chapterId) cb(detail);
};
window.addEventListener(DEMO_CHANNEL, handler);
return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

.....  
ФАЙЛ 25/96: src/data/vizSync.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 883 | РАЗМЕР: 10.5 КБ

```
.....
/**
 * Захардкоженная «карта синхронизации» компилятора с визуализацией
 *
 * Для каждой страницы (главы) здесь вручную перечислены переменные,
 * которые понимает её визуализация: i, j, k, v, dist, ...
 * Компилятор выполняет любой Python-код и передаёт результат
 * имён напрямую; привязки к номерам или тексту строк эмулируются
 */
```

```
export interface SyncVariable {
 /** Имя переменной, как в коде визуализации: i, j, k, ... */
 name: string;
 /** Роль в алгоритме (показывается во всплывающей подсказке) */
 role: string;
 /** Характерный диапазон/значение на демонстрации. */
 range?: string;
}
```

```
export interface PageSync {
 /** Заголовок демонстрации на странице (если интерактивна) */
 vizTitle?: string;
 /** Чему соответствует один шаг визуализации (для тултипов) */
 stepNote?: string;
 /** Переменные синхронизации. Пусто – на странице нет синхронизации */
}
```

## Универсальное пособие • полный исходный код

```

.....
 { name: "lazy", role: "массив обещаний: lazy[v] *
 },
},
treap: {
 vizTitle: "Treap: плоскость → дерево → Split/Merge/
 stepNote: "Шаг = вставка очередной пары (x; y) или
 code: `pairs = [(45, 8), (3, 6), (6, 5), (2, 4), (9
pairs.sort(key=lambda p: -p[1]) # сортируем по y —
x, y = 6, 5 # подсвеченная пар
i = 0 # индекс шага: сор

def insert(root, x, y): # вставка спуском
 ... # ключ <= узла — в
def split(t, x): # разрез по ключу:
 ... # узел целиком ухо
 variables: [
 { name: "x", role: "ключ — координата точки по гор
 o-набора: 1...45" },
 { name: "y", role: "приоритет — координата по вер
 { name: "i", role: "индекс шага: сортировка (0...3)
 },
],
},
"splay-tree": {
 vizTitle: "Splay-дерево: большое дерево",
 stepNote: "Шаг = один кадр splay(key): прицел ребра
 code: `keys = [50, 40, 35, 32, 30, 28, 12, 60, 55,
key = 32 # какой ключ поднимаем в кор
i = 0 # кадр анимации: 0 ... (число
case = "zig-zig" # вкладка «Случай поворота»

print(f"splay({key}): подъем в корень`",
 variables: [
 { name: "keys", role: "демо-дерево страницы (BST
 { name: "key", role: "ключ, который splay поднима
 ,
 { name: "i", role: "номер кадра анимации", range:
 { name: "case", role: "случай поворота для вкладки

```

## Универсальное пособие • полный исходный код

```
.....
vizTitle: "Префиксные суммы (1D и 2D)",
stepNote: "Шаг = заполнение очередного $P[i] = P[i-1]$ ",
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
 P.append(P[i - 1] + a[i - 1])
print("P =", P)\`,
variables: [
 { name: "a", role: "исходный массив демо (вкладка" },
 { name: "i", role: "индекс элемента, который сейч" },
 { name: "P", role: "префиксные суммы: $P[i] = a[0]$ " },
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизаци" },
 stepNote: "Шаг = событие внутри fibМемо(k): вызов" },
 code: `n = 5
memo = {}

def fib(k):
 if k in memo:
 return memo[k]
 if k <= 2:
 return 1
 print(f"считаем fib({k-1}) + fib({k-2})")
 r1 = fib(k - 1)
 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))\`,
variables: [
 { name: "n", role: "целевой номер числа Фибоначчи" },
 { name: "k", role: "аргумент текущего рекурсивного" },
 { name: "memo", role: "кэш уже посчитанных значений" },
 { name: "r1, r2", role: "ответы двух рекурсивных"
```

## Универсальное пособие • полный исходный код

```
.....
q = deque(["A"]) # очередь BFS (FIFO)
q.append("B") # enqueue соседа в хвост
v = q.popleft() # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
 variables: [
 { name: "q", role: "очередь (массив): гости страны",
 { name: "v", role: "вершина, извлечённая из головы"
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к непосещенной",
 code: `adj = {"A": ["B", "C"], "B": ["A", "D", "E"], "C": ["A", "B"], "D": ["B"], "E": ["B"]
visited, order = set(), [] # порядок обхода
stack = ["A"] # DFS: стек (рекурсия)
q = [] # BFS: очередь (deque)
v = "A" # шаг: достали вершину из очереди

--- тело алгоритма ---
while stack: # ---- DFS ----
 v = stack.pop()
 if v in visited:
 continue
 visited.add(v)
 order.append(v)
 stack += [u for u in adj[v] if u not in visited]
print("DFS:", order)

visited, order, q = set(), [], ["A"] # ---- BFS ----
while q:
 v = q.pop(0)
 if v in visited:
 continue
 visited.add(v)
 order.append(v)
 q += [u for u in adj[v] if u not in visited]
print("BFS:", order)`,
 variables: [
```

```

.....
],
 },
 "top-sort": {
 vizTitle: "Топологическая сортировка",
 stepNote: "Шаг = выход рекурсии из вершины v: она д
 code: `adj = {0: [1, 2], 1: [3], 2: [3], 3: [4], 4:
used, order = set(), []

def dfs(v):
 used.add(v)
 for to in adj[v]: # шаг: ребро v → to
 if to not in used:
 dfs(to)
 order.append(v) # выход из v!

--- тело алгоритма ---
for v in adj:
 if v not in used:
 dfs(v)
print(order[::-1]) # разворот – ответ`,
 variables: [
 { name: "v", role: "текущая вершина DFS – подсвеч
 { name: "to", role: "вершина, куда ведёт ребро из
 { name: "order", role: "порядок выхода из рекурсии
 { name: "used", role: "множество посещённых верши
],
 },
 "scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)"
 stepNote: "Шаг = вершина из order (в обратном поряд
 code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [4], 4:
order, used = [], set())

def dfs1(v): # проход 1: порядок выхода
 used.add(v)
 for to in adj[v]:
 if to not in used:
 dfs1(to)

```

## Универсальное пособие • полный исходный код

```

.....
 stepNote: "Шаг = обновление low[v] по ребру; вершина v",
 code: `# демо-граф страницы: треугольник 0-1-2, мост 3-4-5-6-7
DFS из 0: tin/low после полного обхода; точки сочленения
tin = {0: 1, 1: 2, 2: 3, 3: 4, 4: 5, 5: 6, 6: 7}
low = {0: 1, 1: 1, 2: 1, 3: 4, 4: 5, 5: 5, 6: 5}

v, to = 3, 4 # шаг: возврат из 4 в 3
is_cut = low[to] >= tin[v] # 5 >= 4 → 3 – точка сочленения
v2, to2 = 1, 2
is_cut2 = low[to2] >= tin[v2] # 1 >= 2 → нет (1 – точка сочленения)
print(f"v={v}: точка сочленения? {is_cut}; v={v2}: {is_cut2}")
 variables: [
 { name: "tin", role: "время захода DFS в вершину" },
 { name: "low", role: "минимальный tin, достижимый по ребрам" },
 { name: "v, to", role: "ребро дерева DFS: проверяем" },
 { name: "is_cut", role: "low[to] ≥ tin[v] и v – не лист" },
],
},
"bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v, to)",
 code: `# демо-граф страницы: A-B, B-C, A-C, C-D, D-E, E-F, F-G
DFS из A: tin/low после полного обхода
tin = {"A": 1, "B": 2, "C": 3, "D": 4, "E": 5, "F": 6, "G": 7}
low = {"A": 1, "B": 1, "C": 1, "D": 4, "E": 4, "F": 4, "G": 7}

v, to = "B", "G" # шаг: возврат из G в B (G – точка сочленения)
is_bridge = low[to] > tin[v] # 7 > 2 → мост!
v2, to2 = "C", "D"
is_bridge2 = low[to2] > tin[v2] # 4 > 3 → C-D тоже мост!
print(f"{v}-{to}: мост? {is_bridge}; {v2}-{to2}: мост? {is_bridge2}")
 variables: [
 { name: "tin", role: "время захода DFS в вершину" },
 { name: "low", role: "минимальный tin, достижимый по ребрам" },
 { name: "v, to", role: "ребро дерева DFS, для которого проверяем" },
 { name: "is_bridge", role: "low[to] > tin[v] ⇒ из v в to нет пути без ребра (v, to)" },
],
},

```



```

.....
while c in used: c += 1
color[v] = c
k = max(k, c + 1)`,
variables: {
 { name: "V", role: "число вершин графа", range: "1"
 { name: "E", role: "число рёбер", range: "K5: 10"
 { name: "F", role: "число граней в плоской укладке"
 { name: "v", role: "вершина, которую красят (метка)"
 { name: "color", role: "массив/словарь цветов вершин"
 { name: "k", role: "сколько цветов уже использовано"
},
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины",
 code: `import heapq

g = {"A": [("B", 4), ("C", 2)], "B": [("G", 4), ("D", 5)],
 "E": [("F", 1)], "F": []}
dist = {"A": 0}
prev = {}
done = set()
pq = [(0, "A")]
while pq:
 du, u = heapq.heappop(pq)
 if u in done:
 continue
 done.add(u)
 print(f"обрабатываем {u} (dist={du})")
 for v, w in g[u]:
 if v in done:
 continue
 print(f"смотрим ребро {u}→{v} (w={w})")
 if du + w < dist.get(v, float("inf")):
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
print("итог:", dist)`

```

## Универсальное пособие • полный исходный код

```

.....
d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
 [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, INF, 0],
 [INF, INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, INF, INF, 0],
 [INF, INF, INF, INF, INF, INF, 0]]
n = 7
k, i, j = 0, 0, 0 # шаг: посредник k, летим

--- тело алгоритма ---
for k in range(n): # посредник k
 for i in range(n):
 for j in range(n):
 if d[i][k] + d[k][j] < d[i][j]: # через k
 d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны")`
variables: [
 { name: "d", role: "матрица смежности → кратчайшие"},
 { name: "n", role: "число вершин (аэропортов)", g: 1},
 { name: "k", role: "посредник: разрешённые пути и"},
 { name: "i, j", role: "клетка d[i][j], которую пр"},
 { name: "d[i][j]", role: "текущая длина кратчайше"},
],
},
"johnson-algo": {
 vizTitle: "Алгоритм Джонсона (перевзвешивание)",
 stepNote: "Шаг = Беллман-Форд из фиктивной вершины",
 code: `# демо-граф страницы: S→{A,B,C} (0), A→B=-2,
потенциалы – реальные расстояния от фиктивной S (Белл
h = {"A": 0, "B": -2, "C": -1}

u, v, w = "A", "B", -2 # ребро с отрицательным ве
w_new = w + h[u] - h[v] # w' = w + h[u] - h[v] = -
print(f"w'({u}→{v}) = {w_new}")
остальные: B→C: 1 + (-2) - (-1) = 0 · C→A: 2 + (-1)
после Дейкстры ответ = d[v] - h[start] + h[v]`,
variables: [
 { name: "h", role: "потенциалы: кратчайшие пути о"},
 { name: "u, v, w", role: "ребро (u, v) перевзвешив"},
 { name: "w_new", role: "новый вес – всегда ≥ 0, к"},
 { name: "d[v]", role: "расстояние Дейкстры в новы"},
],
},

```

# Универсальное пособие • полный исходный код

```

.....
code: `import heapq

смежность демо-графа страницы: 9 вершин A..I
adj = {"A": [(2, "B"), (5, "D")], "B": [(2, "A"), (4, "C"), (7, "E")], "C": [(4, "B"), (9, "F")], "D": [(5, "A"), (3, "E")], "E": [(7, "B"), (9, "C"), (13, "I")], "F": [(9, "C"), (10, "G")], "G": [(8, "F"), (12, "H")], "H": [(12, "G"), (13, "I")], "I": [(13, "E"), (13, "F")]}
heap = [(0, "A", "-")] # (вес, вершина, откуда)
taken, mst = set(), []
w, v, parent = 2, "B", "A" # шаг: ребро из кучи
i = 0

--- тело алгоритма ---
while heap and len(taken) < len(adj):
 w, v, parent = heapq.heappop(heap)
 if v in taken:
 continue
 taken.add(v)
 if parent != "-":
 mst.append((w, parent, v))
 for w2, u in adj[v]:
 if u not in taken:
 heapq.heappush(heap, (w2, u, v))
print("остов:", mst, "вес:", sum(w for w, _, _ in mst))
variables: [
 { name: "adj", role: "список смежности демо-графа",
 { name: "w, v, parent", role: "ребро из кучи: вес",
 { name: "heap", role: "приоритетная очередь канди",
 { name: "taken", role: "вершины уже в дереве — «з",
 { name: "mst", role: "рёбра остова, вес 51", gang
],
},
"mst-boruvka": {
 vizTitle: "Боруака",
 stepNote: "Шаг = каждая компонента одновременно выб",
 code: `# демо-граф страницы: 9 вершин A..I, 12 рёбер
E = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "D"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10,

```

## Универсальное пособие • полный исходный код

```
.....
 stepNote: "Шаг = обработка символа s[i]: откаты по
 code: `pattern = "aba"
text = "abacaba"
s = pattern + "#" + text # демо-страница ищет pattern
i, j = 1, 0
pi = [None] * len(s) # пустые клетки: значения п
pi[0] = 0

--- тело алгоритма ---
for i in range(1, len(s)): # шаг: считаем pi[i]
 j = pi[i - 1] # длина совпадения
 while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат назад
 if s[i] == s[j]:
 j += 1
 pi[i] = j
print(pi)`,
 variables: [
 { name: "pattern", role: "образец: питон подставл
 { name: "text", role: "текст, в котором ищем", га
 { name: "s", role: "склейка pattern + \"#\n\" + tex
 { name: "i", role: "индекс текущего символа – счи
 { name: "j", role: "длина текущего совпавшего пре
 { name: "pi", role: "префикс-функция: $\pi[i]$ – наиб
],
},
"string-z-func": {
 vizTitle: "Z-функция",
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]
 code: `pattern = "aba"
text = "abacaba"
s = pattern + "#" + text # демо-страница ищет pattern
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * len(s) # пустые клетки: значения п
z[0] = 0

--- тело алгоритма ---
for i in range(1, len(s)): # шаг: вычисляем z[i]
```

Универсальное пособие • полный исходный код

```
.....
code: `cls = "NP" # класс: "P", "NP"
task = "3-SAT" # задача: "3-SAT", "Vertex Cover"

сведение A $\leq$ B: стрелка между задачами
reductions = [{"3-SAT", "Vertex Cover"}]
for a, b in reductions:
 print(f"{a} $\leq$ {b}")`
 variables: [
 { name: "cls", role: "какой класс подсветить: P / NP",
 { name: "task", role: "какую задачу подсветить: 3-SAT, Vertex Cover",
 { name: "reductions", role: "список пар задач — пар (A, B) : A $\leq$ B"
],
},
intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
},
"alg-map": {
 variables: [],
},

/* — Дополнительные демонстрации внутри страниц (включая)

"sparse-table#2d-build": {
 vizTitle: "Разреженная таблица 2D: построение",
 stepNote: "Шаг = новый уровень k: блоки $2^k \times 2^k$ доп.
code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67,
 88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [5
 1, 42, 85, 14]]
r, c, k = 0, 0, 0
st2 = {} # пустой каркас виден до

--- тело алгоритма ---
База не отдельный уровень вычислений: блок 1x1 — это
Записываем её одним шагом, а не проигрываем 64 одноти
Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(((r, c, 0, 0), A[r][c]) for r in range(4
```

Универсальное пособие • полный исходный код

```
.....
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2 # клетки 1-индексные, как
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (
 { name: "S[i][j]", role: "сумма прямоугольника (0
 },
 { name: "r1, c1, r2, c2", role: "углы запроса в 1
 ний", range: "1 ... n, 1 ... m" },
],
},
};

/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null,
 (demoId && chapterId && PAGE_SYNC[`${chapterId}#${demo
 (chapterId && PAGE_SYNC[chapterId]) ||
 { variables: [] });

/**
 * Стартовое содержимое редактора для страницы.
 *
 * Формат: короткий комментарий-шапка + исполняемый реф
 * Визуализация читает из его трассы знакомые имена (i,
 * но точно так же примет пользовательский код с теми ж
 */
export function buildSyncTemplate(chapterId: string, ch
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .
 : `# «${chapterTitle}»\n# демо на странице нет — св
```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 26/96: src/data/tickets/ticket1\_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 373 | РАЗМ

```
import { Chapter } from '../../../types';
import waffleImg from "../../../assets/waffle-grid.jpg";

export const tickets1to5: Chapter[] = [
 {
 id: "segment-trees",
 title: "1. Дерево отрезков с операциями на отрезках",
 type: "html",
 description: "Дерево отрезков – структура данных для",
 category: "Оптимизации и Продвинутое",
 content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div id="intro" class="bg-slate-700/50 p-6 rounded">
 <h3 class="text-xl font-bold text-blue-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
```

```

.....
 </div>
 </details>
 </div>
 </div>
</section>
`,
{
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные за
 ь отрезок длины 7 (перекрывая друг друга по
 ничего складывать, просто пересекаем куски.
 </div>

```



```

.....
 <p class="text-slate-300 text-sm">И
но: у суммы есть вычитание. Поэтому хва
</div>
 <div class="bg-slate-900 p-5 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Топт уровней – sparse table (
 </div>
 <div class="flex flex-col items-center">
 <div class="flex flex-col items-center">
 <div class="flex gap-1"><div cla
der border-rose-300 text-white text-[11px]
 <div class="text-[9px] sm:text-
 </div>
 <div class="flex flex-col items-center">
 <div class="flex gap-1"><div cla
border border-rose-400/60 text-slate-900 t
flex items-center justify-center rounded b
-bold shadow-sm">3</div><div class="w-8 h-8
er-rose-400/60 text-slate-900 text-[11px] s
center justify-center rounded bg-amber-100/
-sm">1</div><div class="w-8 h-8 sm:w-9 sm:h
60 text-slate-900 text-[11px] sm:text-xs fo
 <div class="text-[9px] sm:text-
 </div>
 <div class="flex flex-col items-center">
 <div class="flex gap-1"><div cla
border border-rose-400/60 text-slate-900 t
flex items-center justify-center rounded b
-bold shadow-sm">3</div><div class="w-8 h-8
er-rose-400/60 text-slate-900 text-[11px] s
center justify-center rounded bg-amber-100/
-sm">3</div><div class="w-8 h-8 sm:w-9 sm:h
60 text-slate-900 text-[11px] sm:text-xs fo
fy-center rounded bg-amber-100/90 border bo
 <div class="w-8 h-8 sm:w-9 sm:h-9 flex item
e-900 text-[11px] sm:text-xs font-bold shad
 <div class="text-[9px] sm:text-

```

```

.....
 <tbody class="align-top">
 <tr class="border-t border-slate-100">
 <td class="py-2 pr-4 text-slate-500">
 <td class="py-2 pr-4 font-mono text-slate-500">
 <td class="py-2 font-mono text-slate-500">
 </tr>
 <tr class="border-t border-slate-100">
 <td class="py-2 pr-4 text-slate-500">
 <td class="py-2 pr-4 font-mono text-slate-500">
 <td class="py-2 font-mono text-slate-500">
 </tr>
 <tr class="border-t border-slate-100">
 <td class="py-2 pr-4 text-slate-500">
 <td class="py-2 pr-4 font-mono text-slate-500">
 <td class="py-2 font-mono text-slate-500">
 </tr>
 <tr class="border-t border-slate-100">
 <td class="py-2 pr-4 text-slate-500">
 <td class="py-2 pr-4">только
 <td class="py-2">min / max
 </tr>
 <tr class="border-t border-slate-100">
 <td class="py-2 pr-4 text-slate-500">
 <td class="py-2 pr-4 text-slate-500">
 <td class="py-2 text-rose-300">
 </tr>
 </tbody>
 </table>
</div>
<div class="bg-slate-900/40 rounded-lg border-slate-700 p-4">
 <p class="text-slate-300 text-sm mb-1">
 <p class="text-slate-400 text-sm mb-1">
летки дважды. Сумма ≠ идемпотентна.</p>
 <p class="text-slate-400 text-sm mb-1">
ации.</p>
 <p class="text-slate-400 text-sm">• Выб
>
</div>

```

```
.....
 <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg i
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>
 </div>
</details>
</div>
</div>
</section>`
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансируе
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
```

```
.....
<pre class="bg-slate-950 p-4 rounded-
// Zig, Zag
function rotate(x) {
 let p = x.parent;
 if (p.left === x) { // Right rotate
 p.left = x.right;
 x.right = p;
 } else { // Left rotate
 p.right = x.left;
 x.left = p;
 }
}</pre>
```

```
 </div>
 </details>
```

```
<div class="bg-indigo-950/60 p-6 rounded-xl">
 <h4 class="text-lg font-bold text-indigo">
```

```
 <div class="space-y-4 text-sm text-slate">
 <div>
 <strong class="text-white block">
 корень?
 <p>Оно поднимет узел, на котором
 ска. За счет сдвига этого листа в корень, д
 </p>
```

```
 </div>

 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключо
 находящимися на разных концах дерева, его
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит п
 </div>
```

```
 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоять
```

```

.....
 ый верхний угол отрезается дважды! Поэтому
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-sky-500 shadow-md">
 Аналогия 2: Сетка из вафельных
 </div>
 вынимается по включени
text-emerald-300">вставлять обратно: у
звернёшь), поэтому там (билет 2) печут торт
сетки $O(N \cdot M)$.</p>
 <p class="text-xs font-mono text-sl
ос $O(1)$ • только сумма</p>
 </div>
 </div>
 <div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-400 text-sm">
 <b class="text-emerald-300">суммы; раз
d. Сумму через sparse table считать нел
тной операции).</p>
 </div>
</div>
</div>
</section>
}
};

```

ФАЙЛ 27/96: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

```
import { Chapter } from '../../../types';
```

```

.....
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
</section>`
},
{
 id: "bellman-ford",
 title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="bellman-ford-content" class="mb-12 scroll-py-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15. Графы. Поиск кратчайшего пути. Форд-Беллман
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-rose-400/50">
 <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана (также известный как алгоритм Беллмана-Форда) — это алгоритм поиска кратчайшего пути в графе с весами ребер. Он работает на графах с отрицательными весами ребер, в отличие от алгоритма Дейкстры, который не работает с отрицательными весами. Алгоритм Форда-Беллмана работает за время $O(V \cdot E)$, где V — количество вершин, а E — количество ребер. Алгоритм основан на динамическом программировании и использует массив d для хранения кратчайших расстояний от источника до каждой вершины. Алгоритм работает следующим образом:
 <p class="text-xl font-mono text-white">1. Инициализация: для каждой вершины v установить $d[v] = \infty$, кроме источника s , для которого $d[s] = 0$.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4">Параноик считает, что все дороги в городе имеют отрицательную длину. Он проверяет ВСЕ возможные дороги $V-1$ раз подряд.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Отрицательный цикл
 </div>

```

```

.....
 <ul class="text-sm text-slate-300 space-y-4">
 Шаг k=0: Ты знаешь только
 Шаг k=1: Разрешаем пересадки
e> через путь <code class="text-indigo-300">
 Шаг k=2: Разрешаем пересадки
нутри этих путей уже могут быть пересадки ч
 ...
 Шаг k=N: Ты проверил все

 </div>
</div>
</div>
</section>`
 },
 {
 id: "johnson-algo",
 title: "17. Графы. Алгоритм Джонсона (Фокус с перевозками)",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
er border-amber-500 shadow-md">
 ⚡ Суть перевозаивания
 </div>

```

```
.....
 {проверяем через DSU) - берем в ответ.</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb">
с самых дешевых дорог в стране". Он строит
единую сеть.</p>
 </div>
 </div>
</div>
</div>
</section>`
},
{
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
```



```
.....
 <p class="text-slate-300 text-sm mb-1">
 изкому племени для объединения. К вечеру племени
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
</div>
</section>`
 }
};
```

```
.....
ФАЙЛ 28/96: src/data/tickets/ticket21_24.ts
КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМЕР: 1.5 КБ
.....
```

```
import { Chapter } from '../types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Префикс-функция (п) — Взгляд назад (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-1">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i.</p>
 </div>
 </div>
</div>
 `
 }
];
```

```

.....
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. <code>pi[1] = 0</code>
 ab: Из начала ("a") не получилось. <code>pi[2] = 0</code>
 abc: Опять мимо. <code>pi[3] = 0</code>
 abca: О! Начало ("a") совпало. <code>pi[4] = 1</code>
 abcab: Начало ("ab") совпало. <code>pi[5] = 2</code>
 abcabc: Начало ("abc") совпало. <code>pi[6] = 3</code>
 abcabcd: "d" всё испортило. <code>pi[7] = 3</code>

 </div>
</details>
<details class="bg-slate-900/40 rounded-lg p-4 font-sans text-slate-300 border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
 </div>
</div>
</section>
},

```

```
border-rose-500 shadow-md">
```

Аналогия 2: LLM Самокопиаст

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

В LLM Z-функция может применять  
[i]</code> (где <i>i</i> указывает назад на  
сама себя.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<p>Тот же пример: abcab</p>
```

```
<ul class="list-disc pl-5 space-y-2
```

```
<code>Z[0]</code> – не счита
```

```
<code>Z[1]</code> (bcab
```

```
[1] = 0</code>
```

```
<code>Z[2]</code> (cabcd
```

```

```

```
<code>Z[3]</code> (abcd
```

? Нет, в начале 'a', тут 'd'). Совпало 3 бу

```
<code>Z[4], Z[5], Z[6]</code>
```

```

```

```
</div>
```

```
</details>
```

```
<details class="bg-slate-900/40 rounded-lg
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

Код C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<pre class="bg-slate-950 p-4 rounded
```

```
int l = 0, r = 0;
```

```

.....
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет – мы падаем по суффиксной ссылке ("
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 ou
 r-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-
 <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch)
 }
 return t[v].link;
}

int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v),
 }
}

```

Универсальное пособие • полный исходный код

```
.....
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b0082; border-radius: 10px; box-shadow: 0 10px 15px #4b0082; position: relative; top: 3px; left: 4px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Это отличный переводчик на английский (Сведения о том, что В – NP, а сама В лежит в NP, то В – NP-полное предложение).
 </div>
 </div>
 </div>
</section>`
 }
};
```

ФАЙЛ 29/96: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">6. Графы. Представление, DFS, BFS
 </div>
 <div class="space-y-8">
 <div class="bg-white p-6 rounded-lg">
 <h3 class="text-2xl font-bold">Представление графов
 <p class="text-slate-600">Графы – это структуры данных, которые позволяют моделировать отношения между объектами. В данном разделе мы рассмотрим различные способы представления графов и методы их обхода (DFS, BFS).
 </div>
 <div class="bg-white p-6 rounded-lg">
 <h3 class="text-2xl font-bold">Обход графов
 <p class="text-slate-600">Обход графов – это процесс посещения всех вершин графа. Существует два основных метода обхода: Depth-First Search (DFS) и Breadth-First Search (BFS). В этом разделе мы рассмотрим их реализацию и применение.
 </div>
 </div>
</section>`
 }
];
```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Жадный алгоритм находит ближайшую
 ую вершину (например, минимальную по номеру) и пробует другой путь.
 </p>
 <ul class="text-xs text-indigo-300">
 Где используется:
 Топологическая сортировка (и DFS)
 Поиск мостов и точек сочленения
 Сильно связанные компоненты Косараса
 Поиск циклов и просто проверка связности

 </div>

```

```

 <!-- Аналогия BFS -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
 BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Концентрические волны.
 Работает через Очередь (FIFO). Продвигается от источника.
 </p>
 <ul class="text-xs text-emerald-300">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Алгоритм Ахо-Корасик (сборка слов)
 Алгоритм Форда-Фалкерсона / Поиск в ширину
 Двухнаправленный поиск для ускорения

 </div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 <b border-slate-700/50">

```

```

.....
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 },

```

```
.....

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
еская сортировка – это расписание, которое
р>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
 Душный мод: Код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-indigo-400 font-bold">
```



```
.....

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Сильная связность

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Вспомогательный граф
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Компоненты с сильной связностью
 и <i>ориентированного</i> графа, в котором вершины являются
 </div>
 </div>

 <p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» – Нет!
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300">
 Если у тебя есть цикл <code>А → Б → В → А</code>, то это цикл.
 Если ты добавишь вершину Г к этому циклу, то получишь
 четыре вершины – одна большая SCC.
 Правило: Если два цикла имеют хотя бы одну общую вершину, то
 они являются частью одной SCC.

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия: Интриги в офисе
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Сильная связность – это "клуб сильных связей". Если все
 вершины в одной SCC. Добавить курьера Гошу, который может
 идти наружу (к директору), но обратно в клуб не может.
 </p>
 </div>
 </div>
 </div>
 </div>
</div>
```

```

.....
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 и $f(u,v) > t(u,v)$.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
 border border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
 этот мост, экономикой обоих кусков придет
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутые",
 },

```

```
.....

 Представь город, где два района соед
 её перекроют (удалят вершину) – районы буд
g>хрупкость системы.
 </p>
</div>

<div class="bg-slate-900 p-6 rounded-lg border-2 border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-emerald-500 border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной комнате, соединенных между собой с помощью абелем (мостом). Сами свитчи – это точки со
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border-2 border-emerald-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400 font-bold">Особый случай: Если у вершины нет детей (то выше неё прыгать некуда). Поэтому для неё у него больше 1 независимого ребенка
 </p>
 <div class="bg-slate-950 p-4 rounded-lg border-2 border-emerald-500 shadow-md">
 <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
```

```

{
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">13. Графы. Эйлеров цикл
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">Эйлеров цикл
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Эйлеров цикл — это цикл, который проходит по всем ребрам графа ровно один раз.
 <p class="text-xl font-mono text-white">Граф, в котором существует Эйлеров цикл, называется эйлеровым графом.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрывая
 </div>
 <p class="text-slate-300 text-sm mb-4">Если вы пытаетесь нарисовать фигуру, не отрывая пера от бумаги, то вы можете заметить, что не все фигуры можно нарисовать. Например, вы не можете нарисовать фигуру, у которой есть «зашел-вышел» (то есть, вы зашли в вершину и вышли из нее, но не зашли в нее снова). Это происходит потому, что в такой вершине нечетное число ребер, и вы не можете пройти по всем ребрам ровно один раз. Если же в каждой вершине четное число ребер, то вы можете пройти по всем ребрам ровно один раз. Это и есть Эйлеров цикл.
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
};

```

```

.....
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);
edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 useVizStepSync(currentStepIndex, setCurrentStepIndex,
 const v = vizNumber(vars.v);
 if (v === null) return null;
 return steps.map((step, index) => ({ step, index }
));
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];

```

```

currentLow[v] = Math.min(currentLow[v], currentLow[to]);

recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}],
 v,
);

if (currentLow[to] >= currentTin[v] && p !== ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v},
 v,
);
}
}
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

recordStep("Алгоритм завершен. Найдены все точки сочленения.");

```



```

.....
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-1
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-even
 >
 {node.label}
 </text>

 {isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r + 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "."}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"

```



```

.....
 <div className="flex items-center gap-2" >
 <div className="w-3 h-3 rounded-full bg-rose-500" >
 Текущая
 </div>
 </div>
 <div className="flex items-center gap-2" >
 <div className="w-8 h-1 bg-rose-500 border" >
 Мост
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium" >
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => (
 <div
 key={idx}
 className={`text-xs p-2 rounded $
xt-slate-500`} `}
 >
 {step.desc}
 </div>
))}
 </div>
 </div>
</div>

<div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded" >
 <div
 className="bg-rose-500 h-full transition"
 style={{
 width: `${(currentStepIndex / Math.max(
 />

```

```

.....
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman ford(graph, start'

```

```

.....
simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
 updated ? ` Релаксация успешна! dist[${edge.u}] = dist[${edge.u}] + edge.w`,
 activeCodeLine: updated ? 6 : 5,
});
}

if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: ` Ранний выход после итерации ${iter}.`,
 activeCodeLine: 10
 });
 break;
}
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} → ${edge.v} обновляет ${edge.v}.`,
 activeCodeLine: 9,
 });
 break;
 }
 }
}

if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,

```

```

.....
 }
 : baseStep;

return {
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 <div className="flex flex-wrap items-center justify"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white"
 <p className="text-sm text-blue-300">Полный
 </div>
 </div>

 <div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg border"
 <button onClick={() => setHasCycle(false)}
 bg-blue-600 text-white' : 'text-slate-400'
 <button onClick={() => setHasCycle(true)}
 r gap-1 ${hasCycle ? 'bg-rose-600 text-white'
 </div>
 <button onClick={() => { setIsPlaying(false);
 g-slate-600 text-white px-3 py-2 rounded-lg
 <button onClick={() => setIsPlaying(!isPlaying
 sition-colors text-white ${isPlaying ? 'bg-
 <button onClick={() => { setIsPlaying(false);
 ; }} className="flex items-center gap-1 bg-i
 s">⏏ </button>
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-blue-950/20
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue

```

```

.....
 asNegativeCycle && isCyclePart) ? 4 : 2} ma
 g ? '4,4' : 'none'} />
 <circle cx={({uNode.x + vNode.x) / 2}
Checking ? '#f59e0b' : isNeg ? '#f43f5e' :
 <text x={({uNode.x + vNode.x) / 2} y={
8fafc'} fontSize="11" fontWeight="bold" tex
 </g>
 });
}}}

{nodes.map((node) => {
 const dist = currentStep.distances[node.id]
 const isCheckingNode = currentStep.checkin
 node.id);

 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
 ckingNode ? '#7dd3fc' : '#64748b'} strokeWi
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill=
 ight="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill=
 split(' '){0]}</text>
 </g>
);
 }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald

```

```

 </div>
 </div>
 </div>

 { /* Код алгоритма */ }
 <div className="w-full lg:w-1/2 bg-slate-900/60" >
 <div>
 <h4 className="text-lg font-bold text-slate" >
 <div className="bg-slate-950/80 rounded-lg" >
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-amber
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
))}
 </div>
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 32/96: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'

```

```

const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },

```

```

.....
visited.add(0);
steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по
});

while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}
 });

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
 });

 const neighbors = ADJ[curr];
 if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });
 }
 }
 }
}

```

```

const [stepIdx, setStepIdx] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

const step = STEPS[stepIdx];

const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
}, []);

// @ts-expect-error зарезервировано под кнопку «назад»
const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
}, []);

useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
}, [isPlaying, stepIdx, handleNext]);

return (
 <div className="bg-slate-900 border-2 border-slate-700 p-2">
 <div className="flex flex-col md:flex-row items-center justify-between">
 <div>
 <h3 className="text-2xl font-bold text-white">
 Ан
 </h3>
 <p className="text-slate-400 text-sm mt-1">Вн
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded">
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="C6poc"

```



```

 />
 });
 }}}

 { /* Вершины */
 {NODES.map(n => {
 const isCurrent = step.currentNode === n.id;
 const isInQueue = step.queue.includes(n.id);
 const isVisited = step.visited.includes(n.id);

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;
 void scale;

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-transform"
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fill={fill} stroke={stroke}
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}

```

```

.....
 const isActive = step.activeLine === cl
 return (
 <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5' : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
);
 })}
</div>
</div>

<div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (
 step.queue.map((qId, i) => (
 <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

))
)}
</div>
 <div className="mt-2 text-sm text-amber-30
ms-center justify-center">
 "{step.logs}"
 </div>
</div>
</div>
</div>
</div>
);

```

```
.....
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border`>

 </div>
 <p className={`text-center text-xs mt-2 font-bold`}>{info.title}</p>
 </div>
);
}
```

.....  
ФАЙЛ 34/96: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» — как на Codecademy
```

```
 * Компактный вариант живёт в шапке главы, крупный — под ней
```

```
 */
```

```
export const ChapterNav: React.FC<Props> = ({ prev, next, onGo, compact }) => {
 if (compact) {
 return (

```

```

.....
 <ChevronLeft className="w-3.5 h-3.5" /> Пре
 </div>
 <div className="text-sm font-bold text-slate-1
 </button>
) : (
 <div className="hidden sm:block" />
 })

 {next && {
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg
 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end
 text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3
 </div>
 <div className="text-sm font-bold text-slate-1
 </button>
 })
</nav>
);
};

```

ФАЙЛ 35/96: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....
import React, { useCallback, useEffect, useRef, useState } from 'react';
import { Check, Download, Loader2, Maximize2, MousePointer } from 'lucide-react';
import type { Chapter } from '../types';
import { useTermHints } from '../hooks/useTermHints';
import { TermPopover, type HintAnchor } from '../hints/TermHint';
import { getViz } from '../vizRegistry';
import { VizChapterContext } from '../data/vizStepBus';

```

```
.....
 /** Перейти во вкладку компилятора – там уже подставлен код
 onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter,
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");
```

```
 useTermHints(contentRef, [chapter.id]);
```

```
 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
 }, []);
```

```
 const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(anchor), 2500);
 }, [cancelHide]);
```

```
 useEffect(() => {
 setAnchor(null);
 setSaving("idle");
 }, [chapter.id]);
```

```
 const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
```

.....

```
const viz = getViz(chapter.id);
```

```
return {
```

```
 <
```

```
 <article className="chapter-body">
```

```
 /* панель загрузки: сохранить именно эту страницу
```

```
 <div className="flex flex-wrap items-center gap-
```

```
 <button
```

```
 onClick={saveHtml}
```

```
 disabled={saving === "work"}
```

```
 title="Сохранить эту тему одним автономным HTML-файлом"
```

```
 className="flex items-center gap-1.5 text-x-
```

```
 300 hover:text-white hover:border-emerald-500
```

```
 >
```

```
 {saving === "work" ? (
```

```
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
```

```
) : saving === "done" ? (
```

```
 <Check className="w-3.5 h-3.5 text-emerald-500" />
```

```
) : (
```

```
 <Download className="w-3.5 h-3.5" />
```

```
)}
```

```
 {saving === "done" ? "Файл сохранён" : "Скачать"}>
```

```
 </button>
```

```
 <button
```

```
 onClick={() => window.print()}
```

```
 title="Распечатать или сохранить в PDF с помощью Printium"
```

```
 className="flex items-center gap-1.5 text-x-
```

```
 300 hover:text-white hover:border-indigo-500" />
```

```
 >
```

```
 <Printer className="w-3.5 h-3.5" /> Печать
```

```
 </button>
```

```
 Всего: {viz.length} страниц
```

```
 </div>
```

```
 <div
```

```
 ref={contentRef}
```

```
 className="prose prose-invert max-w-none">
```

```

.....
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
 })
 </div>
 </div>
 {viz.hint && <p className="text-xs text-slate-500">
 <VizChapterContext.Provider value={chapter}>
 <viz.Component />
 </VizChapterContext.Provider>
 </section>
 })

 <RelatedTickets chapterId={chapter.id} onGo={onGo} />

 <ChapterNav prev={prev} next={next} index={index} />
</article>

<TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
/>
</>
);
};

```

```

.....
 adj[v].push(u);
 });
 return adj;
})();

const COLORS = [
 { name: "красный", hex: "#ef4444" },
 { name: "синий", hex: "#3b82f6" },
 { name: "зелёный", hex: "#22c55e" },
 { name: "жёлтый", hex: "#eab308" },
];

const CONFLICT = "#e11d48";

/** Жадная раскраска по возрастанию номеров — честный у
function greedy(): number[] {
 const color: number[] = NODES.map(() => -1);
 for (let v = 0; v < NODES.length; v++) {
 const used = new Set(ADJ[v].map((u) => color[u]).filter(u => color[u] !== -1));
 let c = 0;
 while (used.has(c)) c++;
 color[v] = c;
 }
 return color;
}

export function ColoringViz() {
 const [colors, setColors] = useState<number[]>(NODES.map(() => -1));
 const [brush, setBrush] = useState(0);

 // — Синхронизация с Python-компилятором.
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveVLabel = vizString(vars?.v);
 const liveVNum = vizNumber(vars?.v);
 const liveColorRec = vizRecord(vars?.color) ?? vizRecord({});
 const liveColorArr = vizArray(vars?.color) ?? vizArray([]);
 const liveK = vizNumber(vars?.k);

```



```

.....
 const cu = shownColors[u];
 const cv = shownColors[v];
 if (cu >= 0 && cu === cv) {
 bad.add(u);
 bad.add(v);
 }
 });
 return bad;
}, [shownColors]);

const usedColors = useMemo(
 () => new Set(shownColors.filter((c) => c >= 0)).size,
 [shownColors]
);

const paint = (id: number) => {
 const next = [...colors];
 next[id] = next[id] === brush ? -1 : brush;
 setColors(next);
};

return (
 <div className="bg-slate-800/90 p-5 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-center">
 <div>
 <h3 className="text-lg font-bold text-white">
 <p className="text-xs text-slate-400">
 Смежные вершины не должны совпадать по цвету
 </p>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => setColors(greedy())}
 className="flex items-center gap-1 bg-indigo-500 text-white"
 >
 <Wand2 className="w-3.5 h-3.5" /> Жадная раскраска
 </button>
 <button
 onClick={() => setColors(greedy2())}
 className="flex items-center gap-1 bg-indigo-500 text-white"
 >
 <Wand2 className="w-3.5 h-3.5" /> Жадная раскраска 2
 </button>
 </div>
 </div>
 </div>
 </div>
);

```

```

 });
 }}}
</svg>
{compilerLinked && {
 <div className="absolute top-2 right-2 bg-emerald-300 z-20">
 Python красит: {liveVLabel ?? ({liveVNum
 </div>
}}
{conflicts.size > 0 && {
 <div className="absolute bottom-2 left-1/2
 -[11px] text-rose-300 z-20">
 ▲ Конфликт: смежные вершины одного цвета
 </div>
}}
</div>

<div className="space-y-3">
 <div className="bg-slate-950 rounded-xl border"
 <div className="text-[10px] uppercase tracking
 <Palette className="w-3 h-3" /> Кисть
 </div>
 <div className="grid grid-cols-4 gap-2">
 {COLORS.map((c, i) => {
 <button
 key={c.name}
 onClick={() => setBrush(i)}
 title={c.name}
 className={`h-8 rounded-lg border-2 t
 opacity-70 hover:opacity-100`} `}
 style={{ backgroundColor: c.hex }}
 />
)}}
 </div>
 </div>
 <div className="bg-slate-950 rounded-xl border"
 <div className="text-[10px] uppercase tracking
 <div className={`text-2xl font-extrabold fo

```

```

.....
 cls: ClassId;
 x: number;
 y: number;
 npComplete?: boolean;
}

const PROBLEMS: Problem[] = [
 { id: "sort", name: "сортировка\n $O(n \log n)$ ", cls: "P", x: 150, y: 150, npComplete: false },
 { id: "mst", name: "остов (Краскал)", cls: "P", x: 150, y: 175, npComplete: false },
 { id: "2sat", name: "2-SAT", cls: "P", x: 115, y: 185, npComplete: false },
 { id: "shortpath", name: "Дейкстра / Флойд", cls: "P", x: 150, y: 200, npComplete: false },
 { id: "sat", name: "3-SAT", cls: "NP", x: 330, y: 120, npComplete: true },
 { id: "vc", name: "Vertex Cover", cls: "NP", x: 385, y: 150, npComplete: true },
 { id: "ham", name: "Гамильтонов цикл", cls: "NP", x: 385, y: 175, npComplete: true },
 { id: "knap", name: "Рюкзак (точный)", cls: "NP", x: 385, y: 200, npComplete: true },
 { id: "circuit", name: "QBF / игра на формуле", cls: "EXPTIME", x: 385, y: 225, npComplete: true },
 { id: "chess", name: "обобщённые шахматы", cls: "EXPTIME", x: 385, y: 250, npComplete: true },
];

const REDUCTIONS: [string, string, string][] = [
 ["sat", "vc", "3-SAT $\leq_p$ VC"],
 ["sat", "ham", "3-SAT $\leq_p$ Гамильтон"],
 ["vc", "knap", "VC $\leq_p$ Рюкзак"],
];

const CLASS_META: Record<ClassId, { title: string; color: string; note: string }> = {
 P: { title: "P", color: "#34d399", note: "решаются за полиномиальное время." },
 NP: { title: "NP", color: "#fbbf24", note: "решение можно проверить быстро." },
 PSPACE: { title: "PSPACE", color: "#818cf8", note: "нужно полиномиальная память." },
 EXPTIME: { title: "EXPTIME", color: "#f472b6", note: "нужно экспоненциальное время." },
};

const CLASS_INFO: Record<ClassId, string> = {
 P: "P — класс задач, у которых ЕСТЬ быстрый (полиномиальный) алгоритм.",
 NP: "NP — задачи, у которых быстрой проверки ПРОЧЕСТЬ решение недостаточно. Но",
 P $\subseteq$ NP: "нашёл быстро $\rightarrow$ и проверяй быстро.",
 PSPACE: "PSPACE — хватает полиномиальной ПАМЯТИ (временная сложность не важна).",
 ветку рекурсии.",
};

```

```

.....

const infoText = (() => {
 if (compilerLinked && liveTask) {
 const p = PROBLEMS.find((q) => q.name.toLowerCase() === selectedP)
 if (p) return `${p.name.replace("\n", " ")} ∈ ${p.cls}`
 }
 if (compilerLinked && liveCls) {
 const cls = (activeCls ?? "NP") as ClassId;
 return CLASS_INFO[cls];
 }
 if (selectedProblem) {
 const p = PROBLEMS.find((q) => q.id === selectedP)
 return `${p.name.replace("\n", " ")} ∈ ${p.cls}${p.cls ? " " : ""}${p.cls}`;
 }
 if (activeCls) return CLASS_INFO[activeCls as ClassId];
 return "Кликни на класс или задачу. «A ≤p B» = «A не ≤p B»";
})();

const reset = () => {
 setSelected(null);
 setSelectedProblem(null);
};

return (
 <div className="bg-slate-800/90 p-5 rounded-2xl border" >
 <div className="flex flex-wrap items-center justify-center" >
 <div>
 <h3 className="text-lg font-bold text-white">Задача
 <p className="text-xs text-slate-400">Стрелки
 </div>
 <button
 onClick={reset}
 className="flex items-center gap-1 bg-slate-700" >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>
 </div>
 </div>
)

```

```

.....
 <text x={mx} y={my} textAnchor="middle"
text>
 </g>
);
 }}}
<defs>
 <marker id="arrowhead" markerWidth="7" ma
 <polygon points="0 0, 7 2.5, 0 5" fill=
 </marker>
</defs>

{PROBLEMS.map((p) => {
 const dim = activeCls !== null && activeC
 const hit = problemHit(p);
 return (
 <g key={p.id} onClick={() => setSelecte
 <rect
 x={p.x - 46} y={p.y - 13} width={92
 fill={hit ? "#c2410c" : p.npComple
 stroke={hit ? "#fb923c" : CLASS_MET
 strokeWidth={hit ? 2.5 : 1.2}
 />
 {p.name.split("\n").map((line, li, ar
 <text key={li} x={p.x} y={p.y + 4 +
ontWeight="600">
 {line}
 </text>
)}}
 </g>
);
 }}}
</svg>
</div>

<div className="space-y-3">
 <div className="grid grid-cols-2 gap-2">
 {(Object.keys(CLASS_META) as ClassId[]).map
 <button

```



```

.....
 comp[s] = count - 1;
 push({ queue: [...queue], current: s, scanning: null,
 аргумент с неё.` });
 while (queue.length) {
 const v = queue.shift()!;
 push({ queue: [...queue], current: v, scanning: null,
 й.` });
 for (const to of ADJ[v]) {
 if (comp[to] === null) {
 comp[to] = comp[v];
 queue.push(to);
 push({ queue: [...queue], current: v, scanning: null,
 {comp[v]}, в очередь.` });
 } else {
 push({ queue: [...queue], current: v, scanning: null,
 });
 }
 }
 }
 push({ queue: [], current: null, scanning: null, log: null });
 }
 push({ queue: [], current: null, scanning: null, log: null });
 return frames;
}

```

```
const FRAMES = buildFrames();
```

```

export function ComponentsViz() {
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);

```

```

 useEffect(() => {
 if (!playing) return;
 if (step >= FRAMES.length - 1) {
 setPlaying(false);
 return;
 }

```

```

 }
 if (liveCompByLabel && String(n.id) in liveCompByLabel)
 const c = vizNumber(liveCompByLabel[String(n.id)])
 if (c !== null) return c;
 }
 return frame.comp[n.id];
 };
 const liveQueueLabels = (liveQ
 ? liveQ.map((x) => (typeof x === "string" ? x : vizNumber(x)))
 : null
)?.filter(Boolean);
 const displayCount = liveCount ?? frame.count;
 const displayLog = compilerLinked
 ? `Компилятор: v=${liveVLabel ?? liveVNum ?? "-"}
 ES[q].label)).join(", ")`
 : frame.log;

 const shownCurrent = compilerLinked ? liveCurrent : frame.current;
 const shownScanning = compilerLinked ? null : frame.scanning;
 const shownQueue: number[] = liveQueueLabels
 ? liveQueueLabels
 .map((lab) => NODES.findIndex((n) => n.label === lab))
 .filter((i) => i >= 0)
 : frame.queue;

 return (
 <div className="bg-slate-800/90 p-5 rounded-2xl border" >
 <div className="flex flex-wrap items-center justify-center" >
 <div>
 <h3 className="text-lg font-bold text-white">
 <p className="text-xs text-slate-400">
 Каждой вершине присваивается номер компонента
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => { setPlaying(false); setStep(0); }}
 className="flex items-center gap-1 bg-slate-700 text-white px-2 py-1 rounded">
 Пауза
 </button>
 <button
 onClick={() => { setPlaying(true); setStep(0); }}
 className="flex items-center gap-1 bg-slate-700 text-white px-2 py-1 rounded">
 Продолжить
 </button>
 <button
 onClick={() => { setPlaying(false); setStep(0); }}
 className="flex items-center gap-1 bg-slate-700 text-white px-2 py-1 rounded">
 Сброс
 </button>
 </div>
 </div>
 </div>
);
}

```



```

<line
 key={i}
 x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 stroke={sameComp ? (COMP_COLORS[(live
 strokeWidth={sameComp ? 3.5 : 2}
 opacity={sameComp ? 0.9 : 0.6}
/>
);
}}}
{NODES.map((n) => {
 const c = liveCompOf(n);
 const isCurrent = shownCurrent === n.id;
 const isScanned = shownScanning === n.id;
 const inQueue = shownQueue.includes(n.id);
 const color = c === null ? "#334155" : COMP_COLORS[c];
 return (
 <g key={n.id}>
 {(isCurrent || isScanned) && (
 <circle cx={n.x} cy={n.y} r={18} fill={color} stroke={color}
e-pulse" />
)}
 <circle cx={n.x} cy={n.y} r={13} fill={color} stroke={color} />
 <text x={n.x} y={n.y + 4} textAnchor="left">
 {n.label}</text>
 <text x={n.x} y={n.y - 18} textAnchor="center">
 {c === null ? "-" : `c${c}`}
 </text>
 </g>
);
}}}
</svg>
<div className="absolute top-2 left-2 bg-slate-500 z-20">
 КОМПОНЕНТ:
</div>
{compilerLinked && (
 <div className="absolute top-2 right-2 bg-e

```

```

 </div>
 </div>
 </div>
 </div>
);
}

```

ФАЙЛ 39/96: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "react-";

```

```

interface CodeLine {
 line: string;
 highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
 { line: " // ЭТО МОСТ!", highlight: "normal" },

```

```

.....
};

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 :=",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 },
 {

```

```

.....
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) — не мост
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) — не мост
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) — не мост
 activeCodeLine: [13,14,15,16,17]
},
{
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) — не мост
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D)
 activeCodeLine: [11,13,16,17,18]
},
{
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм DFS завершён
 activeCodeLine: []
}

```

```

.....
return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <div className="flex items-center gap-1 bg-slate-700 rounded"
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
 disabled={dfsIdx === 0}
 className="p-1.5 hover:bg-slate-700 rounded"
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px] ${
 dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
 }`}
 >
 {dfsAuto ? <<Pause className="h-3 w-3" /> : <<Play className="h-3 w-3" />}
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length-1); }}
 disabled={dfsIdx === DFS_STEPS.length-1}
 className="p-1.5 hover:bg-slate-700 rounded"
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
 className="p-1.5 hover:bg-slate-700 rounded"
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>
 <div className="grid grid-cols-1 md:grid-cols-5 gap-4">
 /* Graph visualization */
 <div className="md:col-span-3 bg-slate-950 rounded"
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate_1px)]"
 >
 <svg className="w-full h-[280px] z-10 relative"
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id === e.u);
 const v = GRAPH_NODES.find(n => n.id === e.v);
 const bridge = step.bridges.includes(e.key);

```

```

 "transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">
 { l:{step.low[n.id]|| "?"}}</text>
 </>
)}
</g>;
}})
</svg>
<div className="flex items-center justify-between">
 Step {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-12 rounded-lg">
 <div className="bg-indigo-600 h-full transition-all duration-300">
 </div>
 </div>
</div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg border border-slate-800">
 <div className="bg-slate-900 px-3 py-1.5 text-slate-400">
 </div>
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono leading-relaxed">
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includes(cl.line)
 return <div key={i}
 className={`px-1 transition-all duration-300 ${
 hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"
 }`}>
 {cl.line} || ' '
 </div>;
 })}
 </pre>
 </div>
 </div>
</div>

```

ФАЙЛ 40/96: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
```

```

.....
const prev: Record<string, string | null> = { A: null };

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из ПИ,
 activeCodeLine: 2,
});

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
 });

 // Граф неориентированный: при релаксации смотрим
 const neighbors = edges.filter(e => e.u === u || e.v === u);
 for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited }
 });
 }
}

```



```

.....
 }, {isPlaying, currentStepIndex, steps.length});

 const baseStep = steps[currentStepIndex] || null;

 if (!baseStep) return <div className="text-white">3arg
 const vars = runtime?.variables;
 const liveU = vizString(vars?.u);
 const liveV = vizString(vars?.v);
 const liveDist = vizRecord(vars?.dist);
 const livePrev = vizRecord(vars?.prev);
 const liveDone = vizArray(vars?.done) ?? vizArray(var
 const compilerLinked = liveU !== null || liveV !== nu
 const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: l
 distances: Object.fromEntries(nodes.map((node) =>
 previous: Object.fromEntries(nodes.map((node) =>
 visited: Object.fromEntries(nodes.map((node) =>
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV
 }
 : baseStep;

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl bor
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justi
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-whit

 </div>
 <div>
 <h3 className="text-2xl font-bold text-whit
 <p className="text-sm text-indigo-300">Полн
 </div>
 </div>
 </div>

```

```

<marker id="arrow-dh-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#47556"
</marker>
<marker id="arrow-dh-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0"
</marker>
<marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b98"
</marker>
</defs>
{/* Рёбра */}
{edges.map({edge, idx} => {
 const uNode = nodes.find((n) => n.id ===
 const vNode = nodes.find((n) => n.id ===
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &
 (currentStep.checkingEdge.v === edge.v &

 const isOptimal = currentStep.previous[ed

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-a
 else if (isOptimal) marker = 'url(#arrow-

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : is
 strokeWidth={isChecking ? 5 : isOpt
 markerEnd={marker}
 className="transition-all duration-
 strokeDasharray={isChecking ? '4,4'
 />
 <circle
 cx={({uNode.x + vNode.x) / 2} cy={({u
 r="12" fill="#1e293b"

```

.....

```

 </text>
 </g>
);
 }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded-2xl" >
 <p className="font-semibold text-amber-400" >
 <p className="min-h-[40px] flex items-center" >
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl" >
 <h4 className="text-lg font-bold text-emerald-400" >
 <div className="space-y-2.5 flex-1 overflow-y-hidden" >
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u
 return (
 <div key={u} className={`p-3 rounded-lg ${
 isCurrentNode ? 'bg-emerald-900/60 text-slate-300' : 'bg-slate-300'
 }`} >
 <div className="flex items-center gap-2" >

 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2" >
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep.currentNode === edge.v
 return (
 <span key={idx} className={`px-2 py-0.5 rounded ${
 isCheckingThis ? 'bg-amber-900/60 text-slate-300' : 'bg-slate-300'
 }`} >
 {u}

 {edge.v}
 {edge.w}
)
 })}
 </div>
 </div>
)
 })}
 </div>

```

```

.....
 <td className="py-2.5 px-3 flex"
 <span className={`w-2 h-2 rounded
 {n.name}
 </td>
 <td className="py-2.5 px-3 font"
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text"
 {prev || '-'}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60"
 <div>
 <h4 className="text-lg font-bold text-slate"
 <div className="bg-slate-950/80 rounded-lg p-4"
 {codeLines.map((item) => {
 const isActive = currentStep.activeCodeLine === item.line;
 return (
 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text-white' : ''
 }`>
 <span className="text-slate-600 select-none"
 <span className="whitespace-pre flex"
 {isActive && <span className="text-slate-600"
 }
 </div>
);
 })}
 </div>
 </div>

```

```

.....
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState<boolean>(f
const [speed] = useState<number>(800);
const [activeTab, setActiveTab] = useState<'table' |

```

```

useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });

```

```

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `< Кэш-хит! fibМемо(${n}) уже посчитано
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

```

```

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base case',

```

```

.....
 desc: ` Расчет завершен! fibMemo(${targetN}) = $
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo
 ? Object.fromEntries(Object.entries(liveMemo).flatMap(
 (entry) => {
 const number = vizNumber(entry.value);
 return number !== null ? [1] : [[Number(entry.key), null]];
 }
))
 : baseStep?.memoState || {};
const currentStep: DPStep | null = baseStep && compilerLinked
 ? {
 ...baseStep,
 n: liveK ?? baseStep.n,
 memoState,
 }
 : null;

```

```

.....
 onClick={() => { setIsPlaying(false); set
 className="p-2 text-slate-400 hover:text-
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-
 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:
 : 'bg-emerald-600 text-white hover:bg
 }`}
 >
 {isPlaying ? <Pause className="w-4 h-4" />
 {isPlaying ? 'Пауза' : 'Пуск'}}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.leng
 className="p-2 text-slate-400 hover:text-
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
</div>
</div>

{/* DP Table View */}

```

```

 </div>
)}
 </div>

 { /* Tabs */ }
 <div className="flex items-center gap-2 border-b
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }`}
 >
 <Eye className="w-4 h-4" />
 Стен-бай-стен состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }`}
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

```

```

 { /* Tab Content */ }
 <div className="min-h-[320px] flex flex-col justi
 {activeTab === 'table' && {
 <div className="bg-slate-950 p-6 rounded-2xl i
 <h5 className="font-bold text-white text-sm
 </spa
 </h5>
 <p className="text-sm text-slate-400">

```



```

.....
 {currentStep?.desc}
 </div>
 </div>
)}
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800" >
 <div className="flex items-center justify-between" >
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden" >
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 ></div>
 </div>
</div>
);
}

```

ФАЙЛ 42/96: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 100

```

import { useMemo, useState } from "react";
import { Undo } from "lucide-react";
import { useVizRuntime, vizArray, vizString } from "../viz-runtime";

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },

```

```

.....
 const keys: string[] = [];
 for (let i = 1; i < livePath.length; i++) {
 const a = livePath[i - 1];
 const b = livePath[i];
 if (C1_EDGES.some((e) => (e.u === a && e.v === b))) {
 keys.push(`${Math.min(a, b)}-${Math.max(a, b)}`);
 }
 }
 return keys;
}
return c1VisitedEdges;
}, [livePath, c1VisitedEdges]));

const degrees = useMemo(
 () => C1_NODES.map((n) => ({ label: n.label, deg: C1_NODES[n.label].deg })),
 []
);

const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы пройти все ребра!");
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери другую вершину!");
 return;
 }
 const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощает повторений!");
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];

```

```

.....
 stroke={used ? "#6366f1" : "#475569"}
 strokeWidth={used ? 4 : 2}
 className="transition-all duration-300" />
))}
 {C1_NODES.map(n => {
 const isLast = shownPath[shownPath.length-1] === n.id;
 const visited = shownPath.includes(n.id);
 return <g key={n.id} className="cursor-pointer" >
 {isLast && <circle cx={n.x} cy={n.y} r={11} />
 <circle cx={n.x} cy={n.y} r={11} />
 fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
 stroke={isLast ? "#fff" : visited ? "#6366f1" : "#6366f1"}
 strokeWidth={2.5}
 className="transition-all duration-200" />
 <text x={n.x} y={n.y+4} textAnchor="middle">
 {n.label}</text>
 </g>
)}
 </svg>
 <div className="absolute top-2 left-2 bg-slate-500 p-2" >
 Ребёр: {shownVisited.length}/{C1_EDGES.length}
 {compilerLinked &&
 {compilerLinked}
 }
 </div>
 <div className="absolute bottom-2 left-2 bg-slate-500 p-2" >
 Степени: {degrees.map((d) => `${d.label}: ${d.degree}`)}
 </div>
 </div>

 <div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
 : "bg-slate-900/50 border-slate-700 text-slate-50"
 }`}>
 {c1Msg}
 </div>

```

```
.....
question: 'Почему алгоритм Дейкстры впадает в ступор?'
options: [
 'Потому что он написан только для положительных чисел',
 'Он жадный и считает, что каждый следующий шаг должен
 закрыть все открытые вершины.',
 'Отрицательные ребра создают гравитационный коллапс',
 'Дейкстра просто не любил отрицательные балансы на вершинах',
],
correctAnswer: 1,
explanation: 'Именно! Дейкстра уверен: если ты приедешь в город,
 для чего нужен алгоритм Беллмана-Форда.'
},
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки?'
 options: [
 'Он использует меньше памяти благодаря коммунитарности',
 'Он умеет работать с отрицательными весами без циклов',
 'Он позволяет выполнять шаги слияния колхозов (коммуны)',
 'Он был разработан в Токио и поэтому самый быстрый'
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая коммуна
 может параллельно делить вычисления на GPU или многопроцессорных кластерах'
},
{
 id: 4,
 question: 'В чем главная суть Динамического программирования?'
 options: [
 'Писать код очень быстро, динамично нажимая на клавиши',
 'Запоминать (кешировать) результаты уже решенных подзадач',
 'Использовать динамическую типизацию как в Python',
 'Постоянно менять требования к проекту в процессе разработки'
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизация'
},
{
```

```

 setShowResults(true);
 }
};

const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
};

if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 py-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">Результаты
 <p className="text-slate-400 text-sm mb-6">Ты набрал {score} из {quizQuestions.length} баллов.
 </p>
 <div className="bg-slate-950 border border-slate-800 p-6">
 <p className="text-slate-400 text-sm uppercase font-medium">Оценка:
 <p className="text-5xl font-black text-indigo-500">{score}
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскрыватель! '
 : score >= 3
 ? ' Отличный результат! Главные концепции ты освоил! '
 : ' Стоит повторить главы пособия и еще раз попробовать! '
 }
 </p>
 </div>
 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4 py-2 text-white font-medium border border-white/20 rounded">
 Перезапустить
 </button>
 </div>
);
}

```

```

.....
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-ro
 } else {
 optionStyle = "bg-slate-800/40 border-s
 }
 }
}

return {
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-rose-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full border border-indigo-500 bg-indigo-500 text-white"}>
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div className="flex-1 text-sm md:text-base">
 {option}
 </div>
 </div>
);
 }
}
</div>
</div>

{isAnswered && (
 <div className="bg-slate-950 border border-slate-800 p-4">
 <div className="flex items-center gap-2 mb-2">
 <AlertCircle className="w-4 h-4" />

```

```

.....
 log: string; activeCodeLine: number;
}

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 // k/i/j/d приходят напрямую; номер строки произвольн
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

 const adjList: { [key: number]: { v: number; w: numbe
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],

```

```

.....
 if (dist[i][k] !== 999 && dist[k][j] !== 999 &&
 dist[i][j] > dist[i][k] + dist[k][j]) {
 dist[i][j] = dist[i][k] + dist[k][j];
 simulationSteps.push({
 k: k, i: i, j: j, matrix: dist.map(r => [...r]),
 log: ` Улучшение пути [${i}]→[${j}] через [${k}]`,
 activeCodeLine: 7,
 });
 }
}
}
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ` Все пары отработаны. Алгоритм Флойда завершён`,
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Зарплата...</div>;

const vars = runtime?.variables;

```



```

.....
 text-sm"> Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying
 -amber-600' : 'bg-emerald-600')}`}>{isPlaying
 <button onClick={() => { setIsPlaying(false);
 ; }} className="px-3 py-2 bg-emerald-600 te
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 m
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-indigo-950/2
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indi
 ont-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ?
 </div>

 <svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-fw-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
 th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
 </defs>
 {Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8
 return (
 <g key={` ${u} - ${edge.v} `}>

```

```

.....
 <h4 className="text-lg font-bold text-emerald"
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-ambe
 isK ? 'bg-indigo-900/40 border-ind
 `}>
 <span className="font-bold text-whi
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== -
rentStep.j === edge.v);
 return (
 <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
 {edge.v} ({edge.w})

)
 })}
 </div>
);
 })}
 </div>
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 /* Матрица расстояний */
 <div className="w-full lg:w-1/2 bg-rose-950/10
 <div className="flex justify-between items-ce
 <h4 className="text-lg font-bold text-rose-
 </div>
 <div className="overflow-x-auto">

```

```

.....
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-amber
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
)}}
 </div>
 </div>
 </div>
</div>
</div>
);
}

```

.....  
ФАЙЛ 45/96: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....
import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...
import { Play, Pause, SkipForward, Undo, RefreshCw } from

```

```

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```

```

const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];

```

```

.....
 .stack], visitedNodes: Array.from(vis) }));
 dfs(u);
 }
}

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc:
 ueOrStack: [...stack], visitedNodes: Array.from
 }

 dfs(start);
 return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Нача
 ray.from(vis)] }));

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Д
 q], visitedNodes: [...Array.from(vis)] }));
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, de
 tack: [...q], visitedNodes: [...Array.from(
 }
 }
 }
 }
}

```



```

.....
 className={`str
<text x={n.x} y={n.y}
 className={`tex
 {n.label}
</text>

 {/* Ripple for BFS */}
 {current && mode === 'bfs' ? (
 <circle cx={n.x} cy={n.y}
 className="stroke-rose-500 fill-transparent
)}
 </g>
)
 }
 }
</svg>
</div>

{/* Structure View (Stack / Queue) */}
<div className="bg-slate-900 border border-rose-500 p-10">
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'bg-slate-900' : 'bg-slate-800'}
 border border-emerald-500`} >
 {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 </div>

 <div className="flex flex-col gap-2 p-5">
 {step.queueOrStack && step.queueOrStack.length > 0 ? (
 <div key={idx} className={`w-full flex items-center justify-between
 ${mode === 'dfs' ? 'bg-slate-900' : 'bg-slate-800'}
 text-emerald-200`} >
 {idx === (mode === 'dfs' ? step.queue.length : step.stack.length) ? (

 Узел: {item}

) : (

 {idx === (mode === 'dfs' ? step.queue.length : step.stack.length) ? (

 {item}

) : (

 {item}

)

)
 </div>
) : (
 <div>
 {mode === 'dfs' ? 'Стек пуст' : 'Очередь пуста'}
 </div>
)}
 </div>
</div>

```

```
.....
 {step.desc}
 </div>
 </div>
 </div>
 </div>
 </div>
);
 }
 }
```

.....  
ФАЙЛ 46/96: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

.....  
import React, { useState, useCallback } from 'react';  
import { Heart } from 'lucide-react';  
import { useVizRuntime, vizArray, vizNumber } from '../viz';

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
}
```





```

.....

 const nextHeap = heapInsert(heap, newPatient).map(x => {
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === pr
 const marked = nextHeap.map(x => x.id === (target?.id

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью ${finalPriority}

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);

```

```

.....
 setLog([' База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < visualHeap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
});

return (
 <div className="flex flex-col gap-6">
 { /* МНЕМОНИКА / ПРАВИЛО */ }
 <div className="bg-emerald-950/40 border border-e

```

```

.....
 <div className="flex flex-col items-center min-h-100" style={{width: 100%}}>

 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all cursor-pointer"
 >
 Везти
 </button>
 </form>

 {/* Забрать самого тяжелого */}
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 text-white px-4 disabled:cursor-not-allowed text-white font-mono transition-all cursor-pointer flex items-center justify-center"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white cursor-pointer border border-slate-700"
 >
 Сброс

```

```

.....
 height: isRoot ? 60 : 52,
 }}
 >

 {patient.priority}%

 { /* Tooltip */ }
 <div className="absolute bottom-full" >
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh" >
 <div className="text-emerald-400">
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

 <div className="w-full h-3 bg-gradient-to-r f" >
</div>

{ /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
<div className="lg:col-span-4 bg-slate-900 round" >
 <div className="absolute top-4 left-4 text-[1" >
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-" >
 {healingPatient ? (
 <div className="flex flex-col items-cente" >
 <div className="relative">

 </div>
 </div>
 </div>
 </div>
)
 }
 </div>
</div>

```

```
.....
 }}}
 </div>
 </div>
 });
};
```

.....  
ФАЙЛ 47/96: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

.....  
import React, { createContext, useContext, useEffect, use

/\*\* Общие примитивы для мини-визуализаций подсказок. \*/

export const W = 260;

export const H = 130;

/\*\*

\* Общий множитель скорости. Кадры были слишком короткими

\* заметить, что именно переехало, и анимация читалась

\*/

export const SPEED = 2.1;

/\*\* Длительность переезда узлов. Должна быть заметно ме

export const MOVE\_MS = 800;

export const EASE = "cubic-bezier(.34,.01,.2,1)";

export const MOVE = `all \${MOVE\_MS}ms \${EASE}`;

/\*\* Наведение на карточку ставит анимацию на паузу — мо

export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {

const [step, setStep] = useState(0);

const paused = useContext(DemoPauseContext);

const period = Math.round(ms \* SPEED);

useEffect(() => {

```

.....

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);

```

```

</text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
 </Svg>
);
};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B", "C"],
 ["D", "E"],
 ["F", "G", "H"],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (
 <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
))
)}
 >
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={k}
 })}
 </Svg>
);
};

```

```
.....
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={C.idle}
))}
 </Svg>
);
};
```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```
export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотные индексы"
 }
 >
 {raw.map((v, i) => (
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={30}>
 {v}
 </text>
 </g>
))}

 {step >= 1 &&
 sorted.map((v, i) => (
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} height={20} />
 <text x={30 + i * 52 + 23} y={72}>
 {i}
 </text>
 </g>
))}
 </Svg>
);
};
```



```

.....
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
)))
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x},
 <circle r={13} fill={color(id)} stroke={C.idle}
 <text textAnchor="middle" dominantBaseline="c
 {id}
 </text>
 </g>
)))
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={["р – корень, х под ним", "Один поворот:
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"
]}
 />
);

```

```

/** Zig-Zig – х и р с одной стороны, крутим сначала вер
export const ZigZigDemo: React.FC = () => (
 <SplayFrames
 captions={["Линия g → р → х («бамбук»)", "Поворот В
 frames={[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]

```

```

.....
const wave = step > 0 && step <= order.length ? order
return {
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(({[u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}})
 {Object.entries(GRAPH_POS).map(({[k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
 }}})
 </Svg>
};
};

```

```

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return {
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(({[u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_
 }}})
 {Object.entries(GRAPH_POS).map(({[k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={k ===
 }}})
 </Svg>
 };
};

```

```

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "трусы", x: 52, y: 32 }, { id: "штаны", x: 14

```

```

.....
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь" : ""}>
 <Edge a={{60, 70}} b={{200, 70}} color={improved ? "red" : "black"}>
 <text x={130} y={54} textAnchor="middle" fontSize={16}>12 > 3 + 4 = 7</text>
 <Node x={60} y={70} r={20} fill={C.active} label="A">
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="B">
 <text x={60} y={106} textAnchor="middle" fontSize={16}>A</text>
 <text x={200} y={106} textAnchor="middle" fontSize={16}>B</text>
 <text x={130} y={22} textAnchor="middle" fontSize={16}>12 > 3 + 4 = 7</text>
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge && cut ? "red" : "black"}>
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? C.done : C.active}>
))}
 </Svg>
);
};

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {

```

```

.....
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 }}}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
</Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
)}}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={step :
)}}
 </Svg>
);
};

```

.....  
ФАЙЛ 50/96: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

.....
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

```

## Универсальное пособие • полный исходный код

```

.....
const states = [{"A", "B"}, {"A", "B", "C"}, {"B", "C"}];
const step = useLoop(states.length, 800);
const items = states[step];
return (
 <Svg caption="pop_front слева, push_back справа – к
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>выход</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>вход</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={20} />
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#ff7f50"];
 const root = (i: number): number => (p[i] !== i ? i : p[i]);
 return (
 <Svg caption="find = «кто твой староста», union = объединение"
 {p.map((par, i) => (
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`}
 fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
) : null
))}
 </Svg>
);
};

```

```

 }}}}
 {TRIE_NODES.map((n) => {
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId}
)}}
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};

```

```

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3, 0]];
 const step = useLoop(links.length + 1, 900);
 return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
 fill="none" stroke={C.warn} strokeWidth={1.5}
)};
)}}
 {TRIE_NODES.map((n) => {
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
)}}
 </Svg>
);
};

```

```

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },

```

```

.....
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={10}
 fill={show && i !== 1 ? C.bad : show && i !== 2 ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: 'stroke 0.5s' }} />
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для j`}>
 {Array.from({ length: 8 }).map((_, i) => (
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={10}
 fill={C.dim} />
 <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
 </g>
))}
 {Array.from({ length: count }).map((_, i) => (
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10}
 fill={C.active} opacity={i !== 0 ? 1 : 0.4} stroke={C.dim} />
))}
 </Svg>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);

```

```

.....
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={28}
 fill={step > 0 && (idx < len || (idx > i - 1) ? C.warn : C.idle)}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">
 ext>
 <text x={32 + idx * 33} y={70} textAnchor="middle">
 fill={idx <= i && step > 0 ? C.warn : C.idle} />
 </g>
)))
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={12}>
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
</Svg>
);
};

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : filled}
 style={filled && {
 <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle">
 fontSize={10} fontWeight="700" fill="white" />
 }}
 </g>
);
 })
 }` />
);
};

```



```

.....
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
};

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind!];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}

```

```

.....
const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end"
 <div className="absolute inset-0 bg-slate-950/80"
 <div className="relative w-full sm:max-w-5xl max-h-
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4"
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm"
 {viz.hint && <p className="text-[11px] text-white"
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain"
 <Component />
 </div>
 </div>
 </div>,
 document.body
);
};

```

ФАЙЛ 53/96: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useLayoutEffect, useRef, useImperativeHandle,

```

.....

```
const below = anchor.rect.top < height + GAP + 8;
const top = below ? anchor.rect.bottom + GAP : anchor
```

```
setPos({ top: Math.max(8, Math.min(top, vh - height
}), [anchor]));
```

```
useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);
```

```
if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;
```

```
return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "unde
 visibility: pos ? "visible" : "hidden",
 }}
)
```

```

 </button>
 })
 </div>
 </div>
</div>,
document.body
);
};

```

ФАЙЛ 54/96: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP, (vars) => {
 const edge = `${vizString(vars.u)} ?? ""}${vizString(vars.v)} ?? ""`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
 });

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

 const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
];

```

```

.....
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // otherwise
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#a";
 if (e.id === 'AB' && step > 4) return "url(#arr";
 if (e.id === 'BC' && step === 5) return "url(#a";
 if (e.id === 'BC' && step > 5) return "url(#arr";
 if (e.id === 'CA' && step === 6) return "url(#a";
 if (e.id === 'CA' && step > 6) return "url(#arr";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0}
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2}
 return e.newWeight;
 }

```

```

);
case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгоритм
 ных вершин:

 </div>
);
case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый вес
 <div className="text-center font-monospace">
 Это повышает или понижает вес ребра
 </div>
);
case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра $A \rightarrow B$.
 Старый вес:
 Вычисляем: $-2 + 0 - (-2) =$
 </div>
);
case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра $B \rightarrow C$.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$
 </div>
);
case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра $C \rightarrow A$.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$
 </div>
);

```

```

.....
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 <marker id="arrow-emerald"
o-start-reverse">
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 </defs>

 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 return null;
 }

 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getEdgeColorClass(e);
 const textHigh = isEdgeTextHigh(e);
 const textEmer = isEdgeTextEmer(e);

 return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 x2={v.x} y2={v.y}
 className={`stroke-${edgeColorClass}`}
 markerEnd={getMarkerEnd(e)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="bottom"
 className={`text-${edgeColorClass}`}
 style={{textHigh, textEmer}}
 />
 </g>
);
 })}
 </svg>
)

```

```

 </g>
)
 }}}}
 </svg>
</div>

<div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 rounded
 justify-center leading-relaxed text-sm text-white
 {getDesc()}
 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 border
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border
 -slate-400 transition-colors" title="Сбросить"
 <RotateCcw strokeWidth={3} size={24} />
 </button>
 <button onClick={() => setStep(2)}
 className="p-3 bg-slate-800 border
 -slate-400 transition-colors" title="Варьировать"
 <Undo strokeWidth={3} size={24} />
 </button>
 <button onClick={() => setStep(3)}
 className={`flex-1 font-bold text-white
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:bg-indigo-500'}
 {step === MAX_STEP ? "Готово" : "В процессе"}
 {step !== MAX_STEP && <Skip />
 </button>
 </div>

 {/* Progress Dots */}
 <div className="flex justify-center gap-2"
 {Array.from({length: MAX_STEP + 1,

```



## Универсальное пособие • полный исходный код

```
.....
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (See
};

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "finish";
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode === v));
 return candidates.length > 0 ? candidates[0].step : null;
};
```

```

-1,
});

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
 u,
 false,
 -1,
);
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

```

```

// Run DFS2 using the order (reversed from back to
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {

```

```
.....
<h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
</h3>

<div className="flex items-center gap-2 bg-slate-900"
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-3 rounded-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center rounded-500"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12"
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative"
 <svg className="w-full h-full max-w-[650px]"
 {/* Markers for directed arrows */}
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
```

```

const x1 = step.transposed ? v.x : u.x;
const y1 = step.transposed ? v.y : u.y;
const x2 = step.transposed ? u.x : v.x;
const y2 = step.transposed ? u.y : v.y;

const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
} else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 />

```

```

 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
}
}
</svg>

```

```

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
</div>

```

```

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4

```

```

.....
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
))}
 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 Вы находитесь на шаге {currentStepIdx + 1} из {steps.length} шагов

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 rounded"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 rounded"
 >
 Вперед
 </button>
 </div>
 </div>
</div>

```

```

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC<{ defaultAlgo?:
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>(
 const [activeCheatTab, setActiveCheatTab] = useState<

```



```

.....
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]));
 },
// Step 2: Include A-B
() => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в остоле',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
},
// Step 3: Inspect D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветны
 type: 'info'
 }, ...prev]));
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в остоле',
 description: 'Вершины D и E окрашены в синий цв
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {

```

```

.....
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро В-Е (вес 6)',
 description: 'НО: это ребро соединяет вершины В
 type: 'warning'
 }, ...prev]));
},
// Step 10: Reject В-Е
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро В-Е. Иначе п
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в остоае',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {

```



```

 type: 'info'
 }, ...prev]);
},
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D:
 type: 'success'
 }, ...prev]);
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вер
 type: 'info'
 }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в ку
 type: 'success'
 }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {

```

```

 }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет верши
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',

```

```

.....
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]);
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]);
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.',
 type: 'warning'
 }, ...prev]);
},
// Step 21: Reject E-H
() => {

```

```

.....
// Node 2(C) -> B-C(4)
// Node 3(D) -> D-E(3)
// Node 4(E) -> D-E(3)
// Node 5(F) -> E-F(7)
// Node 6(G) -> D-G(8)
// Node 7(H) -> G-H(10)
// Node 8(I) -> H-I(12)
setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбирает
description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
}, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Components)
() => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/olive)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' }
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' }
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы
description: 'Все выбранные дороги построены ра

```

```

.....
 }, ...prev]);
 }
};

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boruvka') => {
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 if (algo === 'kruskal') {
 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа соединены. Выбираем самую легкую ребро, которое не создает цикла. Если такое ребро есть, добавляем его. Если нет, пропускаем. Повторяем, пока не останется ребро, которое не добавим. Это и будет минимальное остовное дерево (MST).',
 type: 'info',
 },
]);
 } else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стартовую вершину (Heap) и захватываем соседей!',
 type: 'info',
 },
]);
 }
};

```



```

.....
};

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Сунер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white">
 MST
 </h3>
 </div>
 <p className="text-slate-400 text-sm">
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center bg-slate-800/50" >
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 }
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button

```



```

</div>
})

{/* SVG Edges */}
<svg className="absolute inset-0 w-full h-full
spectRatio="none">
 {liveEdges.map(edge => {
 const u = vertices[edge.u];
 const v = vertices[edge.v];
 const strokeColor = getEdgeStroke(edge);
 const isInspecting = edge.status === 'inspect';
 const isRejected = edge.status === 'rejected';
 const isInHeap = edge.status === 'in_heap';

 return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#94a3b8'}
 : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow-sm"
 dy="-1.5"
 />
 </g>
);
 })}
</svg>

```

```
</div>
```

```
{/* Logs / Heap */}
```

```
<div className="bg-slate-950 rounded-2xl border
```

```
 <div className="p-4 bg-slate-900 border-b b
```

```
 <h4 className="font-bold text-slate-200 t
```

```
 Ход выполнения
```

```
 </h4>
```

```
 </div>
```

```
{activeAlgo === 'prim' && {
```

```
 <div className="p-3 bg-slate-900/60 borde
```

```
 <p className="text-[11px] font-bold tex
```

```
 ⚡ Приоритетная очередь (Min-Heap):
```

```
 </p>
```

```
 <div className="flex flex-wrap gap-1.5"
```

```
 {heapQueue.length === 0 ? {
```

```
 <span className="text-xs text-slate
```

```
 } : {
```

```
 heapQueue.map((item, idx) => {
```

```
 <span key={idx} className={"px-2 p
```

```
0 shadow-md animate-pulse" : "bg-slate-800
```

```
 {item}
```

```

```

```
 })
```

```
 })
```

```
 </div>
```

```
</div>
```

```
}}
```

```
<div className="flex-1 p-4 overflow-y-auto
```

```
 {logs.map((log, idx) => {
```

```
 <div
```

```
 key={idx}
```

```
 className={
```

```
 "p-4 rounded-xl border transition-a
```

```
 (log.type === 'success'
```

```
 ? 'bg-emerald-950/40 border-emera
```

```
<button
 onClick={() => setActiveCheatTab('prim')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 style={{ background-color: activeCheatTab === 'prim' ? "bg-emerald-600 text-white" : "bg-slate-950 text-slate-300"}}
>
 Прима (Билет 19)
</button>
<button
 onClick={() => setActiveCheatTab('boruvka')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 style={{ background-color: activeCheatTab === 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-950 text-slate-300"}}
>
 Борувка (Билет 20)
</button>
</div>
</div>

{activeCheatTab === 'kruskal' && {
 <div className="grid grid-cols-1 lg:grid-cols-2">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">Крускалов алгоритъм за намиране на минимално обкръжено дърво в неориентиран граф.</p>
 <Clock className="w-3.5 h-3.5 text-inherit"></Clock>
 <p className="text-2xl font-black text-white">Крускал</p>
 <p className="text-slate-400 text-xs font-medium">Минимално обкръжено дърво</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">Авард алгоритъм за намиране на минимално обкръжено дърво в ориентиран граф.</p>
 <Award className="w-3.5 h-3.5 text-inherit"></Award>
 <p className="text-2xl font-black text-white">Авард</p>
 <p className="text-slate-400 text-xs font-medium">Минимално обкръжено дърво</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">Оженинг алгоритъм за намиране на минимално обкръжено дърво в неориентиран граф.</p>
 <p className="text-xxs text-slate-300 font-medium">Оженинг</p>
 </div>
 </div>
 }
}
```

```

.....
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
 <Clock className="w-3.5 h-3.5 text-em
 </p>
 <p className="text-2xl font-black text-r
 <p className="text-slate-400 text-xs mt
 </div>
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
 <Award className="w-3.5 h-3.5 text-em
 </p>
 <p className="text-2xl font-black text-r
 <p className="text-slate-400 text-xs mt
 </div>
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
положении:</p>
 <p className="text-xs text-slate-300 le
ие чистые вершины через самое дешевое досту
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
 <Code className="w-3.5 h-3.5 text-eme
 </p>
 <pre className="text-xs text-emerald-40
80 flex-1">
{'import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue

```



```

.....
 highlightColor: 'text-emerald-400',
 rest: '}'. Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-500/30',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover grid"
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc} {c.complexity}
 </p>
 <div className={`mt-3 text-xs font-mono px-2`} >{c.rest}</div>
 </div>
 </div>
 </div>
))}
 </div>
);
}

```

ФАЙЛ 58/96: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useState } from "react";

```



```

.....
 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание • {index + 1} из {total}

 </button>
</div>

{open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950">
 <div className="absolute inset-y-0 left-0 w-[100px] h-[100px]
 te-[tocIn_.18s_ease-out]">
 <div className="flex items-center justify-between">

 Содержание
 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
 </div>
 </div>
</div>

```

```
.....
<div className="flex items-center gap-3 w-full"
 <div className="bg-gradient-to-tr from-indigo
 k-0">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-
 Подготовка к экзаменам: Алгоритмы, Структ
 </p>
 </div>
</div>

<div className="flex items-center w-full lg:w-a
 >
 <button
 id="tab-guide-btn"
 onClick={() => window.scrollTo({ top: 0, be
 className="hidden sm:flex flex-1 lg:flex-no
 ite bg-indigo-600/80 shadow-lg shadow-indig
 title="Содержание пособия"
 >
 <BookOpen className="w-4 h-4" /> Учебное по
 </button>
 <button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (на
 страницей"
 className={`flex-1 lg:flex-none flex items-
 uration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }}
 </button>
```

```
.....

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
</div>
</header>
);
};
```

.....  
ФАЙЛ 60/96: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотри
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border
 <div className="absolute top-2 left-2 bg-slate-
 00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:160,y:40},

```

```

.....
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill=
 <text x={p.x} y={p.y+3} textAnchor="m
 </g>);
 }
 return <>{lines}{circles}</>;
 })()}
</svg>
<div className="absolute bottom-2 left-2 right
 z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
 <div className="absolute top-2 left-2 bg-slat
 00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 }
 }

```

```
.....
 </div>
);
}
```

.....  
ФАЙЛ 61/96: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import {
 ArrowRightLeft,
 Check,
 CheckCircle2,
 ChevronLeft,
 ChevronRight,
 Copy,
 Eye,
 HelpCircle,
 Link2,
 Loader2,
 Pause,
 Play,
 Plus,
 RotateCcw,
 SkipBack,
 SkipForward,
 Square,
 Terminal,
 Unlink,
 Variable,
 X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS } from "lucide-react";
import { clearVizState, emitVizState, onVizDemo } from "lucide-react";
import {
 baseVarName,
```



```

.....
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = push",
 code: 'st = []\nst.append("A") # push\nst.append("A")\nst.pop() # pop\n',
},
{
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
 code: 'from collections import deque\nq = deque()\nq.append(1)\nq.popleft()\n',
},
{
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) — как очередь приоритетов",
 code: 'import heapq\nh = []\nheapq.heappush(h, (1, "A"))\nheapq.heappop(h)\n',
},
{
 label: "Матрица n × m",
 tip: "Таблица расстояний: dist[i][j], бесконечность",
 code: 'n, m = 3, 4\ndist = [[float("inf")] * m for i in range(n)]\n',
},
{
 label: "Множество visited",
 tip: "Посещённые вершины: in/add за O(1).",
 code: 'visited = set()\nvisited.add("A")\nprint("A" in visited)\n',
},
],
},
{
 group: "Вывод",
 items: [
 {
 label: "f-строка",
 tip: "Подставить значения переменных шага в строку",
 code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}] = {dist[i][j]}")\n',
 },
],
},
],
},

```

```

.....
/** Принудительный повтор (глаз / Ctrl+Enter), даже е
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "hi
const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);

const [desyncedFrom, setDesyncedFrom] = useState<stri
 savedEditor &&
 savedEditor.chapterId !== chapterId &&
 savedEditor.code !== savedEditor.template &&
 savedEditor.code.trim() !== "" &&
 savedEditor.code !== template &&
 savedEditor.code !== initTemplate
 ? chapterId
 : null
);
const prevTplRef = useRef({ base: initTemplate, full:

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugRes
/** Esc оставляет неизменённый текст в редакторе, пок
const [editHold, setEditHold] = useState<string | null
/** Автопроход включается пользователем; первый шаг п
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num
 if (typeof window === "undefined") return 1;
 const saved = Number(window.localStorage.getItem(PL
 return PLAYBACK_SPEEDS.includes(saved as (typeof PL
));
const listRef = useRef<HTMLDivElement>(null);

useEffect(() => {
 window.localStorage.setItem(PLAYBACK_SPEED_KEY, Str
}, [playbackSpeed]);

useEffect(() => {
 savedEditor = { code, template: initTemplate, chapter

```



```

 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать – бу
 }
}, [code]));

```

/\* — Пошаговый отладчик

/\*\* Связь больше не зависит от эталонных строк: совпа  
const hasViz = sync.variables.length > 0;

/\*\*

\* Снимок ПОСЛЕ выделенной строки. Контейнеры берём и  
\* (там уже виден append/st[i][j]=...), но индексы цикл  
\* события: после st[i][j]= Python уже мог увеличить  
\*/

```

const snapshotAt = useCallback((result: DebugResult,
 const step = result.steps[idx];
 const next = result.steps[idx + 1];
 const afterLocals = next?.locals ?? result.finalLoca
 const afterValues = next?.values ?? result.finalVal
 const beforeValues = step?.values ?? {};
 const line = step?.line ? source.split("\n")[step.l
 // Нас интересуют только простые скалярные цели (`i
 // `i +=`). Контейнеры и так всегда берутся из afte
 // внутри print(f"i={i}") не будет ошибочно принят
 const assignment = line.match(
 /^\\s*([A-Za-z_][A-Za-z0-9_]*(?:\\s*,\\s*[A-Za-z_][A
);
 const assigned = new Set(assignment?.[1].split(",")

```

```

const variables = { ...afterValues };
const locals = { ...afterLocals };
for (const [name, value] of Object.entries(beforeVa
 const scalar = value === null || typeof value !==
 if (scalar && !assigned.has(name) && name in afte

```

```

.....
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
},
[]
);

const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepTo = useCallback((idx: number) => goDebug(i

const stopDebug = useCallback(() => {
 setPlaying(false);
 setEditHold(code);
 setDebug(null);
 clearVizState(chapterId);
}, [chapterId, code]);

// Базовый интервал 650 мс; множитель скорости хранитс
useEffect(() => {
 if (!playing || !debug) return;
 if (debug.idx >= debug.result.steps.length - 1) {
 setPlaying(false);
 return;
 }
 const t = window.setTimeout(() => stepBy(1), 650 /
 return () => window.clearTimeout(t);
}, [playing, debug, stepBy, playbackSpeed]);

```

```

.....
 const message = errorText(error);
 setDebug({ result: { steps: {}}, error: message, t
 setOutput(`Ошибка:\n${message}`);
 setRuntimeMessage("Не удалось выполнить код – про
} finally {
 setRunning(false);
}
}, {chapterId, code, stdin, traceRequest, running, ru

// Код выполняется сам: debounce даёт спокойно допеча
// изменился во время загрузки Pyodide, после заверше
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin &&
 const timer = window.setTimeout(() => void debugRun
 return () => window.clearTimeout(timer);
}, {code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не ст
useEffect(() => {
 if (!debug) return;
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(1);
 }
 if (e.key === "ArrowLeft") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(-1);
 }
 if (e.key === " ") {

```

```

.....
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) step
 setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n")
 const start = ta.selectionStart ?? current.length
 const end = ta.selectionEnd ?? start;
 const padded = (start > 0 && !current.slice(0, start)
 const next = current.slice(0, start) + padded + c
 requestAnimationFrame(() => {
 ta.focus();
 ta.selectionStart = ta.selectionEnd = start + p
 });
 return next;
 });
}, []);

return (
 <aside
 aria-label="Python-компилятор"
 className="fixed z-40 inset-x-0 bottom-0 h-[58dvh
 -[620px] flex flex-col bg-slate-900 border-t lg
 style={typeof window !== "undefined" ? {
 ...(width && window.innerWidth >= 1024 ? { width:
 ...(height ? { height: Math.min(height, window.h
 } : undefined)}
 >
 {/** Верхний край меняет высоту на телефоне и деск

```

```

aria-label="Потяните, чтобы изменить ширину п
title="Потяните, чтобы изменить ширину панели
className="hidden lg:block absolute top-0 bot
onPointerDown={({e}) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX
 onWidthChange(Math.max(380, Math.min(next
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 });
 el.addEventListener("pointermove", onMove a
 el.addEventListener("pointerup", onUp as Ev
}}
>
 <div className="absolute left-1/2 top-1/2 -tr
 digo-500 transition-colors" />
</div>
})
/* — Заголовок панели
<div className="shrink-0 flex items-center gap-2
 <span className={`w-2 h-2 rounded-full shrink-0
 <Terminal className="w-4 h-4 text-emerald-400 s
 <span className="text-[13px] font-bold text-whi
 <span className="hidden sm:inline text-[10px] t

 <div className="ml-auto flex items-center gap-1
 <Tooltip
 side="bottom"
 content={running ? "Python выполняется..." :
 запустить автопроход"}
 >
 <button

```

```

.....
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Заккрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip
 side="bottom"
 content="Вставить полный эталонный код этого"
 >
 <button
 type="button"
 onClick={() => {
 setCode(template);
 setDesyncedFrom(null);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);
 }}
 className="p-1.5 rounded-lg text-indigo-300"
 aria-label="Вставить эталонный код"
 >
 <Eye className="w-4 h-4" />
 </button>
 </Tooltip>
 <label
 className="ml-1 inline-flex items-center gap-2 border border-slate-700 hover:border-emerald-500 rounded-3xl px-2"
 title="Скорость автоматического проигрывания"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" id="400" /> : null}
 {running ? "авто..." : "авто"}
 <select
 aria-label="Скорость автопрохода"

```

```

.....
 ? "Связь идёт по именам ниже: реальный
а не сравнивается."
 : "На выбранной странице нет интеракта
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex item
 hasViz ? "text-emerald-400" : "text-s
 }`}
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет виз

 </Tooltip>
</div>
</div>

{stripTab === "vars" ? (
 <div className="px-3 pb-2.5 space-y-2 max-h-3
 <div className="flex items-center gap-2 min
 <Tooltip side="bottom" content="Открыть э
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center g
700 text-slate-300 hover:text-white hover:b
 >
 {chapterTi
 </button>
 </Tooltip>
 </div>
 {desyncedFrom !== null && (
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страниц
т заменён). "
 >

```

```

.....
 <span
 tabIndex={0}
 className="cursor-help inline-flex
text-[10px] text-slate-400 hover:text-white
 >
 <ArrowRightLeft className="w-3 h-3" />

 </Tooltip>
)}
</div>
)}
</div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow-y-auto" >
 {SNIPPETS.map((gItem) => (
 <div key={gItem.group}>
 <div className="text-[9px] font-bold uppercase" >
 {gItem.group}
 <div className="flex flex-wrap gap-1.5" >
 {gItem.items.map((s) => (
 <Tooltip key={s.label} side="bottom" >
 <button
 type="button"
 onClick={() => insertSnippet(s)}
 className="inline-flex items-center
text-[10.5px] text-indigo-300 hover:text-white
 >
 <Plus className="w-3 h-3 text-slate-400" />
 {s.label}
 </button>
 </Tooltip>
))}
 </div>
 </div>
))}
 </div>
)}
<p className="text-[10px] text-slate-600">H
</div>

```



```

tion-colors">
 <ChevronRight className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip content="В конец трассы">
 <button type="button" onClick={() => {
 step || debug.idx >= debug.result.steps.length
 0 disabled:opacity-40 transition-colors">
 <SkipForward className="w-4 h-4" />
 </button>
</Tooltip>
<span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <
 var {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>"
 </>
) : (
 "шагов нет – исправьте ошибку и начини
)}

{debug.result.truncated && {
 <Tooltip content={`Выполнение остановлен
 <span tabIndex={0} className="cursor-
rounded-full px-1.5 py-0.5">
 ЛИМИТ

 </Tooltip>
}}
<Tooltip content={hasViz ? "Визуализация
алонный текст не требуется." : "У страницы
 <span
 tabIndex={0}
 className={`cursor-help inline-flex i
"border-emerald-500/40 bg-emerald-500/10 t
 >
 {hasViz ? <Link2 className="w-3 h-3"
 {hasViz ? "имена → демо" : "свободный

```

```

.....
<div className="shrink-0 max-h-[34%] overflow-hidden">
 <div className="flex items-center gap-1.5">
 <Variable className="w-3 h-3 text-indigo-500">
 Переменные на этом шаге

 </div>
 {debug.result.error && (
 <pre className="mt-1 whitespace-pre-wrap">
)}
 {debugOrdered.length !== 0 && !debug.result.error && (
 <div className="mt-1 text-[11px] text-slate-500">
 /div>
)}
 <div className="mt-1 flex flex-wrap gap-1">
 {debugOrdered.map(({name, value}) => {
 const changed = debugChanged[name];
 const synced = syncVarSet.has(name);
 return (
 <span
 key={name}
 className={`inline-flex items-baseline gap-1.5
 ${changed
 ? "border-amber-500/50 bg-amber-500/10"
 : synced
 ? "border-emerald-500/40 bg-emerald-500/10"
 : "border-slate-700 bg-slate-700/10"}
 `}>
 >

 {name}

 {value}

 {syncVarSet.get(name)}

);
)}
 </div>
</div>

```

```

 </div>
 </div>

 { /* — Вывод —————
 <div className="shrink-0 border-t border-slate-800"
 <button
 type="button"
 onClick={() => setOutputOpen(o => !o)}
 className="flex items-center justify-between
 colors"
 >
 <span className="inline-flex items-center gap-1"
 <Terminal className="w-3.5 h-3.5 text-emerald-500"

 <span className="inline-flex items-center gap-1"
 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500"
 {outputOpen ? 'Закрыть' : 'Открыть'}

 </button>
 {outputOpen && (
 <pre className="flex-1 min-h-[70px] overflow-hidden leading-5 text-slate-300">
 {output}
 </pre>
)}
 <div className="px-3 py-1.5 border-t border-slate-800"
 </div>
</aside>
);
}

```

ФАЙЛ 62/96: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
```

```

.....

const runtime = useVizRuntime();
const liveQ = vizArray(runtime?.variables?.q);
const liveV = vizString(runtime?.variables?.v);
const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: " ",
 color: "from-emerald-500/20 to-emerald-600/20 b",
 bubbleText: `q[${index}] из Python`,
 animState: "idle",
 }))
 : queue;

const [servedHistory, setServedHistory] = useState<st
const [isServing, setIsServing] = useState<fa
const [shakeEmpty, setShake] = useState<fa
const [log, setLog] = useState<st

const addLog = (msg: string) => setLog(prev => [msg,

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяже
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

const preset = PRESETS[counter % PRESETS.length];
counter++;

const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,

```

```

.....
 addLog(` ${headGuest.name} поел и ушел сытым.
 }, 400);
 }, 900);
}, {queue, isServing});

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: ' ', color: 'red', text: 'Я
т голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: ' ', color: 'blue', text: 'Я
апишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: ' ', color: 'green', text: 'Я
без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
 });

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-500 p-4">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В очереди за супом всё честно. Кто первый встал, тот и ест.

 Новые гости встают строго в конец очереди.

 Здесь нет обхода и блата – только строгий порядок.
 </p>
 </div>
 </div>
 </div>
);
}

```

```
.....
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ &&
</div>

<div className="flex-1 flex flex-col justify-between">
 <div className="flex items-end justify-between">

 {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
 <div className="flex flex-col items-center">
 <div className="relative">
 <div className="absolute -top-10 left-10">
 <div className="w-1.5 h-6 bg-slate-500">
 <div className="w-2 h-4 bg-slate-400">
 </div>
 </div>

 </div>
 <div className="text-center">
 </div>

 ПОВАР

 </div>
 </div>
 </div>

 {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
 <div className="flex flex-col items-center">

 </div>

 {/* СПИСОК ОЧЕРЕДИ */}
 <div className="flex-1 flex items-end gap-2">
 {displayQueue.length === 0 ? (
 <div className="flex-1 flex flex-col justify-between">
 </div>
 <p className="text-xs font-bold font-mono">
 <p className="text-[10px]">Все сыты
```

```

.....
 <span className={`
 absolute -bottom-2 text-[10px]
 ${isHead ? 'bg-amber-500' : 'bg-white'}
 `}>
 {isHead && isTail ? 'h+t' : ''}

)}
 </div>
 </div>
);
})
}
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-amber-500 to-amber-200">
 </div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-end gap-2 font-mono">
 <div className="order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse">
 СЫТЫ
 </div>
 </div>

 <div className="flex-1 flex flex-col justify-end gap-2">
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center">

 <p className="text-xs font-bold font-mono">

```





```

.....
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX;
 if (leftX === -1) currentX++;
 }
};

layoutDFS(0, 0);

```

```

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главной
 очереди суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полную
 таблицу переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.ch

```

```
.....
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссы
 </p>
</div>
</div>
<div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate
ransition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
</div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center just
 <div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45
 <path d="M 40 85 Q 100 55 160 85 Q 100 115
 </path>

 {/* Хвост */}
 <path
```

## Универсальное пособие • полный исходный код

```
.....
 <div className="absolute -top-1 right-4 text-rose-400">
 <div className="absolute top-8 right-1 text-rose-400">
 </div>

 Эхо-Лосось Сеня

 </div>

 {/* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 border border-slate-700
 tween min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -translate-x-3
 slate-800 border-b-[12px] border-b-transparent">
 </div>

 <div className="flex items-center space-x-2 text-rose-400">
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`>
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leading-relaxed">
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-slate-700">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400">
 Статус: {simulationStep === 0 ? 'Борется с рыбой' : 'Поедает'}
 </div>
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500
 -1 shadow-lg shadow-emerald-900/40 transition-colors">
 >
```

```

.....
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 {/* Draw Edges */}
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x ||

 const isCurrent = currentNode === node.

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x} / 2 - 10)}
 y={({parent.y + node.y} / 2)}

```

```

.....
{Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node;
 const isInQueue = bfsQueue.includes(node);

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >

```

```

 title="Удалить"
 >
 x
 </button>

 }}}
</div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >

 Таблица переходов и fail-

 Всего вершин: {Object.keys(nodes).length}

 </h5>
</div>

```

```

 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'
 }` }
 >
 #{node.id}
 {node.id === 0 && <span className=
</td>
<td className="py-2.5 px-3">
 <span className="bg-slate-800 b
 {node.char}

</td>
<td className="py-2.5 px-3 text-s
 {node.id === 0 ? '-' : `#${node
</td>
<td className="py-2.5 px-3">
 {node.id === 0 ? {
 <span className="text-slate-5
 } : {
 <span className="bg-indigo-95
 #{node.fail}

 }}
</td>
<td className="py-2.5 px-3">
 {node.id === 0 || node.term_lin
 <span className="text-slate-5
 } : {
 <span className="bg-rose-950
0 transition-colors cursor-help" title={`Ck
 #{node.term_link}

 }}
</td>
<td className="py-2.5 px-3 text-s
 {Object.keys(node.children).len
 ? 'ø'

```



```

desc: string;
codeLine: number;
treeState: Record<number, number>; // node index
highlightNodes: number[]; // current
mergeChildren?: {number, number}; // pair be
arrayHighlight?: {number, number}; // [L, R]
result?: number; // partial
lazyState?: Record<number, number>; // node in
promisePushed?: number; // узел, у
}

```

```

// ===== PURE LOGIC: build tree =====
export function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}

```

// ===== STEP GENERATORS =====

```

// 1. Build steps (recursive top-down)
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2

```

```

.....
const n = arr.length;
const tree = buildTreeFull(arr);
const steps: Step[] = [];
let id = 0;

function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v] });
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 4, treeState: tree, highlightNodes: [v] });
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v] });
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
 codeLine: 8, treeState: tree, highlightNodes: [v],
 rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [2 * v, 2 * v + 1] });
 return res;
}
const ans = go(1, 0, n - 1, ql, qr);
steps.push({ id: id++, desc: `Запрос sum([${ql}..${qr}]`,
 codeLine: 10, treeState: tree, highlightNodes: [1],
 arrayHighlight: [ql, qr], result: ans });
return steps;
}

```

```

// 3. Bottom-Up Query steps (iterative on implicit tree)
function genQueryBottomUpSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
}

```

```
.....

// ===== TREE RENDERING HELPER =====
interface VNode { v: number; l: number; r: number; val: number; }

function buildVisualTree(treeState: Record<number, number>): VNode {
 if (l === r) return { v, l, r, val: treeState[v] ?? 0 };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
 treeState, v + 1, m + 1, r) };
}

// ===== CODE SNIPPETS =====
const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
];

const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];

const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",

```



## Универсальное пособие • полный исходный код

```
.....
 query(2 * v + 1, m + 1, r);
 }
 res = 0;
 snap(`Теперь запрос sum([${uL}..${uR}]) – проверим, ч
 query(1, 0, n - 1);
 snap(` sum([${uL}..${uR}]) = ${res} – совпадает с че
 return steps;
}
```

```
// ===== PLAYER COMPONENT =====
```

```
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-300 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' } };

 return { step, idx, setIdx, playing, setPlaying, speed };
}
```

```
// ===== SIMULATION PANEL =====
```

```

.....
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full"
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
</div>
);
}, [step, clr, liveNode]);

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 /* Header */
 <div className={`px-4 py-2.5 flex items-center ju
 -emerald-950/50' : 'bg-amber-950/50'} border-b
 <h4 className="font-bold text-sm text-white fle
 <span className="text-[10px] font-mono text-sla
 </div>

 /* Array strip */
 <div className="px-3 py-2 bg-slate-950 border-b b
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= s
 return (
 <div key={i} className={`flex flex-col item
 go' ? 'bg-indigo-950 border-indigo-500' : c
 er-500'} -translate-y-0.5` : 'bg-slate-900 i
 <span className="text-[7px] text-slate-50
 <span className="text-[11px] font-bold fo
 </div>
);
)}}
 </div>

 /* Tree */
 <div className="flex-1 p-2 overflow-x-auto bg-sla

```

```

.....
 0 hover:text-white hover:bg-slate-800 rounded
 <button onClick={() => setPlaying(!playing)}
 white transition-all ${playing ? 'bg-rose-500' : 'bg-slate-800'}
 {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }}
 .5 text-slate-400 hover:text-white hover:bg-slate-800 rounded px-1.5 py-1 outline
 </button>
 </div>
 <select value={speed} onChange={e => setSpeed(Number(e.target.value))}
 text-[9px] font-bold rounded px-1.5 py-1 outline
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Тыкбо</option>
 </select>
</div>

 { /* Progress bar */ }
 <div className="h-1 bg-slate-950"><div className="bg-rose-500" style={{ width: ((current + 1) / total) * 100 : 0}% }} /></div>
</div>
);
}

//
// MAIN EXPORTED COMPONENT
//
export function SegmentTreeVisualizer() {
 const runtime = useVizRuntime();
 const liveI = vizNumber(runtime?.variables?.i);
 const liveV = vizNumber(runtime?.variables?.v);
 const liveAdd = vizNumber(runtime?.variables?.add);
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8, 3, 12');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [uL, setUL] = useState(1);

```

```

<div className="flex flex-col lg:flex-row gap-4"
 {/* Array input */}
 <form onSubmit={handleApply} className="flex-
 <div className="flex items-center justify-b
 <label className="text-[10px] font-bold up
 <label>
 <button type="button" onClick={randomize}
 "><RefreshCw className="w-3.5 h-3.5" /></bu
 </div>
 <div className="flex gap-2">
 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.v
 className="flex-1 bg-slate-900 border b
 one focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indig
 ition-colors">Применить</button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1"
 {arr.map((v, i) => (
 <span key={i} className={`border px-2 p
 ext-amber-200 ring-2 ring-amber-400/30" : "
 <span className={liveI === i ? "text-

)))}
 <span className="text-xs text-slate-500 f
 </div>
</form>

 {/* Query range */}
 <div className="bg-slate-950 p-4 rounded-xl b
 <label className="text-[10px] font-bold upp
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 <span className="text-xs text-slate-400
 <input

```



```

er:text-slate-200'}}`}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
</button>
<button onClick={() => setView('lazy')} className=
 colors ${view === 'lazy' ? 'border-amber-500'
 e-200'}}` } title="Массовое обновление на отрез
 Обновление на отрезке (Lazy)
</button>
<button onClick={() => setView('theory')} class=
 n-colors ${view === 'theory' ? 'border-blue-5
 te-200'}}`}>
 <Info className="w-3.5 h-3.5" /> Почему / Сра
</button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && {
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
}}

{view === 'queries' && {
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху
 icon=" "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}

```

```

 </p>
 </div>
 <SimPanel
 key={`lazy-${arr.join(' - ')}-${uL}-${uR}-${e
 title={`update([${uL}..${uR}], +${effAdd})`
 icon=""
 steps={lazySteps}
 code={LAZY_CODE}
 color="amber"
 arr={arr}
 liveNode={liveV}
 />
 </div>
)}

{view === 'theory' && (
 <div className="space-y-5">
 /* Why it splits this way */
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="text-base font-bold text-whi
 во разбилось именно так для N={arr.length}?
 <p className="text-sm text-slate-300 leadin
 Каждый узел берёт <code className="bg-sla
 к получает <code className="bg-slate-900 px
 -slate-900 px-1 rounded font-mono text-emerg
 евая часть забирает на 1 больше (округление
 </p>
 <div className="bg-slate-900 p-4 rounded-lg
 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = {(0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правы
 h - 1) >> 1)) эл.)</div>
 <div className="text-slate-500 mt-2">Всег
 } внутренних.</div>
 </div>
 </div>
)

```

```

.....
 </div>

 {/* Verdict */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="font-bold text-blue-400 mb-2
 <p className="text-sm text-slate-300 leadin
 <strong className="text-white">Итеративны
 кальности. Но если нужны <strong className=
 ame="text-white">рекурсивный (сверху-вниз)<
 </p>
 </div>
 </div>
 })
</div>
);
}

```

ФАЙЛ 65/96: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

.....

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } f
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели
 onNavigate?: () => void;
}

export const Sidebar: React.FC<SidebarProps> = ({ selec
 const [query, setQuery] = useState("");

```

```

<div className="flex-1 overflow-y-auto px-3 pb-4" style={
 {groups.length === 0 && <p className="text-sm text-gray-500">
 No groups found.

 {groups.map(({category, items}) => (
 <div key={category}>
 <div className="text-[10px] font-bold uppercase">{category}</div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }}
 className={`w-full text-left px-3 py-2 border rounded-sm
 ${isSelected
 ? "bg-indigo-600/15 border-indigo-600/20"
 : "bg-slate-800/40 border-transparent"}
 `}
 >
 {chapter.title}
 {hasViz && (
 <span
 className="w-1.5 h-1.5 rounded-sm border border-indigo-600/20"
 title="Есть интерактивная визуализация"
 />
)}
 </button>
)
)}

 <ChevronRight
 className={`w-4 h-4 shrink-0 mt-0.5
 ${isSelected ? "text-indigo-400" : "text-gray-500"}
 `}
 />

 </div>
)}
)}
 </div>

```

## Универсальное пособие • полный исходный код

```
.....
* поворот — кадры «прицел → отстёгиваем среднее по
* 3. «Собери сам» — песочница: подними узел кликами,
*/
type Tab = "cases" | "big" | "play";

export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) => {
 const [tab, setTab] = useState<Tab>(defaultTab);

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Один
 </p>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5" >
 {[
 { id: "cases" as Tab, label: "1 · Механика: Z", type: "cases" },
 { id: "big" as Tab, label: "2 · Большое дерево", type: "big" },
 { id: "play" as Tab, label: "3 · Собери сам", type: "play" },
]}.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 ${tab === t.id ? "bg-indigo-600 text-white" : ""}
 `}
 >
 {t.label}
 </button>
))
 </div>
 </div>
);
}
```

```

.....
 right: TNode | null;
}

const leaf = {key: number}: TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
 },

```

```

.....
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
 }

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number) {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя поправка)" };
}

/* раскладка

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

function layout(root: TNode | null): { nodes: Placed[]; } {
 const nodes: Placed[] = [];

```

```

.....

const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)));
const expected = useMemo(() => (solved ? null : canonical));

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want === clone(tree) }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return {

```



```

.....
 role="img"
 >
 {edges.map(({a, b}, i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return {
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.6, 1)" }}
 />
 };
 })}

 {nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return {
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.6, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6366f1" stroke-width={2} />
)}
 <circle

```

```

.....
 {showAnswer ? <EyeOff className="w-3.5 h-3.5"
 {showAnswer ? "Скрыть разбор" : "Сдаться / по
 </button>

 <span className="text-[11px] text-slate-500 ml-
 поворотов: {moves.length} • минимум: {minimal

 </div>

 {showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40
 Сейчас splay крутил бы узел{" "}
 {expected
 </div>
)}

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60
 : "bg-rose-950/30 border-rose-800/60 text
 `}
 >
 <div className="flex items-center gap-2 font-i
 {wrong === 0 ? <CheckCircle2 className="w-4
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length}
 : `Узел ${target} в корне за ${moves.length}
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили <span classN
 {m.expected}

```

```
.....

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, {number, number}>>;
```

```
interface Frame {
 pos: Pos;
 edges: {NodeId, NodeId}[];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
 focus?: {NodeId, NodeId};
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
 moved?: NodeId[];
 ms: number;
}
```

```
interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}
```

```
const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,
```

```
/* Zig
const ZIG BEFORE = {
```

```

 },
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C: [160, 112],
 edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g: [255, 40],
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p — дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым — верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 — 40",
 text: "Главная хитрость этого случая: первым крутит
focus: [{"g", "p"}],
 ms: AIM_MS,
 },
],
};

```

## Универсальное пособие • полный исходный код

```
.....
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p – дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым – нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по
 ",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте
 focus: [{"p", "x"}],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле
 moved: [{"x", "p", "C"}],
 ms: RESULT_MS,
 },
],
};
```

## Универсальное пособие • полный исходный код

```
.....
const { pos, edges, focus, moved } = frame;
const dim = (id: NodeId) => (focus ? !focus.includes(id) : true);
const ghosts = frame.kind === "result" ? (moved ?? []) : [];

return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-full h-full">
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" refX="0" refY="0">
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 {/* призраки: где узел стоял до поворота */}
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill="none" />
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
);
 })}
 </svg>
);
```

```

.....
 {hot && <circle r={r + 7} fill="none" stroke
 <circle r={r} fill={c.fill} stroke={c.stroke}
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font
 {id}
 </text>
)}
 </g>
);
}})
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveCase = vizString(vars?.case);
 const liveFrame = vizNumber(vars?.i);
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

 // Компилятор: case = "zig" / "zig-zig" / "zig-zag",
 useEffect(() => {
 if (liveCase) {
 const ci = CASES.findIndex((c) => c.key.toLowerCase() === liveCase);
 if (ci >= 0) setCaseIdx(ci);
 }
 }, [liveCase]);

```

```

</div>

<div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корень
 ({active.rotate}
</div>

{/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом картинка */}
<div className="mb-3 rounded-xl border border-slate-200">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`>
 >
 {current.kind === "aim" ? "прицел" : current.kind}

 Шаг {idx + 1} из {total}. {current.title}

</div>
<p className="text-[13px] text-slate-400 leading-relaxed">
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800">
 <Tree frame={current} prev={prevPos} keys={activeKeys}>

 {/* обход слева направо – одинаковый на всех карточках */}
 <div className="mt-2 pt-2 border-t border-slate-800">
 <div className="flex items-center gap-1.5 flex-wrap">
 Обход
 {active.inorder.map((t, i) => (
 <span
 key={i}

```



```

<button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
>
 {last ? "в Сначала" : "Дальше →"}
</button>
<button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-2
 ext-xs font-bold transition-colors"
>
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
</button>
<button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-2
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.5"}
 </button>
<button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-2
 ext-xs font-bold transition-colors"
>
 <RotateCcw className="w-3.5 h-3.5" /> Сброс

```

## Универсальное пособие • полный исходный код

```
.....
*
* Чистые функции (splaySteps, rotateUp, bstInsert, col
*/
```

```
export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}
```

```
export function cloneWithParents(t: SN | null): SN | null {
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null, parent: null };
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
 n.left = l;
 l.parent = n;
 }
 if (r) {
 n.right = r;
 r.parent = n;
 }
 return n;
}
```

```
export function bstInsert(root: SN | null, key: number): SN | null {
 if (!root) return { key, left: null, right: null, parent: null };
 const r = cloneWithParents(root)!;
 let cur: SN = r;
 for (;;) {
 if (key < cur.key) {
 if (!cur.left) {
 cur.left = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.left;
 }
 }
```

```
.....
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
};
walk(t);
return es.sort();
}
```

```
const diffEdges = (before: string[], after: string[]):
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];
 for (const a of ad) {
 const {p, c} = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
 }
 return parts.join("; ");
};
```

```
export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
 kind: SplayStepKind;
 root: SN | null;
 aimEdge?: [number, number];
 /** Кадр cut: ребро, по которому отстёгивается среднее */
 cutEdge?: [number, number];
 cutLabel?: string;
 /** Куда β переедет: сторона у верхнего узла. */
 cutDest?: string;
 pathKeys?: number[];
 caseLabel?: string;
 note: string;
 rotCount: number;
}
```

```
/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
```

```

let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string) => {
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge,
 caseLabel: label,
 note: aimNote,
 rotCount,
 });
 const p = child.parent!;
 const mid = p.left === child ? child.right : child.left;
 steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `β = поддерезо ${mid.key}` : "β пусто",
 cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : "нигде",
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддерезо β = ${mid.key}
 и у них общий – после поворота переедет ${p.left ? "справа у p" : "слева у p"}`
 : `Среднего поддереза нет (β пусто): ${child.key}`,
 rotCount,
 });
 const before = collectEdges(root);
 rotateUp(child);
 fixRoot();
 rotCount += 1;
 const diff = diffEdges(before, collectEdges(root));
 steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: label,
 });
};

```

```

.....
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}
);
} else {
 const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}
 emitRotation(
 target,
 "Zig-Zag",
 [p.key, target.key],
 `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: по часовой
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Второе ребро: ${target.parent.key}—${target.key}
 rotCount,
));
 emitRotation(
 target,
 "Zig-Zag",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}
);
}
}

steps.push({
 kind: "done",
 root: cloneWithParents(root)!,
 note: ` ${target.key} в корне за ${rotCount} ${rotCount}
 rotCount,
});
return { steps, found };
}

```

```

.....
const MechanismInset: React.FC<{ step: SplayStep | null
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border b
 <h4 className="text-xs font-bold text-slate-300 m
 <p className="text-[10px] text-slate-500 mb-2">
 Как вправление вывиха: одна кость ходит вокруг
 </p>
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">
 <text y="10" fill="#94a3b8" fontSize="11" fon
 <line x1="55" y1="46" x2="30" y2="86" stroke=
 <line x1="55" y1="46" x2="80" y2="86" stroke=
 <line x1="30" y1="86" x2="12" y2="126" stroke=
 <line
 x1="30" y1="86" x2="48" y2="126"
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 3 : 2}
 strokeDasharray={isCut ? "5,3" : "none"}
 />
 <circle cx="55" cy="46" r="15" fill="#1e293b"
 <text x="55" y="50" textAnchor="middle" fill=
 <circle cx="30" cy="86" r="15" fill="#78350f"
 <text x="30" y="90" textAnchor="middle" fill=
 <circle cx="12" cy="126" r="13" fill="#0f172a"
 <text x="12" y="130" textAnchor="middle" fill=
 <circle
 cx="48" cy="126" r="13"
 fill={isCut ? "#78350f" : "#0f172a"}
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 2.5 : 1.5}
 strokeDasharray={isCut ? "4,3" : "none"}
 />
 <text x="48" y="130" textAnchor="middle" fill=
 <circle cx="80" cy="126" r="13" fill="#0f172a"
 <text x="80" y="130" textAnchor="middle" fill=
 <text x="98" y="92" fill={isCut ? "#f59e0b" :

```

Универсальное пособие • полный исходный код

```
.....
 числа поворотов. Единственный, кто меняет хозяина
 </p>
</div>
);
};

/* рендер

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>(null);
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);
 const timer = useRef<ReturnType<typeof setTimeout> | null>(null);

 // — Синхронизация с Python-компилятором: key (или x) — это номер узла
 // i — перейти к кадру расшейвливания.
```

```

.....
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot]);
const nodes = useMemo(() => [...pos.entries()].map(([k, v]) => v));
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.w));
const autoH = Math.max(300, ...nodes.map((n) => n.py));

const dBefore = useMemo(() => (steps ? depthOf(steps[steps.length - 1].root) : 0));
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1].root) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1].root) : 0));
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
 if (!steps || !step || idx <= 0) return null;
 return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);
const movedCount = useMemo(() => {
 if (!prevEdgeSet) return 0;
 return edges.filter((e) => !prevEdgeSet.has(`${e.a}-${e.b}`)).length;
}, [prevEdgeSet, edges]);

const ino0 = useMemo(() => (steps ? inorderKeys(steps[steps.length - 1].root) : ''));
const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;

```



```

.....

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Клик
 </div>
</div>
<div className="flex flex-wrap items-center gap-2">
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current);
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 placeholder="ключ + Enter"
 className="w-28 bg-slate-950 border border-indigo-500"
 />
 <button
 onClick={() => {
 baseRef.current = buildInitial();
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers((v) => v + 1);
 }}
 className="px-3 py-2 rounded-lg text-xs font-medium"
 >
 Исходное
 </button>

```

```

.....
 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel} && (

 {step.caseLabel}

)
</div>
})

{/* Дерево */}
<div className="bg-indigo-950/20 rounded-xl p-4 border border-indigo-950/20">
 {!steps && (
 <div className="absolute top-3 left-3 bg-indigo-950/20 text-indigo-950 font-bold">
 Кликни по любому узлу – splay покадрово
 </div>
)}
 <svg viewBox={`0 0 ${autoW} ${autoH}`} className="w-full h-full">
 {edges.map((e) => {
 const a = pos.get(e.a);
 const b = pos.get(e.b);
 if (!a || !b) return null;
 const isAim = step?.aimEdge && step.aimEdge === e;
 const isCutEdge =
 step?.kind === "cut" && step.cutEdge && step.cutEdge[1] === e;
 const isMoved = !isAim && !isCutEdge && preStep?.kind === "move";
 return (
 <g key={`e-${e.a}-${e.b}`}>
 <line
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={isAim ? "#f59e0b" : isCutEdge ? "#f59e0b" : "#ccc"}
 strokeWidth={isAim ? 4 : isCutEdge ? 2 : 1}
 strokeDasharray={isAim ? "6,4" : isCutEdge ? "6,4" : []}
 className="transition-all duration-500"
 />

```

```

 { /* Лог + механика */ }
 <div className="w-full flex flex-col lg:flex-row" >
 <div className="flex-1 bg-slate-950/80 p-4 rounded" >
 <p className="font-semibold text-amber-400" >
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? " Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-center" >
 {step ? step.note : "Кликни по узлу или в
 ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? " " : " "}`} >
 {checkOk ? " " : " "} проверка кадра: 0
 {inorderKeys(viewRoot).length}
 {movedCount > 0 ? ` ` : ` `} переехало рёбер:
 </p>
)}
 </div>
 <div className="w-full lg:w-[300px] shrink-0" >
 <MechanismInset step={step} />
 </div>
 </div>
 </div>

 { /* Счётчики + код */ }
 <div className="flex flex-col lg:flex-row gap-6 m" >
 <div className="w-full lg:w-1/2 bg-rose-950/10" >
 <h4 className="text-sm font-bold text-rose-300" >
 {steps ? (

```

```

 });
 }
 }
 </div>
</div>
</div>
</div>
);
};

```

export default SplayWalkViz;

ФАЙЛ 70/96: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

```

```

interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}

```

```

const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: '🍝', color: '#f96' },
 { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'f' },
 { name: 'Блюдец от торта', emoji: '🍰', color: 'from-p' },
 { name: 'Тарелка от тако', emoji: '🌮', color: 'from-y' },
 { name: 'Тарелка из-под суши', emoji: '🍣', color: 'fr' },
 { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'f' },
];

```

```

.....
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESET
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newP
 }, 500);
}, {dirtyStack, isWashing});

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(` POP: Начинаем мыть верхнюю тарелку с ${top

```

```

.....

return (
 <div className="flex flex-col gap-6">
 /* МНЕМОНИКА / ПРАВИЛО */
 <div className="bg-blue-950/40 border border-blue-950">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно последнюю.
 Попытаешься вытащить нижнюю — всё рухнет!
 </p>
 </div>
 </div>

 /* УПРАВЛЕНИЕ */
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Помыть верхнюю (POP)
 </button>
 </div>
 </div>
);

```

```

{displayStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center" >

 <p className="text-white font-black text-4xl" >
 <p className="text-slate-500 text-xs mt-1" >
 </p>
 </div>
) : (
 <div className={`flex-1 flex flex-col justify-between`} >

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-center"
 style={{ bottom: `${24 + topIndex * 44}px` }}
 >

 er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 </div>

 {/* Тарелки (отрисовываем в обратном порядке) */}
 {displayStack.slice().reverse().map((plate, idx) => {
 const idx = displayStack.length - 1 - idx;
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
 shadow-md select-none transition-all duration-300
 ${plate.color}
 ${plate.animState === 'in' ? 'animate-in' : ''}
 ${plate.animState === 'washing' ? 'animate-washing' : ''}
 ${isTop ? 'ring-4 ring-blue-500/50' : ''}
 `}
 >

```

```

.....
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full flex
items-center justify-center gap-2 opacity-100"
 >
 {plate.name}
 {plate.id}
 </div>
))}
 </div>
))}
 </div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500"
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-700 p-4"
 <div className="text-[10px] font-mono text-slate-200"
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center gap-2`}
 >
 {idx === 0 ? ACTION :
 {entry}
 </div>
))}
 </div>
</div>
);
};

```



```

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource
 падают! Окно растёт.` });
 z[i]++;
 }
 }
}

```

```

.....
const runtimePattern = vizString(runtime?.variables
const s = runtimeString ?? demoString;
// Длина образца: приоритет – переменная pattern из
// разделителя "#" в склейке pattern#text; иначе вс
const patternLength = runtimePattern
 ? runtimePattern.length
 : runtimeString
 ? (runtimeString.includes("#") ? runtimeString
 : demoPatternLength;

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

```

```

.....
 <div className="bg-slate-800 p-1 flex items-center justify-between" >

 <input value={pattern} onChange={onPatternChange} type="text" />

 </div>
 <div className="bg-slate-800 p-1 flex items-center justify-between" >

 <input value={text} onChange={onTextChange} type="text" />

 </div>
 </div>
 </div>

 <div className="bg-slate-950 rounded-xl border-[1px] flex items-center justify-start" >
 <div className="flex gap-1.5 px-4 h-full" >
 {s.split("").map((char, index) => {
 const isPattern = index < pattern.length;
 const isSeparator = index === pattern.length;

 let borderColor = "border-slate-900";
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "text-white" : "text-slate-400";

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include(index)) {
 borderColor = "border-emerald-500";
 bgColor = step.match === index ? "bg-emerald-500" : "bg-slate-900/40";
 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-500";
 }
 }
 })}
 </div>
 </div>

```

```

 </div>
 }}
 {mode == "kmp" && step.i
 <div className="absolu
 <span classNam
 </div>
 }}
 {mode == "kmp" && step.i
 <div className="absolu
 <span classNam
 </div>
 }}

 {/* Z fingers */}
 {mode == "z" && step.i
 <div className="absolu
 <span classNam
 </div>
 }}
 {mode == "z" && step.z
 <div className="absolu
 <span classNam
 </div>
 }}
 {mode == "z" && step.z
 <div className="absolu
 <span classNam
 </div>
 }}

 {/* Found flash effect */}
 {step.found && mode == "
 <div className="absolu
 } else z<0 bg-emerald-500/20 shadow-[0_0_15px_#fff]
 }}
 {step.found && mode == "

```

```

.....
 <div className="text-sm text-slate-500">
 {step.desc}
 </div>
 </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl">

 <p className="text-xs text-slate-400 leading-relaxed">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значение {mode === 'kmp' ? "применяется" : "не применяется"}
 было найдено!
 Пройди шаги вручную, посмотри, как работает алгоритм
 для пропуска работы (в Z).
 </p>
</div>

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 px-4 py-2 border border-slate-700">
 Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})
 </div>
 <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 }
}
`}</pre>

```

```

.....
 className={`relative inline-flex ${className} ?? ""
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px]
 l duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full
 } ${open ? "visible translate-y-0 opacity-100"
 >
 {content}
 <span
 className={`absolute left-1/2 h-2 w-2 -transl
 side === "top" ? "bottom-[5px] border-b bo
 }`}
 />

);
};

```

ФАЙЛ 73/96: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

.....

import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...
import {
 Play,
 Pause,
 RotateCcw,

```

```

.....
 dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };

 recordStep(
 "Начало топологической сортировки. Используем кла

```

```

.....
 for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
 }

 recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
 setSteps(listSteps);
 setCurrentStepIdx(0);
 }, []);

 const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
 };

 useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

```



```

.....
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] rela
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
 </defs>

 {/* Edges */}

```

```

} else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textcolor = "text-emerald-300";
} else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textcolor = "text-indigo-200";
}

return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" "
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"

```

```

 getTopoListString()
 })
</div>
</div>

{/* Color key */}
<div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Назад
 </button>
 </div>

```

Универсальное пособие • полный исходный код

```
.....
* Узел идентифицируется парой id = "x:y".
*/

export interface Pair {
 key: number;
 grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
 id: NodeId;
 key: number;
 grade: number;
 left: TNode | null;
 right: TNode | null;
}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован */
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
```

```

}
const sub = go(t.left);
trace.push({ aId: t.id, goLeft: false, note: `(${t
 о левое поддерезо.` });
t.left = sub.r;
return { l: sub.l, r: t };
};
const res = go(root);
return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; }
}
export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, note: `Обе вершины
уточнее целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `Выбран узел
а, потому что его правое поддерезо с (${b.key}, ${b.grade})
меньше, чем у узла b.` });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `Выбран узел
b, потому что его левое поддерезо с (${a.key}, ${a.grade})
меньше, чем у узла a.` });
 b.left = go(a, b.left);
 return b;
 };
 return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): TNode | null {

```

```

.....
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

/* ----- раскладка сцены дерева -----

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id }); }
 if (n.right) { es.push({ from: n.id, to: n.right.id }); }
 };
 walk(root);
 return es;
};

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
}

```

```

.....
 <g key={`tn-${n.id}`} transform={`translate
 <circle r={pulse ? 24 : 20} fill={nodeColor}
 } strokeWidth={pulse ? 4 : 2} className="tr
 <text y={-2} fill="#fff" fontSize="13" fo
 <text y={13} fill="#cbd5e1" fontSize="10"
 </g>
);
)))
</svg>
 {subtitle && <p className="text-xs text-slate-400
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 + grade) * ((PL_H

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => vo
}

```

```

.....
 {interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDash
 />
)}

 {points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key ===
 const isSel = selected && selected.key === p.key
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ?
 onPointerUp={interactive && onPointUp ? () :
 style={interactive ? { cursor: "pointer" }
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={
 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8"
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
 ">{p.key}</text>
 </g>
);
)}}
)}}

 {pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.g
 >
 <text x={PX(pending.key)} y={PY(pending.grade
 ly="monospace">{pending.key}</text>
 </g>
)}
 </svg>
);
};

```



## Универсальное пособие • полный исходный код

```
.....
const hasParent = new Set(edges.map((e) => gpid(e.b))
const roots = all.filter((p) => !hasParent.has(gpid(p
const ok = (n: Pair | undefined, lo: number, hi: number) => {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
};
return roots.every((r) => ok(r, -Infinity, Infinity))
}

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
 const heap = edges.every((e) => e.a.grade > e.b.grade)
 const oneParent = new Set(edges.map((e) => gpid(e.b)))
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b))
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === p.key))
 const complete = edges.length === all.length - 1 && roots.length === 1
 return { heap, oneParent, acyclic, bst, complete, win: true }
}

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
 if (sameP(a, b)) return { ok: false, reason: "Это одна и та же точка" }
 if (a.grade === b.grade)
 return { ok: false, reason: "«!» равны – «кто выше?»" }
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpid(child)))
 return { ok: false, reason: `У точки ${child.key} уже есть родительская линия.` };
 if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
 const next = [...edges, { a: parent, b: child }];
 return { ok: true, reason: "ОК", next };
}
```

```

.....

/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
 a: Pair;
 b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
 edges: GameEdge[],
 x: Pair,
 y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
 if (x.key === y.key && x.grade === y.grade) return { ok: true };
 for (const e of edges) {
 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
 (pid(e.a) === pid(y) && pid(e.b) === pid(x))) {
 return { ok: false, reason: "такая линия уже проведена" };
 }
 }
 if (x.grade === y.grade) return { ok: false, reason: "не может быть" };
 const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
 if (edges.some((e) => pid(e.b) === pid(child))) {
 return { ok: false, reason: `у точки ${child.key} уже есть ребро` };
 }
 // цикл: родитель уже сидит в компоненте ребёнка (ребёнок уже в компоненте родителя)
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid(e.a))) {
 childComp.add(pid(e.a));
 grew = true;
 }
 }
 }
 if (childComp.has(pid(parent))) return { ok: false, reason: "уже есть ребро" };
 return {
 ok: true,
 edge: edges.find((e) => pid(e.a) === pid(parent) && pid(e.b) === pid(child)),
 };
}

```

```

.....
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v.key);
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0].key > sorted[1].key) {
 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяем
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
 };
 walk(roots[0]);
 if (seen.size !== n) bstOk = false;
 for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
}

```

```

.....
const [algo, setAlgo] = useState<SortAlgo>("none");
const [order, setOrder] = useState<ConnectOrder>("y-desc");
const [sortIdx, setSortIdx] = useState(0);
const [connIdx, setConnIdx] = useState(0);
const [playing, setPlaying] = useState(false);
const [sortCost, setSortCost] = useState<string | null>(null);
const live = useTreapLive();

const parsed = useMemo(() => parsePairs(raw), [raw]);
const userPairs = parsed.errors.length ? [] : parsed.pairs;

const n = userPairs.length;

const userTree = useMemo(() => (userPairs.length >= 1 ?
const canonEdges = useMemo(() => {
 const es: { a: Pair; b: Pair }[] = [];
 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, group: node.group, left: node.left }, b: node.left }); }
 if (node.right) { es.push({ a: { key: node.key, group: node.group, right: node.right }, b: node.right }); }
 };
 walk(userTree);
 return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges], [canonEdges]);

useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return; }
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.length : canonEdges.length;
 if (connIdx < total) setConnIdx(connIdx + 1);
 }
 }, 1000);
}, [playing, phase, algo, sortIdx, connIdx, order]);

```

```

.....
 setBadEdge(null);
 setHintUsed(false);
};
const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair);
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 setBadEdge({ a: x, b: y });
 window.setTimeout(() => setBadEdge(null), 900);
 }
};
const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) — пере
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
};

```



```

.....
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
 />
 <div className="mt-2 text-xs text-slate-300 bg-
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} па
 ть.</>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <>⚡ quicksort: счётчик сравнений: с
 {comparisons} ($n \cdot \log_2 n$).</>
) : (
 <> counting: точки в корзины по значени
 , потому что у – маленькие целые.</>
)
) : (
 <>Отсортировано ({sortCost ?? "способ не
 Игру </>
)
) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказан
 ево: линии вниз по y, обход по x отсортирован
) : gameMsg ? (
 <>⚠ {gameMsg}</>
) : (
 <>Клики точку-родителя (выше по «!»), за
)
) : connDone ? (
 <> Проведено {connEdges.length} линий в по
 : точки прибиты, дерево одно. А теперь
) : (
 <>Линий проведено: {connIdx} из {connEdgesA
)}
)}

```

## Универсальное пособие • полный исходный код

```
.....
 <div className="bg-emerald-950/20 rounded-xl p-4">
 <h4 className="text-sm font-bold text-emerald-500">История
 <TreeView root={gameTree} subtitle="Слева-направо существует." />
 </div>
 })

 {/* Дерево */}
 {order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">История
 <TreeView root={userTree} subtitle="Слева-направо существует." />
 </div>
)}
</div>
);
};

/* mode: 'split' или 'desc' РЕЖИМ SPLIT mode: 'split' или 'desc' */

const SplitMode: React.FC = () => {
 const [x0, setX0] = useState(7);
 const [stepIdx, setStepIdx] = useState(0);
 const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
 const split = useMemo(() => splitTree(buildTree(POINTS), fullTree), [fullTree]);
 const steps = split.trace;
 const live = useTreapLive();
 // Компилятор: x – ключ разреза, i – номер шага списка
 useEffect(() => { if (live.x !== null) setX0(live.x); }, [live.x]);
 useEffect(() => { if (live.i !== null) setStepIdx(Math.min(stepIdx, live.i)); }, [live.i]);
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;

 const inL = new Set<NodeId>();
 const inR = new Set<NodeId>();
 steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? inL.add(s.id) : inR.add(s.id)));

 const lTree = split.l;
```



```

.....
 <div className="bg-rose-950/20 rounded-xl p-3">
 <h5 className="text-sm font-bold text-rose-500">
 <TreeView root={rTree} width={480} height={400} />
 </div>
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900 p-3">
 <LiveChip live={live} />{' '}
 Как это работает на самом деле: идём с
 деревом), отдаётся вместе с веткой — д
 </div>
</div>
);
};

/* very old code from library that was not used anymore РЕЖИМ MERGE very old code from library that was not used anymore */

const MergeMode: React.FC = () => {
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 // Компилятор: i — номер шага слияния.
 useEffect(() => { if (live.i !== null) setStepIdx(live.i); }, [live.i]);
 const { l, r } = useMemo(() => splitTree(buildTree(P0, live.i)), [live.i]);
 const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)), [l, r]);
 const steps = merged.trace;
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;
 const cur = steps[idx];

 return (
 <div className="space-y-4">
 <div className="flex items-center gap-3">
 <button onClick={() => setStepIdx(Math.max(0, idx - 1))}
 className="bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(stepIdx, steps.length - 1))}
 className="font-bold bg-indigo-600 text-white disabled:opacity-30">
 war {i

```

```

.....
/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const fullTree = useMemo(() => buildTree(POINTS), [])
 const res = useMemo(() => eraseNode(buildTree(POINTS)
 const steps = res.trace;
 const live = useTreapLive();
 // Компилятор: x – ключ удаляемой вершины, i – шаг сп
 useEffect(() => { if (live.x !== null && POINTS.some(
 useEffect(() => { if (live.i !== null) setStepIdx(Math
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;

 const pathIds = useMemo(() => {
 const ids: NodeId[] = [];
 for (const s of steps.slice(0, idx + 1)) if (s.atId
 return ids;
 }, [steps, idx]);
 const pathSet = new Set(pathIds);

 return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Удалить
 <LiveChip live={live} />
 <select value={key} onChange={(e) => { setKey(N
 te-700 rounded-lg px-3 py-1.5 text-sm font-mon
 {POINTS.map((p) => <option key={p.key} value=
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 war {i
 </div>

```

```

.....
const full = useMemo(() => buildTree(POINTS), [1]);
// Честная слотовая разметка treap: индексы как в куче
const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null);
 const walk = (n: TNode | null, i: number) => {
 if (!n || i >= 16) return;
 arr[i] = n.id;
 walk(n.left, 2 * i);
 walk(n.right, 2 * i + 1);
 };
 walk(full, 1);
 return arr;
}, [full]);
const nodeCount = slots.filter(Boolean).length;

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 <button onClick={() => setPick("heap")} className="text-white bg-slate-700 text-slate-300">
 <button onClick={() => setPick("treap")} className="text-white bg-slate-700 text-slate-300">
 </div>

 {pick === "heap" ? (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">
 <div className="flex gap-2 overflow-x-auto mb-4">
 {heapArr.slice(1).map((v, i) => (
 <div key={i} className="px-3 py-2 rounded">
 <div className="text-slate-500 text-[10px]>
 <div className="text-white font-bold">{v}
 <div className="text-emerald-400 text-[10px]>
 </div>
))}
 </div>
 <p className="text-xs text-slate-400">Полное

```

## Универсальное пособие • полный исходный код

```
.....
// mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
const shuffled: Pair[] = [
 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
 { key: 45, grade: 8 },
 { key: 9, grade: 3 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 7, grade: 2 },
 { key: 1, grade: -4 },
];
const target = 7;
const sortedByKey = [...shuffled].sort((a, b) => a.key - b.key);
const [stage, setStage] = useState<0 | 1 | 2>(0); // 0 - не найдено, 1 - найдено, 2 - ошибка
const [bidx, setBidx] = useState(0);

// шаги бинарного поиска на отсортированном массиве
const bsSteps = useMemo(() => {
 const steps: { lo: number; hi: number; mid: number; }[] = [];
 let lo = 0, hi = sortedByKey.length - 1;
 while (lo <= hi) {
 const mid = Math.floor((lo + hi) / 2);
 if (sortedByKey[mid].key === target) { steps.push({ lo, hi, mid, found: true, noSteps: 0 }); }
 if (sortedByKey[mid].key < target) { steps.push({ lo, hi, mid, found: false, noSteps: 1 }); lo = mid + 1; }
 else { steps.push({ lo, hi, mid, found: false, noSteps: 2 }); hi = mid - 1; }
 }
 return steps;
}, [sortedByKey]);

const wrongMid = Math.floor((shuffled.length - 1) / 2);
const comparisons = stage === 2 ? bidx + 1 : 0;

return (
 <div className="space-y-4">
 <div className="flex gap-2">
```

```

.....
 реставил: массив как был в беспорядке, та
 {stage === 2 && bidx < bsSteps.length - 1 && <
 {stage === 2 && bidx >= bsSteps.length - 1 && <
 сравнения. Но чтобы поиск вообще заработал,
 сравнения на всех. «Отсортирую бинарни
</div>

<div className="text-xs text-slate-400 bg-slate-900" >
 Есть ли разница, чем сортировать? Сравн
 ная граница, каждое сравнение даёт 1 бит. Но
 ля маленьких целых у (вкладка «Собрать», кноп
 задать порядок соединения.
</div>
</div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
 const [edges, setEdges] = useState<GEdge[]>([]);
 const [sel, setSel] = useState<Pair | null>(null);
 const [hover, setHover] = useState<Pair | null>(null);
 const [msg, setMsg] = useState<string>("Кликни точку-");
 "));

 const v = gameVerdict(edges, POINTS);

 const click = (p: Pair) => {
 if (!sel) {
 setSel(p);
 setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кли
 return;
 }
 if (sameP(sel, p)) {
 setSel(null);
 return;
 }
 }

```

```

.....
 {edges.map((e, i) => {
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}
 stroke={i === edges.length - 1 ? "#34d399" : "#a5b4fc"}
 strokeWidth={i === edges.length - 1 ? 3.5 : 2}
 className="transition-all duration-300"
 />
 })}

 {POINTS.map((p) => {
 const isSel = sel && sameP(sel, p);
 const isHover = hover && sameP(hover, p) &&
 return (
 <g key={`gp-${p.key}:${p.grade}`} onClick={
 () => {
 if (isSel) {
 setSel(p);
 } else {
 setHover(p);
 }
 }
 } className="cursor-pointer">
 <circle cx={PX(p.key)} cy={PY(p.grade)}
 fill={isSel ? "#34d399" : isHover ? "#94a3b8" : "#38bdf8"}
 stroke="black" stroke-width="2"
 transition="all duration-200" />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5}
 space">{p.key}</text>
 <text x={PX(p.key)} y={PY(p.grade) + 24}
 space">{p.grade}</text>
 </g>
);
 })}
 </svg>
</div>

/* Судья – условия, как у Ахо–Корасика */
<div className={`rounded-xl p-4 border transition
 ate-900/40`} >
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {v.hear ? " " : " "} каждое ребро – вниз
 {v.oneParent ? " " : " "} у каждой точки

```

```
.....
}

const LiveChip: React.FC<{ live: { x: number | null; y:
 live.linked ? {

 Python: x={live.x ?? "-"} · y={live.y ?? "-"} ·

 } : null;
}>

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Пап
 </div>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}
 </button>
)))}
)
)
}
```

## Универсальное пособие • полный исходный код

```
.....
import { ComplexityClassesViz } from "../ComplexityClassesViz";
import { QueueViz } from "../QueueViz";
import { SalmonAutomatonWidget } from "../SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "../SegmentTreeVisualizer";
import SplayPlayground from "../SplayPlayground";
import { StackViz } from "../StackViz";
import { StringAlgorithmsViz } from "../StringAlgorithmsViz";
import { TopologicalSortViz } from "../TopologicalSortViz";
import { WaterfallAnimationWidget } from "../WaterfallAnimationWidget";
import ChapterImage from "../ChapterImage";
import { PrefixSumViz } from "../visualizers/PrefixSumVisualizer";
import { SparseTableViz } from "../visualizers/SparseTableVisualizer";
import { TreapBuildViz } from "../TreapBuildViz";

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки – это ровно то, что нужно для стека.",
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
```



```

.....
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"splay-rotations": {
 title: "Splay: механика → большое дерево → собери с
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
"top-sort": { title: "Топологическая сортировка", Component:
"scc-kosaraju": { title: "Компоненты сильной связности", Component:
"graph-components": {
 title: "Компоненты связности: BFS-разметка",
 hint: "Каждый запуск обхода красит один «остров»; ч
 Component: ComponentsViz,
},
"graph-articulation": { title: "Точки сочленения", Component:
"bridges-code": { title: "Мосты: DFS + tin/low", Component:
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component:
"planarity-euler-formula": {
 title: "Планарность: K5/K3,3 + раскраска",
 hint: "Слева – непланарные графы с подсветкой пересеч
 Component: () => (
 <div className="space-y-10">
 <PlanarityDemo />
 <ColoringViz />
 </div>
),
},
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем
 Component: () => (
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
),
},

```

```

 },
 },
 "complexity-classes": {
 title: "Классы сложности: матрёшка $P \subseteq NP \subseteq PSPACE$ ",
 hint: "Кликни класс или задачу; стрелки $\leq$ р показывают",
 Component: ComplexityClassesViz,
 },
 intro: { title: "Мнемокарточки", Component: MnemonicCards },
};

export const getViz = (id?: string): VizEntry | undefined

```

.....  
 ФАЙЛ 76/96: src/components/WaterfallAnimationWidget.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useState } from 'react';
import { useVizRuntime, vizNumber, vizString } from '...';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {

```

```

const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>(
 'Режим поиска: Введите текст и жмите «Следующий шаг»');
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({
 from: 0, to: 0 });
const [foundMatches, setFoundMatches] = useState<{ index: number; text: string }>({
 index: 0, text: '' });
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

// — Синхронизация с Python-компилятором: s/текст, i/индекс, v/вершина
// v — текущая вершина автомата. Влияет на режим поиска.
const runtime = useVizRuntime();
const liveText = vizString(runtime?.variables?.s) ?? '';
const liveI = vizNumber(runtime?.variables?.i);
const liveNode = vizNumber(runtime?.variables?.v) ?? 0;
useEffect(() => {
 if (liveText) {
 setTextToScan(liveText.toUpperCase().replace(/[\^A-Z]/g, ''));
 resetSearchSimulation();
 }
}, [liveText]);
useEffect(() => {
 if (liveI !== null) setCurrentIndex(Math.max(0, Math.min(liveI, textToScan.length)));
}, [liveI, textToScan.length]);
useEffect(() => {
 if (liveNode !== null && WATERFALL_NODES[liveNode])
 setFoundMatches({ index: liveNode, text: WATERFALL_NODES[liveNode] });
}, [liveNode]);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/[\^A-Z]/g, '');
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {

```

```

}
const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Логика

 setJumpPath(null);
} else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} не

 рыжок по fail-ссылке в #${curr} (${WATERFALL_TRANSITIONS[curr][char]}

 . . .);
 setJumpPath({ from: originalCurr, to: nextNode,
 });
 curr = nextNode;
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) => {
 currentMatches.push({ index: currentIndex, word });
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}

 setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMatches]);
}

```

```

.....
{
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель :
 а-разведчик плывет в ROOT и ищет ветку по "E".
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
 H" и "S" не подходят. Ветки нет, fail[HI] = ROOT
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТА
 desc: ' ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
). Рыба плывет в ROOT и ищет ветку "H". У корня
 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель :

```

```
.....
 checkingBranchesFrom: 0,
 },
};
```

```
const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainStep];
```

```
return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border" >
 { /* Шапка и переключатель режимов */ }
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400" >
 Визуализация пошагового построения всех функций
 </p>
 </div>
 </div>
 </div>
 </div>
```

```
{ /* Переключатель режимов */ }
<div className="bg-slate-900 p-1.5 rounded-xl border" >
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p-2`}
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 > </button>
```

```
>
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
</button>
<button
```

```

 >
 {step
 {idx === trainStep && <span className=
 </button>
 }}}
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg
ion-all"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === TRAINING_STEPS_
 className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>
</div>

 /* Блок с кодом построения fail-ссылки и опи
 <div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 ❌ Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

```

```

.....
 <div>
 <h5 className="text-slate-200 font-bold mb-2">
 <Lightbulb className="w-4 h-4 text-amber-500"/>
 </h5>
 <div className="bg-slate-800 p-4 rounded-xl w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
 </div>
 </div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3">
 <div className="bg-slate-900/60 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-2">
 ✎ Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4">
 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-blue-500
 focus:outline-none focus:border-blue-500"
 />
 </div>
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map((char, idx) =>
 <span

```



```
.....
 ❌ Текущее действие в коде
</h5>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <div className={`p-1.5 rounded flex items-center
border-blue-500 text-blue-300 font-bold` :
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && Инициализация
 </div>
 <div className={`p-1.5 rounded flex items-center
border-rose-500 text-rose-300 font-bold` :
 2. while (curr !== root && !trie[curr].fail) {
 {activeSearchCodeLine === 2 && Выход по fail!
 </div>
 <div className={`p-1.5 rounded flex items-center
border-blue-400 text-blue-200 font-bold` :
 3. curr = trie[curr].get(char) || root;
 {activeSearchCodeLine === 3 && Продолжение по течению
 </div>
 <div className={`p-1.5 rounded flex items-center
border-emerald-500 text-emerald-300 font-bold` :
 4. if (is_terminal[curr]) report(curr);
 {activeSearchCodeLine === 4 && Получение овладения!
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold mb-2">
 <AlertCircle className="w-3.5 h-3.5 text-slate-400"/>
 </h5>
 <div className="bg-slate-800 p-3.5 rounded-3xl flex items-center">
 {searchLog}
 </div>
</div>
```

```

.....
 { depth: 1, y: 160, label: 'Depth 1 (H, S'
 { depth: 2, y: 260, label: 'Depth 2 (HE, I'
 { depth: 3, y: 360, label: 'Depth 3 (HER, '
 { depth: 4, y: 460, label: 'Depth 4 (HERS
].map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14"
 />
 <text x="32" y={level.y + 4} fill="#94a0b4"
 {level.label}
 </text>
 </g>
)})

 /* Рендер прямых переходов (потоков воды)
 {Object.entries(TRIE_TRANSITIONS).map(({[fromId, toId]) => {
 const fromNode = WATERFALL_NODES[Number(fromId)];
 return Object.entries(children).map(({[childId, child]) => {
 const toNode = WATERFALL_NODES[toId];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ?
 type === 'normal';

 // Подсветка в режиме обучения (когда и
 let isTrainingExamined = activeMode === 'train' ?
 let isMatchBranch = isTrainingExamined &

```

```

 });
 });
}}}

{/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.fail];

 // В режиме обучения показываем fail-ссылку
 // Найдем индекс шага, на котором обработали
 const stepIdxForNode = TRAINING_STEPS_INFO[node.fail];
 if (activeMode === 'training' && trainStep === stepIdxForNode) {

 let isHighlighted = activeMode === 'search' || activeMode === 'fail';
 let isJustEstablished = activeMode === 'training' && !isHighlighted;

 // Вычисляем изогнутую кривую для красоты
 const dx = targetNode.x - node.x;
 const dy = targetNode.y - node.y;
 const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -10 : 10);
 const cy = node.y + dy / 2 - 30;

 return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
 fill="none"
 stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
 strokeWidth={isHighlighted || isJustEstablished ? 2 : 1}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b98f">

```

```

.....
 y={node.y + 5}
 fill={isCurrent || isTargetChild ?
 fontSize={isCurrent || isTargetChild
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме об
 {isTargetChild && (
 <g transform={`translate(${node.x},
 <rect x="-35" y="-12" width="70"
 <text x="0" y="2" fontSize="10" f
 ИЩЕМ FAIL
 </text>
 </g>
)}

 {/* Индикатор словарных слов */}
 {hasWords && (
 <g transform={`translate(${node.x +
 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
)}
 </g>
)}
);
)}
)}

 {/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
 <g>
 {/* Круг-подложка под рыбу */}
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}

```

```
.....
 key={idx}
 className="bg-emerald-950 border border-
-sm font-bold shadow animate-[scaleUp_0.3s_
>
 <CheckCircle2 className="w-4 h-4 text
 {match.word}
 <span className="text-[10px] bg-emera
 Индекс {match.index}

 </div>
)})
 </div>
})
</div>
};
};
```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 77/96: src/components/visualizers/PrefixSumViz.tsx  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 415 | РАЗ

```
import React, { useContext, useEffect, useMemo, useStat
import {
 emitVizDemo,
 useVizRuntime,
 vizArray,
 vizNumber,
 VizChapterContext,
} from "../../data/vizStepBus";
```

```

 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
 mode === key ? "bg-indigo-600 text-white"
 `}
 >
 {label}
 </button>
))}
</div>
{mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]],
 []
);
};

```

```

/** Что сейчас под курсором: префикс P[i] или элемент
const [hover, setHover] = useState<{ kind: "pref" | "el" }>({ kind: "pref" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, "P");
const compilerLinked = liveI !== null || hasLiveP;
// i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked

```

```

.....
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r
 className={` ${cellBase} cursor-pointer
 isDrivenAdd
 ? "bg-amber-500 text-black ring-2
 : inCover
 ? "bg-indigo-500 text-white rin
 : inRange
 ? "bg-indigo-900/70 text-indi
 : "bg-slate-800 text-slate-300
 `}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const liveValue = liveP?.[i];
 const blank = hasLiveP && (liveValue ===
 const masked = !hasLiveP && shownDriven !=
 const isDrivenCur = shownDriven !== null &
 const shownValue = hasLiveP ? liveValue :
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-help $
 isDrivenCur

```

```

 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[{hover.i}]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[{hover.i}]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.
 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку,
 </p>
)
})
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1
 {pref[range
 {pref[range.l

```



```

const c2 = vizNumber(vars?.c2);
if (r1 === null || c1 === null || r2 === null || c2
// Python-шаблон (и глава) используют 1-индексные к
// клетка (1, 1) – левый верхний элемент матрицы. В
// 0-индексные, поэтому переводим: (1,1) → (0,0).
const to0 = (v: number, hi: number) => Math.max(0,
setRect({
 r1: to0(r1, n - 1),
 c1: to0(c1, m - 1),
 r2: to0(r2, n - 1),
 c2: to0(c2, m - 1),
}));
}, [vars, n, m]);

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
 if (i === rect.r2 + 1 && j === rect.c2 + 1) return
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 /* Исходная матрица */
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => (
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {

```

```

.....
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
return (
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={`${cellBase} cursor-he
 isCompilerCell
 ? "bg-emerald-600 text-white
 : isHover
 ? "bg-amber-500 text-white
 : blank
 ? "bg-slate-950 text-slate
 : cornerColor
 `}
 title={hasLiveS ? `S[${i}][${j}]` :
 >
 {blank ? "." : String(shownValue)}
 {corner && !isHover && (
 <span className="absolute -top-
ht">
 {corner}

)}
 </div>
);
 }}}
</div>
)))
</div>
</div>
</div>

```

```

.....
 vizNumber,
 vizRecord,
 VizChapterContext,
} from ".../.../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
 }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];

```



```

.....
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >

```

```

.....
 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}} = \{1 \ll hover.j\} : \{ " " \}$
 <span className="font-mono text-slate-400"
 min({arrID.slice(covered.from, covered.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое чи
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-
 <div className="grid grid-cols-{auto_repeat(4
 {/* Headers */}
 <div className="text-slate-500 font-mono te
 i \ j
 </div>
 <div className="text-center font-bold text-
 rap">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-
 rap">
 2^3 (L=8)
 </div>

```

```

// уровень j=0 всегда берём из уже сущ
const shownValue = compilerLinked ? r
const displayValue =
 shownValue === undefined || shownVa
 ? "."
 : typeof shownValue === "object"
 ? JSON.stringify(shownValue)
 : String(shownValue);

return {
 <div key={j} className="relative fl
 {!isValid ? (
 <div className="w-[clamp(30px,9
) : (
 <motion.div
 onMouseEnter={() => (compiler
 onMouseLeave={() => setHover(
 onClick={() => (compilerLinker
 }}}

 initial={{ opacity: 0, scale:
 animate={{
 // При st = [] каркас уже с
 opacity: compilerLinked ? 1
 scale: compilerLinked || is
 }}
 className={`w-[clamp(30px,9vw
clamp(11px,3vw,17px)] font-bold transition-
 hover && hover.i === i && h
 ? "bg-sky-500 text-white
 : isActive
 ? "bg-indigo-500 text-whi
 : isSrc1
 ? "bg-emerald-500 text-r
 : isSrc2
 ? "bg-amber-500 text-r
 : isVisible
 ? "bg-slate-800 tex
 : compilerLinked

```

```
.....
</div>
```

```

/* Formula helper text */
<div className="bg-slate-900 border border-slate-
 {compilerLinked && liveTarget ? (
 <p className="text-base text-slate-300">
 <span className="text-emerald-400 font-bold
 i =
 – активна ячейка <span className="font-mono
 {(hasLiveTable || hasLiveSource) && <= <span
 ?.[liveTarget.j] ?? (liveTarget.j === 0 ? \
 </p>
) : step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps(step - 1);
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][[{c.j}]]
 {" "}
 = min(
 <span className="text-emerald-400 font-
 ST[{c.i}][[{c.j} - 1]]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][[{c.j} - 1]]

) → min(
 {st1

 {st1D[{c.i + half}][[{c.j} - 1]]}

) ={" "}
 <span className="text-indigo-400 font-b
 {st1D[{c.i}][[{c.j}]]}


```



```

// Уровень и клетка берутся прямо из k/r/c; номер строки
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 scroll-smooth">
 <h3 className="text-xl font-bold text-indigo-400">
 <p className="text-sm text-slate-400 mb-6 text-slate-500">
 Для наглядности показываем только <b className="text-white">

 иц!
 </p>
 </h3>
 <div className="flex bg-slate-950 p-1 rounded-lg">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setUserPicked(false); }}
 className={`px-4 py-2 rounded-md text-sm text-white text-slate-400 hover:text-white hover:bg-slate-900`}
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
 </button>
))}
 </div>
 {userPicked && (
 <button
 onClick={() => { setUserPicked(false); setK(0); }}
 className="mb-4 text-[11px] font-bold px-3 py-2 text-white hover:border-indigo-500 transition-colors"
 >

```

```

.....
 ? ST2D[r][c][step][step]
 : liveValue !== undefined &&
 ? liveValue
 : aCell !== null
 ? aCell
 : undefined;
const isBlank = shownValue ===
let highlightClass = isBlank
 ? 'bg-slate-950 text-slate-600'
 : isCurr
 ? 'bg-slate-800/80 text-slate-600'
 : 'bg-slate-800/50 text-slate-600';
let zIndex = 'z-0';

const isActive = !!activeCell &&
 (l.c + L) && ((r - activeCell.r) % (1 << st

if (isActive) {
 if (isCurr) {
 highlightClass = 'bg-indigo-500 text-white';
 } else {
 const prevL = 1 << (shownK
 const isTop = r < activeCell.r;
 const isLeft = c < activeCell.c;

 if (isTop && isLeft) {
 highlightClass = 'bg-rose-500 text-white';
 } else if (isTop && !isLeft) {
 highlightClass = 'bg-blue-500 text-white';
 } else if (!isTop && isLeft) {
 highlightClass = 'bg-emerald-500 text-white';
 } else {
 highlightClass = 'bg-amber-500 text-white';
 }
 zIndex = 'z-10';
 }
}

```



```

.....
 tiveCell.r][activeCell.c + prevL][shownK-1]
 </div>
 <div className="flex item
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}())}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][shownK
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : (
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L
 </div>
)}
</div>
)}
</div>
</div>
</div>
);

```



```

 </p>
 <p className="text-sm font-sans text-slate-500">
 Чтобы покрыть весь прямоугольник, мы
 Name="font-mono bg-[#0a0f1e] px-1 rounded b
 роса. (Цветные прямоугольники слева).
 </p>

 <div className="bg-[#0a0f1e] rounded-xl p-3
 text-slate-300 shadow-inner overflow-x-auto">
 <div className="absolute right-3 top-3">
 // 1.
 int<
 ext-amber-300">1); <span className="
 int<
 ext-amber-300">1); <span className="
 int<
 "text-slate-500">// Выс: {Lx}

 int<
 "text-slate-500">// Шир: {Ly}
<b
 // 2.
 int<

 В запроса)

 r1][<span className="text-white bg-r

 во, чтобы правый край уперся в c2)<b

 r1][<span className="text-white bg-b

 px, чтобы нижний край уперся в r2)<b

 nded">r2 - Lx + 1][<span className="


```

```

.....

 {txt}
 {badge && <div className=
r-slate-700 text-white px-1 font-bold rounded
 </div>
)
 }}}
 </div>
</div>
</div>

 <div className="mt-auto bg-slate-950 font
)}">
 <div className="text-center font-bold
wrap bg-[#0a0fle] py-3 px-2 rounded-lg bord
 <span className="text-slate-400 font

 <span className="text-blue-400 m
n>
 <span className="text-emerald-40
span>
 <span className="text-amber-400
 <span className="text-slate-4
 <span className="ml-3 text-emerald-
+ 1]][c1]][kx]][ky], ST2D[r2 - Lx + 1]][c2 - Ly
 </div>
 </div>

</div>
</div>
)
}

```

```

.....
 if (questionsContainer) questionsContainer
 if (wtfContainer) wtfContainer.classList
 }
}

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]')
 btn.classList.remove('outline', 'outline
 });

 if (theme === 'dark') document.getElementById
 if (theme === 'light') document.getElementById
 if (theme === 'terminal') document.getElemen
);

 document.getElementById('btn-theme-dark').c
 t-white transition-colors' : 'px-3 py-1 rou
 n-colors';
 document.getElementById('btn-theme-light')..
 ext-white transition-colors' : 'px-3 py-1 r
 ion-colors';
 document.getElementById('btn-theme-notebook
 -600 text-white transition-colors' : 'px-3
 ransition-colors';
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highligh
document.addEventListener('DOMContentLoaded', (
 const mobileMenuBtn = document.getElementById
 const mobileDrawer = document.getElementById

```



```

.....
 }
 });
 });
 });

 const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
 };

 const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href
 link.classList.add('bg-
 } else {
 link.classList.remove('
 }
 }
 }));
 }
));

 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
});
</script>
</body>
</html>

```

```
.....
.uno-card.mini::before, .uno-card.mini::after {

/* Global formula scroll settings */
.formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
}
.formula-scroll::-webkit-scrollbar {
 display: none;
}

mjx-container {
 max-width: 100% !important;
}
mjx-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
}
mjx-container[display="true"]::-webkit-scrollbar {
 display: none;
}

@media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px;

 .uno-card.mini { width: 28px !important; he
```

```

}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-color:
body.theme-light .bg-slate-700, body.theme-light
 important; }
body.theme-light .text-slate-200, body.theme-light
body.theme-light .text-slate-400 { color: #475568; }
body.theme-light .border-slate-600, body.theme-light
body.theme-light .border-slate-500 { border-color:
body.theme-light #top-controls { background-color:
}

body.theme-notebook {
 background-color: #f8d7da !important; /* light pink */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(45deg,
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook
 .bg-slate-700 {
 background-color: rgba(255, 255, 255, 0.7);
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook
 .text-slate-400 { color: #475568; }
body.theme-notebook .text-slate-400 { color: #475568; }
body.theme-notebook .border-slate-600, body.theme-notebook
 .border-slate-500 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f8d7da; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
}

```

```

.....
 <button onclick="setTheme('notebook')"
 over:text-white hover:bg-slate-700 transition
 </div>
</div>
</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
 </marker>
 <marker id="arrow-orange" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d"
 </marker>
 <marker id="arrow-pink" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1"
 </marker>
 <marker id="arrow-cyan" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
 </marker>
 <marker id="arrow-indigo" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
 </marker>
 <marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d"
 </marker>
 <marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80"
 </marker>

 <!-- ЛЕГО БЛОКИ -->
 <g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>

```

```

<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform
 <svg xmlns="http://www.w3.org/2000/svg" class="
 <path stroke-linecap="round" stroke-linejoin="round"
 </svg>
</button>

```

```

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed inset-0

```

```

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-col

```

```

<!-- Левое меню (Оглавление) -->

```

```

<aside id="mobile-drawer" class="fixed inset-y-0 left-0
 nslate-x-full transition-transform duration-300
 w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
 <div class="h-full flex flex-col p-6 overflow-y-auto
 lg:rounded-2xl lg:border lg:border-slate-700
 <div class="flex justify-between items-center
 <h3 class="font-black text-transparent bg-clip-text
 ider text-sm">

```

```

 Содержание

```

```

 </h3>

```

```

 <button id="close-drawer-btn" class="lg:hidden
 <svg xmlns="http://www.w3.org/2000/svg" class="
 <path stroke-linecap="round" stroke-linejoin="round"
 </svg>
 </button>

```

```

 </div>

```

```

 <nav class="space-y-3 text-sm font-medium"

```

```

 <details class="group" open>

```

```

 <summary class="list-none cursor-pointer"

```

```



```

```

 1. Алгебраические структуры

```

```



```

```

 </summary>

```

```
.....
 <summary class="list-none cursor
 <a href="#question-4" class=
er-amber-500 pl-3 font-bold text-amber-300">
 4. Степени и корни

</summary>
<div class="pl-6 space-y-2 text
 <a href="#q4-def" class="block
 <a href="#q4-moivre" class="blo
 <a href="#q4-root" class="block
 <a href="#q4-count" class="bloc
</div>
</details>

<details class="group">
 <summary class="list-none cursor
 <a href="#question-5" class=
er-fuchsia-500 pl-3 font-bold text-fuchsia-
 5. Группа корней из единицы

</summary>
<div class="pl-6 space-y-2 text
 <a href="#q5-def" class="block
 <a href="#q5-group" class="bloc
 <a href="#q5-prim" class="block
 <a href="#q5-crit" class="block
 <a href="#q5-cyclic" class="blo
</div>
</details>

<details class="group">
 <summary class="list-none cursor
 <a href="#question-6" class=
er-lime-500 pl-3 font-bold text-lime-300">
 6. Делимость, НОД и Евклид

</summary>
```

```
.....

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-green-500 pl-3 font-bold text-green-400">
 9. Кольцо многочленов

 </summary>
 <div class="pl-6 space-y-2 text-green-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

 </summary>
 <div class="pl-6 space-y-2 text-rose-500">

a>

 </div>
</details>
```

```


 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q13-1" class="blo
 <a href="#q13-2" class="blo
 <a href="#q13-3" class="blo
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-14" clas
der-fuchsia-500 pl-3 font-bold text-fuchsia
 14. Кратность корня. Ко

 </summary>
 <div class="pl-6 mt-2 space-y-2
 <a href="#q14-1" class="blo
 <a href="#q14-2" class="blo
 <a href="#q14-3" class="blo
deg)
 <a href="#q14-4" class="blo
ition-colors border border-rose-500/30"> K
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-15" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2
 <a href="#q15-1" class="blo
 <a href="#q15-2" class="blo
 <a href="#q15-3" class="blo
 <a href="#q15-4" class="blo

```



```
.....
 <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuc

 <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m
 Δ Ликбез: Алгоритмы выживания

</nav>
<script>
 document.querySelectorAll('details.
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA
 if (other !== detail
 other.removeAtt
 }
 });
}
});
});

// Scroll spy logic
document.addEventListener('DOMConten
 const sections = document.query
="q5-"], div[id^="q6-"], div[id^="q7-"], di
v[id^="q13-"], div[id^="q14-"], div[id^="q1
 let isScrolling = false;

 document.querySelectorAll('.gro
 link.addEventListener('clie
 isScrolling = true;
 setTimeout(() => isScro
 });
 });
```

```

.....
 observer.observe(section);
 });
});

function fallbackModal(contentRaw,
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = 'black';
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'column';
 modal.style.alignItems = 'center';
 modal.style.justifyContent = 'center';

 const box = document.createElement('div');
 box.style.width = '80%';
 box.style.height = '80%';
 box.style.backgroundColor = '#1a1a1a';
 box.style.padding = '20px';
 box.style.borderRadius = '10px';
 box.style.display = 'flex';
 box.style.flexDirection = 'column';
 box.style.color = '#fff';

 const header = document.createElement('div');
 header.innerText = 'Экспорт в ' + sectionName;
 header.style.marginBottom = '10px';

 const subHeader = document.createElement('div');
 subHeader.innerText = 'Из-за не

ние веб-студии в новой вкладке (через меню)';
 subHeader.style.marginBottom = '10px';
 subHeader.style.color = '#f87171';
 subHeader.style.fontSize = '0.9em';

```

```

.....
 closeBtn.style.color = '#fff';
 closeBtn.style.border = 'none';
 closeBtn.style.borderRadius = '10px';
 closeBtn.style.cursor = 'pointer';
 closeBtn.onclick = () => modal.close();

 box.appendChild(header);
 box.appendChild(subHeader);
 if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(closeBtn);
 }
 btnContainer.appendChild(closeBtn);
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal);
 }

function downloadMD() {
 let clone = document.querySelector('#md-content');

 // Smart markdown parsing based on DOM

 // Smart markdown parsing based on DOM
 clone.querySelectorAll('h2').forEach(el => {
 clone.querySelectorAll('h3').forEach(el => {
 clone.querySelectorAll('.analog').forEach(el => {
 clone.querySelectorAll('li').forEach(el => {
 clone.querySelectorAll('svg').forEach(el => {
 let caption = '';
 if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
 caption = el.nextElementSibling.alt;
 }
 el.textContent = '\n\n[Изображение: ' + caption + ']';
 });
 });
 });
 });
 });
 clone.querySelectorAll('.img-caption').forEach(el => {
 let text = clone.innerText;

```

.....

```
body.bg-white section h1 { color: #334155; }
body.bg-white main { color: #334155; }
body.bg-white main p { color: #334155; }
/* Universal text colors */
body.bg-white .text-slate { color: #334155; }
body.bg-white .text-slate { color: #334155; }
body.bg-white .text-slate { color: #334155; }
body.bg-white [class*="text-"] { color: #334155; }

/* Specific backgrounds */
body.bg-white aside > div { background-color: #f1f3f4; }
body.bg-white [class*="bg-"] { background-color: #f1f3f4; }

body.bg-white [class*="bg-"] { background-color: #f1f3f4; }

body.bg-white [class*="bg-"] { background-color: #f1f3f4; }
body.bg-white .analogy-block { background-color: #f1f3f4; }
body.bg-white .img-placeholder { background-color: #f1f3f4; }
body.bg-white .info-block { background-color: #f1f3f4; }

body.bg-white .analogy-block { background-color: #f1f3f4; }
or: #94a3b8 !important; }
body.bg-white .img-caption { background-color: #f1f3f4; }

/* Tweak card blocks */
body.bg-white .card-red { background-color: #f1f3f4; }

body.bg-white aside { background-color: #f1f3f4; }
#334155 !important; }
body.bg-white .text-white { color: #334155; }

/* Ensure active links are visible */
body.bg-white aside a.text-white { color: #334155; }
body.bg-white aside a.text-white { color: #334155; }
body.bg-white aside a.text-white { color: #334155; }
body.bg-white aside a.text-white { color: #334155; }
body.bg-white aside a.text-white { color: #334155; }
```

```

.....
 document.head.appendChild(style
 }
</script>

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И
<div class="mt-8 flex flex-col gap-4 bo
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-
 <button onclick="downloadWord()
ont-bold flex items-center justify-center g
 Word
 </button>
 <a href="algebra.pdf" target="_
-sm font-bold flex items-center justify-cen
 PDF

 <button onclick="downloadMD()"
-bold flex items-center justify-center gap-
 Markdown
 </button>
 <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-ce
 HTML
 </button>
 </div>

<!-- Смена темы -->
<div>
 <div class="text-xs text-slate-
 <div class="flex items-center g
 <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
 <button id="theme-light" on
r justify-center text-amber-50 hover:text-wh

```

```

.....
 13.
 14.
 15.
 17.
 18.
 19.
 20.
 21.
 22.

</div>
</header>

<!-- WTF CONTAINER -->
<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4"
 <div class="grid grid-cols-3 gap-4"
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slat

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
 <div class="text-xs text-orange
 <div class="font-bold text-slat


```

```

.....
 <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>
 <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

```

```
.....

 <a href="#question-22" onclick="setf
-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia
 <div class="font-bold text-slate

 <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
 <div class="col-span-1 md:col-span-1
border-b border-red-500/30 pb-2">Кластер 4:

 <a href="#proof-algorithms" onclick=
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
 <div class="text-xs text-red-500
 <div class="font-bold text-slate

</div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
 <div class="flex items-center mb-6"
 <div class="bg-red-500/20 p-3 r
 <svg class="w-8 h-8 text-re
rg/2000/svg">
 <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border
</div>

 <div class="prose prose-invert max-
 <p class="text-lg">
 Хватит тупить. Доказательств
 есь для того, чтобы жать на кнопки. Вот жес
 </p>
```



```
.....
 <span class="text-w
счет ограничений (размер остатка) докажи, ч

</div>
```

```
<!-- Индукция -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-yel
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 sp
 <span class="text-w
валось.
 <span class="text-w
Просто перепиши формулу с буквой k .
 <span class="text-w
усок k , замени его на жареного карася из

</div>
```

```
<!-- Двойная делимость -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-ora
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 sp
 <span class="text-w
вверх по уравнениям).
 <span class="text-w
рху вниз, показывая, что $\Delta \vdots r_i$
 <span class="text-w
х. Твой d больше.

</div>
```

```

 <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md
```

```
.....
 <ul class="space-y-4 pb-10">
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 <code class="text-x
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 x двух, потом последних.

 <code class="text-x
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 есть сложение – есть ноль. Если есть умноже
 <code class="text-x
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 авь их драться. Умножение всегда врывается :
 <code class="text-x
 </div>
```

## Универсальное пособие • полный исходный код

```
.....
* Выгрузка главы (или всего пособия) в самостоятельный
*
* Файл получается автономным: стили защиты внутрь, инт
* его можно открыть с флешки, отправить в мессенджер и
* Живые React-визуализации в статику не переносятся –
* подставляется заметка со ссылкой на онлайн-версию, а
* которые в приложении всплывают по наведению, собираю
* в конце документа.
*/

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
 }
 return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; paddi
.export-head { border-bottom: 1px solid #1e293b; paddin
```

```

.....
 host.querySelectorAll<HTMLElement>(".term-hint").forEach(
 const id = e.dataset.termId;
 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = e.textContent ?? "";
 e.replaceWith(link);
));

 // <details> в файле лучше раскрыть: иначе код и разб
 host.querySelectorAll("details").forEach((d) => d.set

 return { body: host.innerHTML, termIds: ids };
 }

function glossarySection(termIds: string[]): string {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escapeHtml(e.title)}>
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
</div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
 ны сюда.</p>
${items}

```

```

}

function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 document.body.appendChild(a);
 a.click();
 a.remove();
 setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» – без кириллицы и пробелов */
function safeName(title: string, id: string): string {
 const num = title.match(/^s*(\d+)/)?[1];
 const base = id.replace(/[^a-z0-9-]/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body, termIds } = prepareBody(chapter.content);
 const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
 triggerDownload(html, safeName(chapter.title, chapter.id));
}

/** Выгрузить всё пособие одним файлом со сквозным оглавлением */
export async function downloadBookHtml(chapters: Chapter[]) {

```

## Универсальное пособие • полный исходный код

ФАЙЛ 83/96: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * {блок «Аналогия»} и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
```

```

}
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(`  * ${r.title}  {`);
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

ФАЙЛ 84/96: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});

```

```
.....
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlight
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 1
);
}
console.log("\nГлав без единой подсказки:", bad.length,
console.log("Терминов в глоссарии:", GLOSSARY.length, "
console.log("Не встретились:", GLOSSARY.filter((g) => !
```

.....  
ФАЙЛ 85/96: scripts/export/build-guide-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 28 | РАЗМ

```
.....
/**
 * Оркестратор: бандлит TSX-генератор и запускает его.
 * Использование: node scripts/export/build-guide-pdf.m
 */
import { build } from "esbuild";
import { pathToFileURL } from "node:url";
import path from "node:path";
import fs from "node:fs";

const root = path.resolve(import.meta.dirname, "../..");
const outfile = path.join(root, "tmp/guide-pdf-bundle.c

await build({
 entryPoints: [path.join(root, "scripts/export/guide-p
 outfile,
 bundle: true,
 platform: "node",
 format: "cjs",
 target: "node20",
```



```
.....

if (pages.length === 0) throw new Error("Не найдено ни одного файла");

ensureDir(OUT_DIR);

const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`    страниц: ${pages.length}`);
console.log(`    выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
});
```

## Универсальное пособие • полный исходный код

```
.....
 [/^src\/components\/\\/, "Интерактивные компоненты и сим
 [/^src\/layout\/\\/, "HTML-разметка (layout)"],
 [/^src\/utils\/\\/, "Утилиты"],
 [/^src\/\\/, "Ядро приложения"],
 [/^scripts\/\\/, "Скрипты сборки и экспорта"],
 [/^\.github\/\\/, "CI / автоматизация"],
 [/^[^\/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([rel]) => r

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
```

```

.....
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
 });
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода: ${humanSize(totalByt
 push(`Глав/билетов в пособии:        ${chapters.length}`)
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
 push(HR);
 chapters.forEach((c, i) => {
 push(` ${String(i + 1).padStart(3, " ")} ${c.titl
 });
 push();

 push(HR);

```

```

.....
 push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сген
 push(HR);

 const txt = out.join("\n") + "\n";
 fs.writeFileSync(TXT_PATH, txt, "utf8");

 console.log("[1/3] код -> txt");
 console.log(`      файлов:  ${files.length}`);
 console.log(`      глав:    ${chapters.length}`);
 console.log(`      строк:   ${txt.split("\n").length}`);
 console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

ФАЙЛ 88/96: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

```

/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся браузеру) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */

```

```
.....
 execFileSync(cmd, ["-version"], { stdio: "ignore"
 return cmd;
 } catch {
 /* пробуем следующий */
 }
}
throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get
);
}
```

.....  
ФАЙЛ 89/96: scripts/export/guide-pdf-entry.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 673 | РАЗМЕР: 10.5 КБ

```
.....
/**
 * Автосборка PDF-конспекта `guide_all_topics.pdf`.
 *
 * Что внутри:
 * • весь ТЕКСТ страниц пособия (больше текста, чем HTML-версия);
 * • картинки визуализаций – React-компоненты рендерятся
 * {renderToStaticMarkup}), из разметки берутся SVG-снимки;
 * • НИКАКОГО кода компилятора и интерфейсных кнопок.
 *
 * Результат: public/export/guide_all_topics.pdf, ~40–70 КБ.
 */
```

```
import * as fs from "node:fs";
import * as path from "node:path";
import { createElement } from "react";
import { renderToStaticMarkup } from "react-dom/server";
import PDFDocument from "pdfkit";
// @ts-ignore - у пакета нет типов
import SVGtoPDF from "svg-to-pdfkit/source.js";
import { chapters } from "../../src/data/content";
import { VizChapterContext } from "../../src/data/vizSt...
```

Универсальное пособие • полный исходный код

```
.....
 .replace(/&#(\d+);/g, (_, d) => String.fromCodePoint
 .replace(/&[a-zA-Z#0-9]+;/g, (e) => ENTITIES[e] ??
}
```

/\*\* Убрать инлайн-теги, оставив чистый текст с пробелами

```
function stripTags(html: string): string {
 return decodeEntities(
 html
 .replace(/<(script|style){\s\S}*?<\/\1>/gi, " ")
 .replace(/<br\s*\n?>/gi, "\n")
 .replace(/<\/(p|div|li|h[1-6]|tr|summary)>/gi, "\n")
 .replace(/<[^\>]+>/g, " ")
)
 .replace(/[\t]+/g, " ")
 .replace(/ *\n */g, "\n")
 .replace(/\n{2,}/g, "\n")
 .trim();
}
```

/\*\* Найти закрывающий тег с учётом вложенности однотипных

```
function findClose(html: string, openEnd: number, tag: string): number {
 const re = new RegExp(`<\/?${tag}(\s[^\>]*)?>`, "gi")
 re.lastIndex = openEnd;
 let depth = 1;
 let m: RegExpExecArray | null;
 while ((m = re.exec(html))) {
 depth += m[0][1] === "/" ? -1 : 1;
 if (depth === 0) return m.index;
 }
 return html.length;
}
```

```
function findCloseAny(html: string, openEnd: number, tag: string): { end: number; inner: string } {
 const close = html.toLowerCase().indexOf(`<\/${tag}`);
 if (close < 0) return { end: html.length, inner: html.slice(openEnd, html.length) };
 const inner = html.slice(openEnd, close);
 const end = html.indexOf(">", close) + 1;
 return { end, inner };
}
```

```

.....
 if (text) blocks.push({ kind: "h", level: Number(
 i = close < 0 ? html.length : close + 5;
 continue;
}
if (lower.startsWith("<summary", at)) {
 const openEnd = html.indexOf(">", at) + 1;
 const { end, inner } = findCloseAny(html, openEnd);
 const text = stripTags(inner);
 if (text) blocks.push({ kind: "p", text: "• " + text });
 // содержимое details выводим тоже: оно идёт сразу
 const detEnd = html.toLowerCase().indexOf("</details>");
 if (detEnd >= 0) {
 blocks.push(...htmlToBlocks(html.slice(end, detEnd)));
 i = detEnd + "</details>".length;
 } else i = end;
 continue;
}
if (lower.startsWith("<li", at) || (liM && at === 1)) {
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("", at);
 const text = stripTags(html.slice(openEnd, close));
 if (text) blocks.push({ kind: "li", text });
 i = close < 0 ? html.length : close + 5;
 continue;
}
if (lower.startsWith("<tr", at)) {
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("</tr>", at);
 const rowHtml = html.slice(openEnd, close < 0 ? html.length : close);
 const cells: string[] = [];
 const tdRe = /<(t{dh})(\s{^>}*)?>([\s\S]*?)<\/\1>/;
 let tm: RegExpExecArray | null;
 while ((tm = tdRe.exec(rowHtml))) cells.push(stripTags(tm[3]));
 if (cells.some(Boolean)) blocks.push({ kind: "table", cells });
 i = close < 0 ? html.length : close + 5;
 continue;
}
// <p>

```

```

.....
 const nc = html.toLowerCase().indexOf("</svg>",
 if (nc < 0) { pos = html.length; break; }
 if (no >= 0 && no < nc) { depth++; pos = html.in
 else { depth--; pos = nc + 6; }
 }
 let svg = html.slice(at, pos);
 if (!/xmlns=/.test(svg)) svg = svg.replace("<svg"
 // выкинуть элементы с tailwind-классами без инла
 svgs.push(svg);
 idx = pos;
}
const imgRe = /<img[^>]*src="(data:image\/(png|jpeg
let im: RegExpExecArray | null;
while ((im = imgRe.exec(html))) images.push(im[1]);
const grids = extractDivGrids(html);
return { svgs, images, grids };
} catch (e) {
 return { svgs: [], images: [], grids: [], error: St
}
}

```

/\*\*

\* Визуализации бывают и без SVG: матрицы из div-ячеек  
 \* префиксные суммы). Собираем такие «таблицы» в текст  
 \*/

```

function extractDivGrids(html: string): string[][] {
 const grids: string[][] = [];
 const rows: string[] = [];
 const rowMark = /<div class="[^"]*(?:flex gap-1\.5|gr
 let m: RegExpExecArray | null;
 while ((m = rowMark.exec(html))) {
 // сбалансированно находим конец строки-контейнера
 let depth = 1;
 let pos = m.index + m[0].length;
 while (depth > 0 && pos < html.length) {
 const no = html.indexOf("<div", pos);
 const nc = html.indexOf("</div>", pos);
 if (nc < 0) break;
 }
 }
}

```



```

.....
pages = 0;
chapterStart = new Map<string, number>();
constructor(stream: fs.WriteStream) {
 this.doc = new PDFDocument({ size: "A4", margins: {
 this.doc.pipe(stream);
 this.doc.on("pageAdded", () => this.pages++);
 this.setupFonts();
 this.bodyFont();
 }
 setupFonts() {
 if (F_REG) this.doc.registerFont("Body", F_REG);
 if (F_BOLD) this.doc.registerFont("BodyBold", F_BOLD);
 if (F_MONO) this.doc.registerFont("Mono", F_MONO);
 if (F_MONO) this.doc.registerFont("MonoBold", F_MONO);
 }
 bodyFont() { this.doc.font(F_REG ? "Body" : "Helvetica"); }
 boldFont() { this.doc.font(F_BOLD ? "BodyBold" : "Helvetica-Bold"); }
 monoFont() { this.doc.font(F_MONO ? "Mono" : "Courier"); }

 get y() { return this.doc.y; }
 set y(v: number) { this.doc.y = v; }
 get pageBottom() { return A4[1] - M.bottom; }
 ensureSpace(need: number) {
 if (this.y + need > this.pageBottom) this.doc.addPage();
 }
 footerAll(skipCover = true) {
 const range = this.doc.bufferedPageRange();
 const oldBottom = this.doc.page.margins.bottom;
 for (let p = range.start; p < range.start + range.count; p++) {
 if (skipCover && p === range.start) continue;
 this.doc.switchToPage(p);
 this.doc.page.margins.bottom = 0;
 this.doc.font(F_REG ? "Body" : "Helvetica").fontSize(12)
 .text(String(p + 1), 0, A4[1] - 30, { width: A4[1] });
 this.doc.page.margins.bottom = oldBottom;
 }
 }
}
}

```

```

};
// постранично
let from = 0;
let yTop = top;
while (from < lines.length) {
 const avail = d.pageBottom - yTop;
 let fit = Math.floor((avail - 8) / lead);
 if (fit <= 0 && from === 0 && lines.length > 1) { d
 fit = Math.max(fit, 1);
 const to = Math.min(lines.length, from + fit);
 drawPortion(from, to, yTop);
 from = to;
 if (from < lines.length) { d.doc.addPage(); yTop = 1
}
d.y = yTop + Math.min(lines.length, Math.floor((d.pag
d.doc.moveDown(3));
}

function drawTableBlock(d: Doc, rows: string[]) {
 const size = 8.4;
 d.ensureSpace(size * 3);
 const text = rows.map((c) => c.replace(/\s*\n\s*/g, "
 d.doc.fontSize(size).fillColor("#374151").text(text, 1
 d.doc.moveDown(2.2);
}

function drawGrid(d: Doc, rows: string[]) {
 const size = 8;
 const lead = size * 1.6;
 d.ensureSpace(rows.length * lead + 16);
 const top = d.y;
 d.monoFont().fontSize(size).fillColor("#111827");
 const cellW = Math.min(
 34,
 (CONTENT_W - 16) / Math.max(...rows.map((r) => r.sp
);
 rows.forEach((row, ri) => {
 const cells = row.split("\u2009");

```

```
.....
 }
 d.y = d.y + h + 18;
}

function drawImage(d: Doc, dataUrl: string) {
 const b64 = dataUrl.split(",")[1];
 if (!b64) return;
 const buf = Buffer.from(b64, "base64");
 const maxW = CONTENT_W * 0.82;
 const avail = d.pageBottom - d.y;
 if (avail < 200) d.doc.addPage();
 try {
 d.doc.image(buf, M.left + (CONTENT_W - maxW) / 2, d
 } catch { return; }
 d.y = d.y + 10 + maxW * 0.6; // примерная высота фото
 d.doc.moveDown(2);
}

// ----- сборка -----

const FRONT_TOC_LINES = 44; // строк содержания на стра

function tocEntries() {
 return chapters.map((c) => ({ id: c.id, title: c.titl
})

function tocPagesNeeded(): number {
 return Math.max(1, Math.ceil(tocEntries().length / FR
})

interface BuildOpts {
 outPath: string;
 record: boolean;
 /** id → стартовая страница (для второго прохода) */
 pageMap?: Map<string, number>;
 frontPages: number;
}
```

```

 doc.moveDown(0.8);
 doc.y = doc.y;
 d.y = doc.y;
 };
 drawToHeader();
 entries.forEach((ch, i) => {
 if (line > 0 && line % FRONT_TOC_LINES === 0) {
 doc.addPage();
 drawToHeader();
 }
 const pageNo2 = (opts.pageMap?.get(ch.id) ?? 0);
 const isTopic = /^\\d/.test(ch.title);
 doc.fontSize(isTopic ? 10.5 : 10).font(isTopic ?
 .fillColor(isTopic ? "#1f2937" : "#6b7280");
 // заголовок + точечная отбивка + номер страницы
 const title = ch.title.length > 86 ? ch.title.s
 const w = CONTENT_W - 40;
 const tw = doc.widthOfString(title);
 const pw = doc.widthOfString(String(pageNo2));
 doc.text(title, M.left, d.y, { width: w, lineBr
 const dots = ".".repeat(Math.max(3, Math.floor(
 doc.fillColor("#d1d5db").text(dots, M.left + tw
 doc.fillColor("#111827").text(String(pageNo2),
 d.y += 16.2;
 line++;
 }));
 // добиваем до расчётного числа страниц содержания
 const used = Math.ceil(entries.length / FRONT_TOC
 for (let p = used; p < opts.frontPages - 1; p++)
 doc.addPage();
 if (!opts.record) console.log(`TOC: ${entries.len
}

// — главы —
const stats: Stats = { chapterStart: new Map(), tota

chapters.forEach((ch, ci) => {
 const startPageOf = () => {

```

```

 drawTableBlock(d, b.rows);
 }
}

// визуализация главы
const vizEntry = VIZ_REGISTRY[ch.id];
if (vizEntry) {
 const art = renderVizArt(ch.id);
 if (opts.record) {
 const n = (art?.svgs.length ?? 0) + (art?.images.length ?? 0);
 console.log(` viz ${ch.id}: svg=${art?.svgs.length} images=${art?.images.length}
 for ? " ОШИБКА: " + art.error : ""`);
 if (n !== 0) console.log(` Δ ${ch.id}: визуализация`);
 }
 if (art && (art.svgs.length || art.images.length)) {
 d.ensureSpace(60);
 doc.moveDown(0.6);
 doc.fontSize(10).font(F_BOLD ? "BodyBold" : "Body");
 .text("■ Визуализация: " + vizEntry.title, 10, 0);
 d.y = doc.y + 6;
 let drawn = 0;
 for (const img of art.images) {
 if (drawn >= 4) break;
 drawImage(d, img);
 drawn++;
 }
 for (const svg of art.svgs) {
 if (drawn >= 5) break;
 drawSvg(d, svg);
 drawn++;
 }
 for (const grid of art.grids) {
 if (drawn >= 7) break;
 drawGrid(d, grid);
 drawn++;
 }
 } else if (art?.error) {
 // SSR упал – рисуем текстовую заглушку, PDF

```

Универсальное пособие • полный исходный код

```
.....
 console.log(`Страниц: ${pages}`);
}
```

```
main().catch((e) => {
 console.error(e);
 process.exit(1);
});
```

.....  
ФАЙЛ 90/96: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ: 1.5 КБ

```
.....
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
```

```
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";
```

```
const execFileAsync = promisify(execFile);
```

```
/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(/
 if (expanded.length <= PAGE.cols) return {expanded};
 const parts = [];
```

## Универсальное пособие • полный исходный код

```
.....
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 10));
let done = 0;
let cursor = 0;

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`отрендерено ${done} страниц\n`);
 }
 }
}
```

Универсальное пособие • полный исходный код

```
.....
for (const v of ["v, l, r", "m =", "ul, ur, add", "tree
 ok(seg.includes(v), `segment-trees init содержит «${v}
}
ok(!seg.includes("def build"), "segment-trees init не с

// 2. splay: числовой key + кадр i
const sp = init("splay-tree");
ok(sp.includes("key = 32") && sp.includes("keys = {50"}

// 3. КМП/Z: pattern/text/s попадают в init
const kmp = init("string-kmp");
ok(kmp.includes('pattern = "aba"') && kmp.includes('s =

// 4. Форду-Беллману — рёбра реального графа (S→A = 4,
const bf = PAGE_SYNC["bellman-ford"].code ?? "";
ok(bf.includes('("S", "A", 4)') && bf.includes('("D", "
ok(!bf.includes('("S", "A", 5)'), "bellman-ford: загрузи
ok(!/break/.test(bf), "bellman-ford: без break");

// 5. Джонсон: h(B) = -2, ребро -2, w' = 0
const jh = PAGE_SYNC["johnson-algo"].code ?? "";
ok(jh.includes('"B": -2') && jh.includes("w_new = w + h

// 6. Краскал/Прим/Борувка: 9 вершин и вес 51
for (const id of ["mst-kruskal", "mst-boruvka"]) {
 const c = PAGE_SYNC[id].code ?? "";
 ok(/ABCDEFGHI/.test(c), `${id}: 9 вершин A..I`);
}
ok((PAGE_SYNC["mst-prima"].code ?? "").match(/"I":/g)?
ok((PAGE_SYNC["mst-kruskal"].code ?? "").includes('{13,

// 7. prefix-sums-2d#2d: пометка 1-индексных клеток
const p2 = PAGE_SYNC["prefix-sums-2d#2d"].code ?? "";
ok(p2.includes("клетки 1-индексные"), "prefix#2d: комме

// 8. графы top-sort/scc/graph-dfs-bfs/graph-components
ok((PAGE_SYNC["top-sort"].code ?? "").includes("0: {1,
ok((PAGE_SYNC["scc-kosaraju"].code ?? "").includes("2:
```



Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 3 | РАЗМЕР: 1.1 КБ

```
export { PAGE_SYNC } from "../../src/data/vizSync";
export { buildDebugRunner, DEBUG_STEP_CAP } from "../../src/debug";
```

ФАЙЛ 94/96: scripts/selftest/lazy.ts

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 42 | РАЗМЕР: 2.1 КБ

```
// Самотест lazy-вкладки дерева отрезков: пять массовых
// Корень после обновления и query(uL, uR) = base(uL, uR)
import { genLazySteps, buildTreeFull } from "../../src/algorithm";
```

```
const base = [5, 8, 3, 12, 7, 2];
const sum = (l: number, r: number) => base.slice(l, r + 1).reduce((a, b) => a + b, 0);
```

```
const cases: [number, number, number][][] = [
 [1, 3, 10], [0, 5, 1], [2, 2, 7], [0, 5, 100], [4, 5, 100],
];
```

```
// query по дереву с обещаниями: pending накапливается
function q(step: { treeState: Record<number, number>; lazyState: Record<number, number>; pending: number; }, v: number, l: number, r: number, ql: number, qr: number): number {
 if (qr < l || r < ql) return 0;
 // Собственный lazy[v] уже учтён в tree[v] (обещание)
 // добавляем только накопленные обещания предков (pending)
 if (ql <= l && r <= qr) return (step.treeState[v] ?? 0) + step.pending;
 const m = (l + r) >> 1;
 const d = pending + (step.lazyState[v] ?? 0);
 return q(step, 2 * v, l, m, ql, qr, d) + q(step, 2 * v + 1, m, r, ql, qr, d);
}
```

```
let fails = 0;
for (const [uL, uR, add] of cases) {
 const steps = genLazySteps(base, uL, uR, add);
 const result = buildTreeFull(steps);
 const expected = sum(uL, uR);
 if (result !== expected) {
 fails++;
 console.log(`FAIL: ${uL}, ${uR}, ${add}, ${result}, ${expected}`);
 }
}
```

## Универсальное пособие • полный исходный код

```
.....
on:
 push:
 branches:
 - main
 # Позволяет проверить Pages прямо из рабочей ветки
 - "arena/**"
 workflow_dispatch:

permissions:
 contents: read
 pages: write
 id-token: write

concurrency:
 group: pages
 cancel-in-progress: true

jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build
```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 137 | РАЗМЕР: 5.1

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
#
→ page-0001.png ... page-NNNN.png (
#
→ full_code_all_pages.pdf
#
→ guide_all_topics.pdf
PDF-конспект: текст всех тем + с
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умол

required: false

default: "150"

concurrency:

group: rebuild-pdf-\${{ github.ref }}

cancel-in-progress: true

## Универсальное пособие • полный исходный код

- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
  EXPORT\_DENSITY: \${github.event.inputs.density}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Step 4 – PDF-конспект (текст тем + визуал)  
run: npm run export:guide
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
  {  
    echo "### Экспорт пересобран"  
    echo ""  
    echo "| Артефакт | Размер |"  
    echo "| --- | --- |"  
    echo "| \public/export/full\_code\_all\_pages  
    echo "| \public/export/full\_code\_all\_pages  
    echo "| \public/export/guide\_all\_topics.pdf  
    echo "Страниц в конспекте: \$(node -e "console.log(require('fs').readdirSync('tmp/export-pages').length)"  
    echo "| PNG-страниц | \$(ls tmp/export-pages.  
  } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
  name: full-code-export  
  path: |